2º curso / 2º cuatr.

Grados en
Ing. Informática

Arquitectura de Computadores
Tema 3

Arquitecturas con paralelismo a nivel de thread (TLP)

Material elaborado por Mancia Anguita

Profesores: Mancia Anguita, Maribel García y Christian Morillas



UNIVERSIDAD DE GRANADA



ICAR
INGENIERÍA DE COMPUTADORES, AUTOMÁTICA Y ROBÓTICA

Lecciones

- Lección 7. Arquitecturas TLP
  - Clasificación de arquitecturas con TLP explícito y una instancia del SO
  - Multiprocesadores y multicores
  - Cores Multithread
  - Hardware y arquitecturas TLP en un chip
- Lección 8. Coherencia del sistema de memoria
- Lección 9. Consistencia del sistema de memoria
- Lección 10. Sincronización

Objetivos Lección 7

- Distinguir entre cores multhread, multicores y multiprocesadores.
- Comparar entre cores multithread de grano fino, cores multithread de grano grueso y cores con multithread simultánea.

Bibliografía Lección 7

- Fundamental
  - T. Rauber, G. Ründer. *Parallel Programming: for Multicore and Cluster Systems*. Springer 2023. Disponible en línea (biblioteca UGR): https://link.springer.com/book/10.1007/978-3-031-28924-8
  - Theo Ungerer, Borut Robic, Jurij Silc, “A survey of Processors with Explicit Multithreading”, ACM Computing Surveys, vol. 35, no. 1, pp. 29-63, March 2003. Disponible en línea (biblioteca UGR): http://dl.acm.org/citation.cfm?id=641867
  - Capítulo 3, Sección 2. M. Anguita, J. Ortega. Fundamentos y problemas de Arquitectura de Computadores, Editorial Técnica Avicam. ESIIT/C.1 ANG fun

Contenido Lección 7

- Clasificación de arquitecturas con TLP explícito y una instancia del SO
- Multiprocesadores y multicores
- Cores Multithread
- Hardware y arquitecturas TLP en un chip

Arquitecturas con DLP, ILP y TLP (thread=flujo de control o de instrucciones)

Arq. con **DLP** (*Data Level Parallelism*)

Ejecutan las ***operaciones*** de una instrucción **concurr.** o en **paralelo**

Unidades funcionales vectoriales o SIMD (90)

Arq. con **ILP** (*Instruction Level Parallelism*)

Temas[1,4], BP4

Ejecutan múltiples ***instrucciones*** **concurr.** o en **paralelo**

Cores escalares (60) segmentados (80), superescalares (90) o VLIW/EPIC (90)

Arq. con **TLP** (*Thread Level Parallelism*) explícito y **una** instancia de SO

Temas[1-3], BP[0-3]

Ejecutan múltiples ***flujos de instrucciones*** **concurr.** o en **paralelo**

Cores que modifican la arquit. ILP para ejecutar threads *concurr*. o en *paralelo* (se extendieron en 2000)

Multi-procesadores (60): ejecutan threads en *paralelo* en un computador con múltiples cores (incluye **multicores (2000)**)

Arq. con **TLP** explícito y **múltiples** instancias SO

IC.SCAP

Ejec. múltiples ***flujos de instr.*** en **paralelo**

Multi-computadores (85): ejecutan threads en *paralelo* en un sistema con múltiples computadores

Contenido Lección 7

- Clasificación de arquitecturas con TLP explícito y una instancia del SO
- Multiprocesadores y multicores
  - Ejecutan varios flujos de instrucciones (*threads*) en *paralelo* en un **computador** con varios cores/procesadores (cada thread en un core/procesador distinto).
- Cores Multithread
- Hardware y arquitecturas TLP en un chip

Multiprocesadores. Criterio de clasificación: nivel de empaquet./conexión

Sistema

Armario (*cabinet*)

Placa (*board*)

Encapsulado (package)

chip



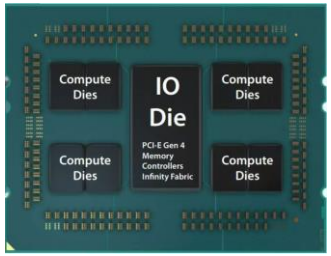
Armario HPE Superdome Flex con 8 servidores con 4 sockets cada uno

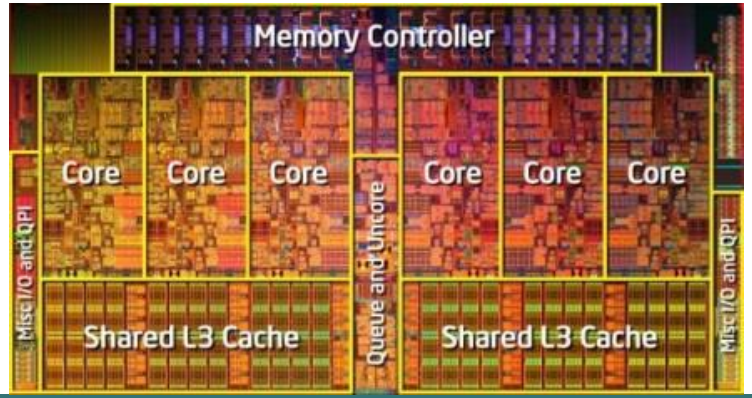


Placa de servidor HPE Superdome Flex (4 sockets)



MCM (cores + Red) AMD EPYC

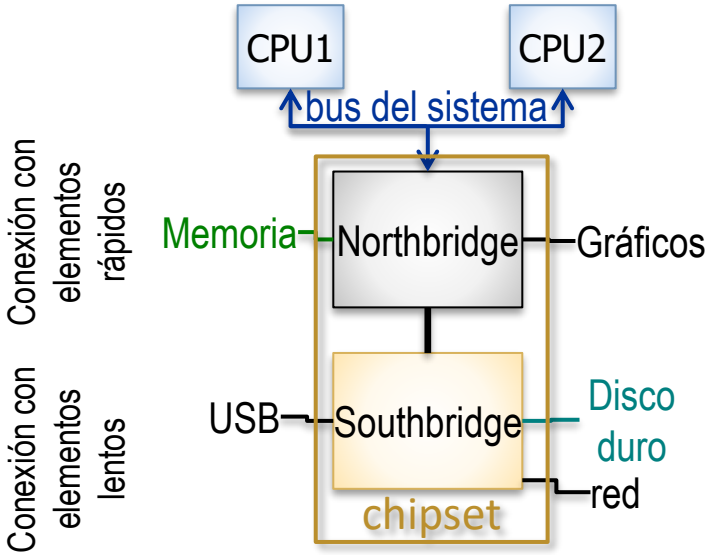


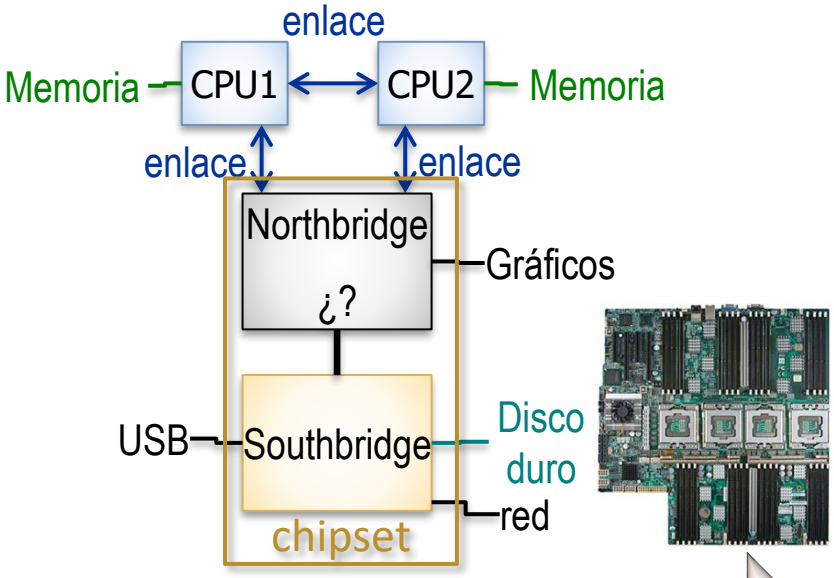


Multicore

https://www.hpe.com/es/es/servers/superdome.html

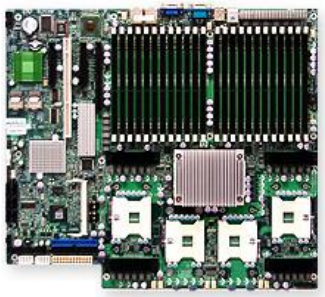
Multiprocesador en una placa: evolución de UMA a NUMA





UMA → **NUMA**

- Controlador de memoria en **chipset** (*Northbrigde* chip)
- Red: bus (medio compartido)



- Controlador de memoria en chip del procesador
- Red: enlaces (conexiones punto a punto) y conmutadores (en el chip del procesador)
- Ejemplos en servidores:
  - AMD Opteron (2003): enlaces HyperTransport (2001)
  - Intel (Nehalem) Xeon 7500 (2010): enlaces QPI (*Quick Path Interconnect*, 2008)

Multiprocesador en encapsulado (Módulo multichip) – SoC (System on a Chip)

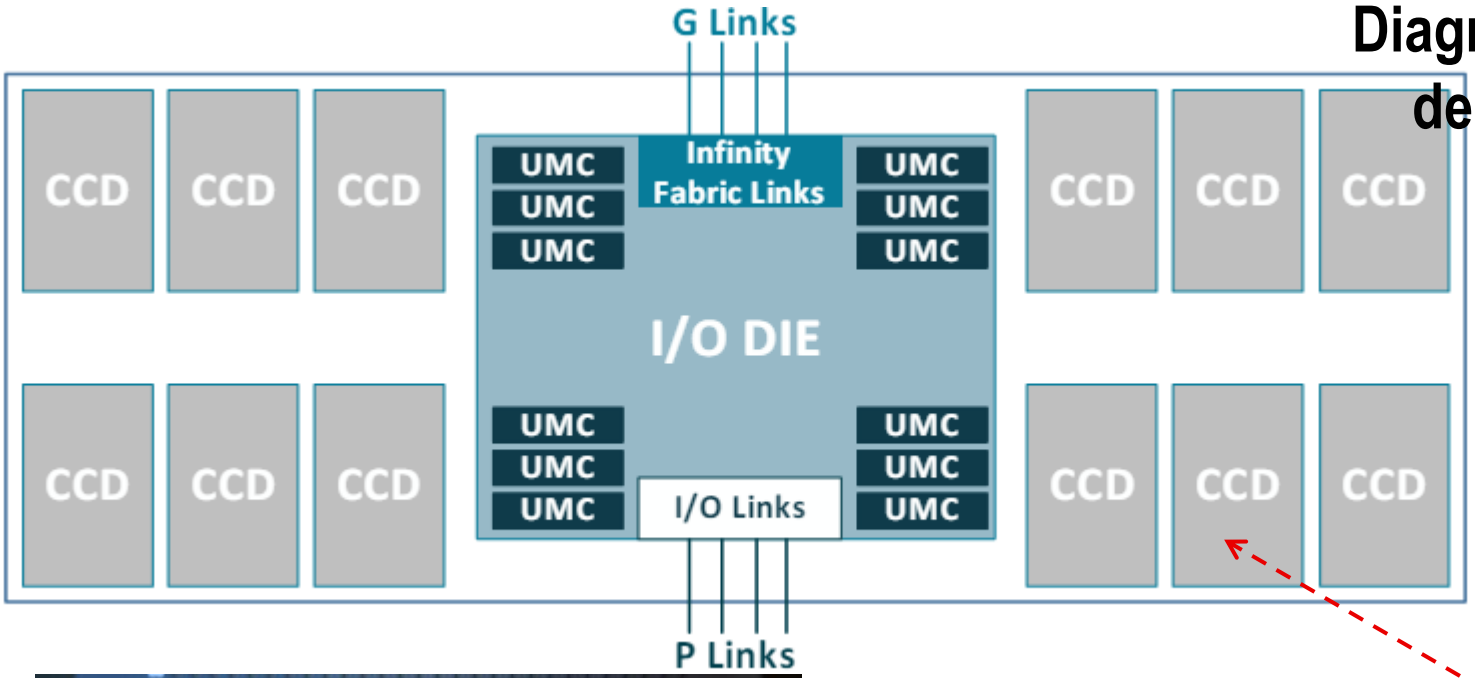
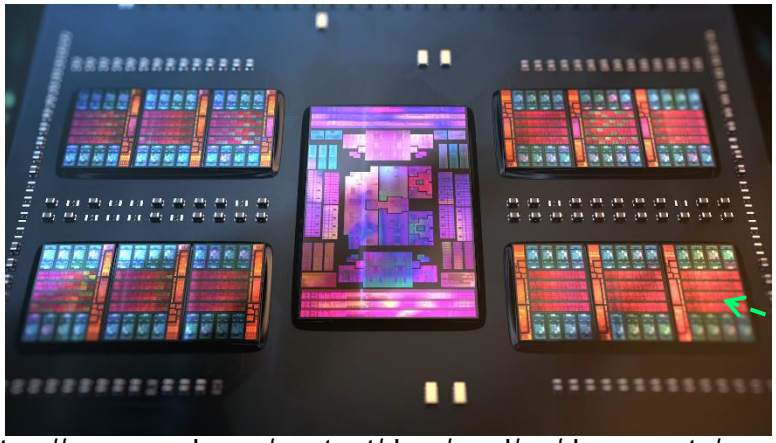


Diagrama de bloques del encapsulado

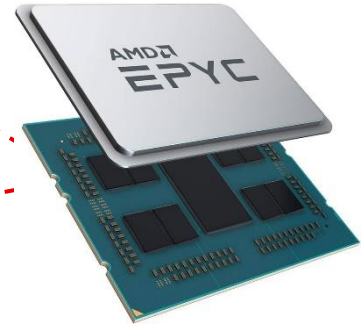
CCD: Core Complex Die

UMC: controladores de memoria



5 dados

CCD con 8 núcleos cada uno

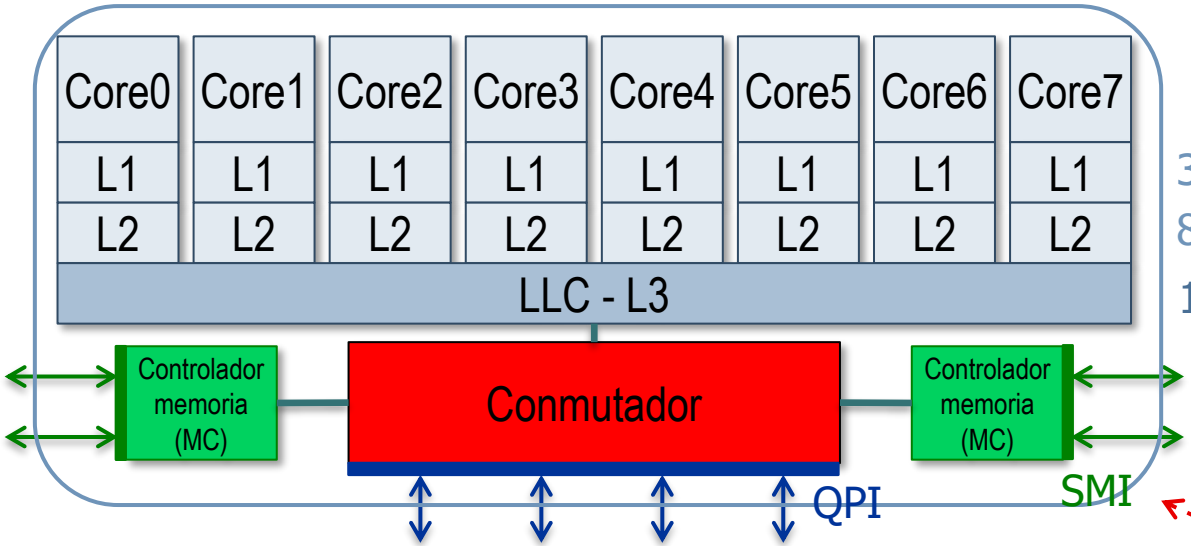


Encapsulado AMD EPYC

https://www.amd.com/content/dam/amd/en/documents/epyc-technical-docs/white-papers/58015-epyc-9004-tg-architecture-overview.pdf

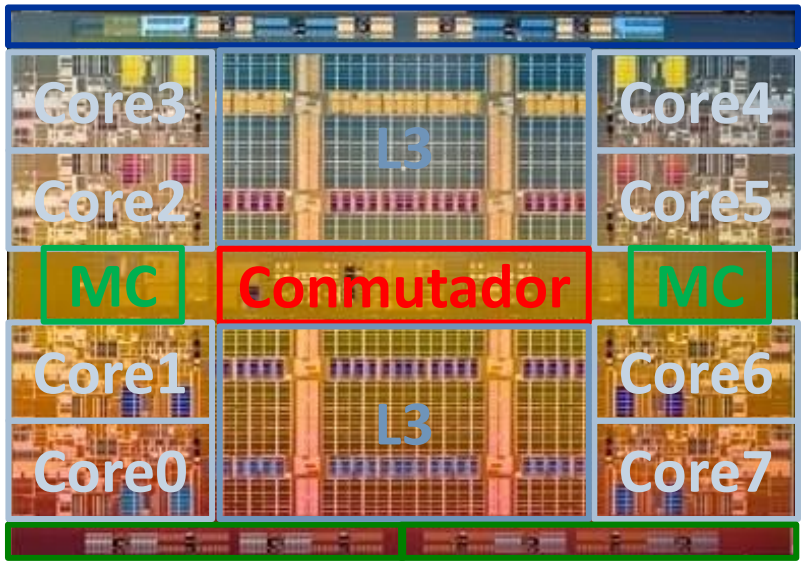
Multiprocesador en un chip o Multicore o CMP (*Chip MultiProcessor*) (2000)

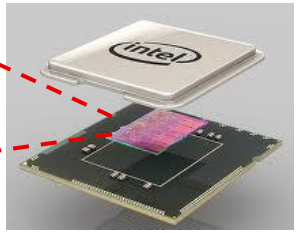
Intel Xeon 7500



32 KB Icaché + 32KB Dcaché
8x256 KB
12 a 24 MB

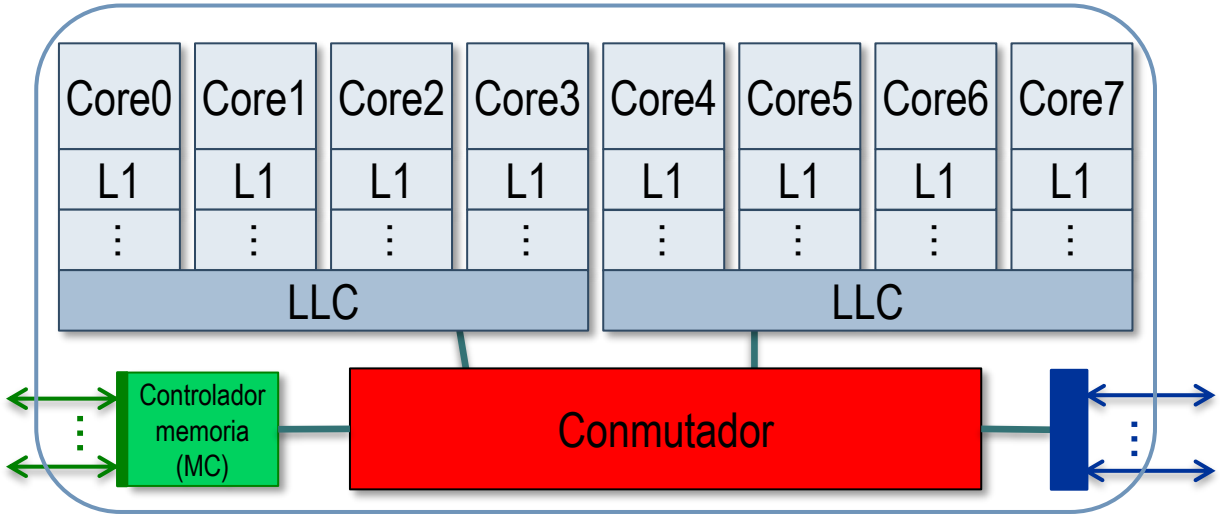
Diagrama de bloques del dado

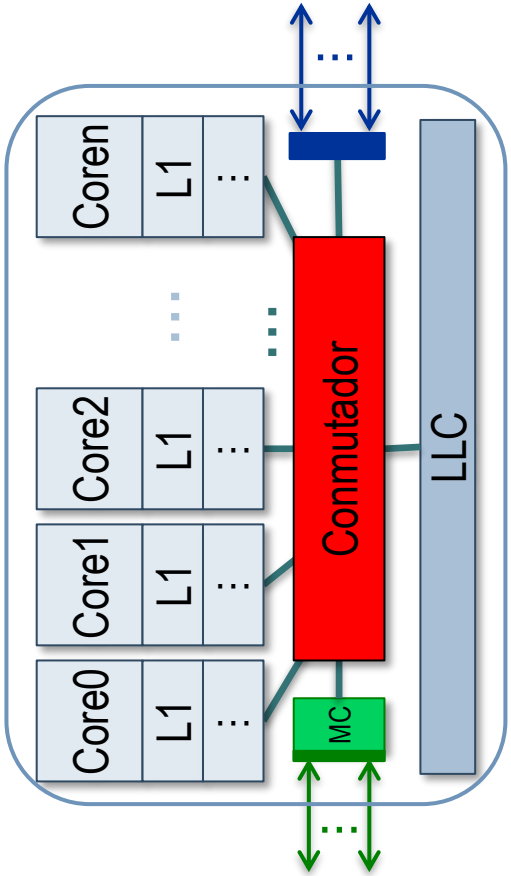


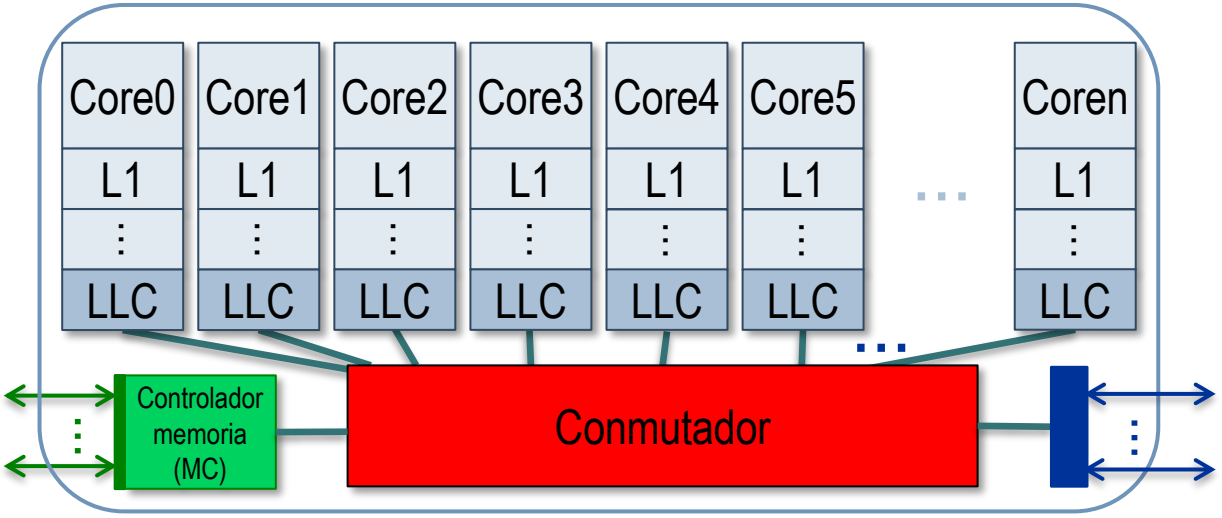


Chip o dado de silicio del chip de procesamiento Intel Xeon 7500

Multicore: otras posibles estructuras







Link Controller (LC)

Contenido Lección 7

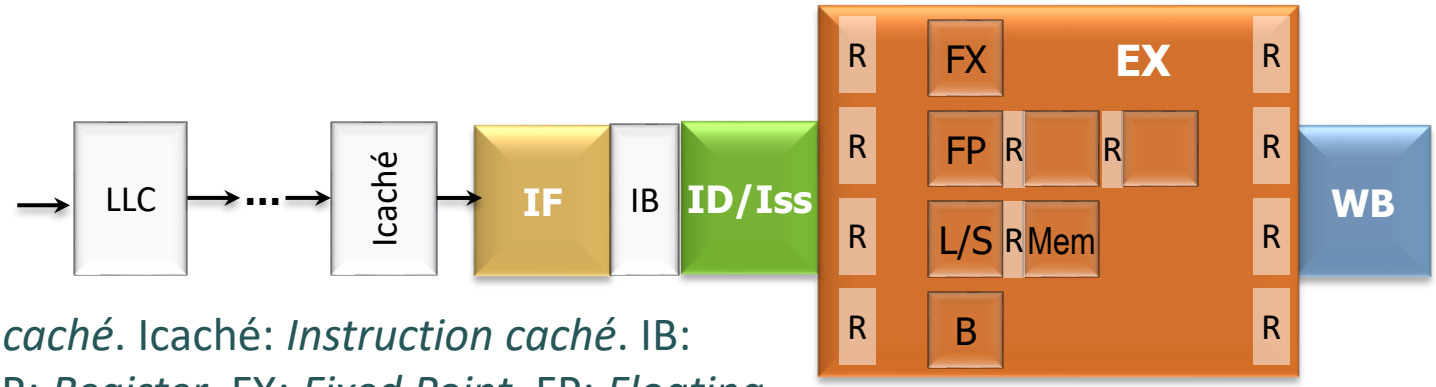
- Clasificación y estructura de arquitecturas con TLP explícito y una instancia del SO
- Multiprocesadores y multicores
- Cores Multithread (1975, extensión en 2000)
  - Modifican su arquitectura ILP (segmentada, superescalar o VLIW) para ejecutar threads concurrentemente o en paralelo
- Hardware y arquitecturas TLP en un chip

Arquitecturas ILP

Escalar segmentada



VLIW y superescalar

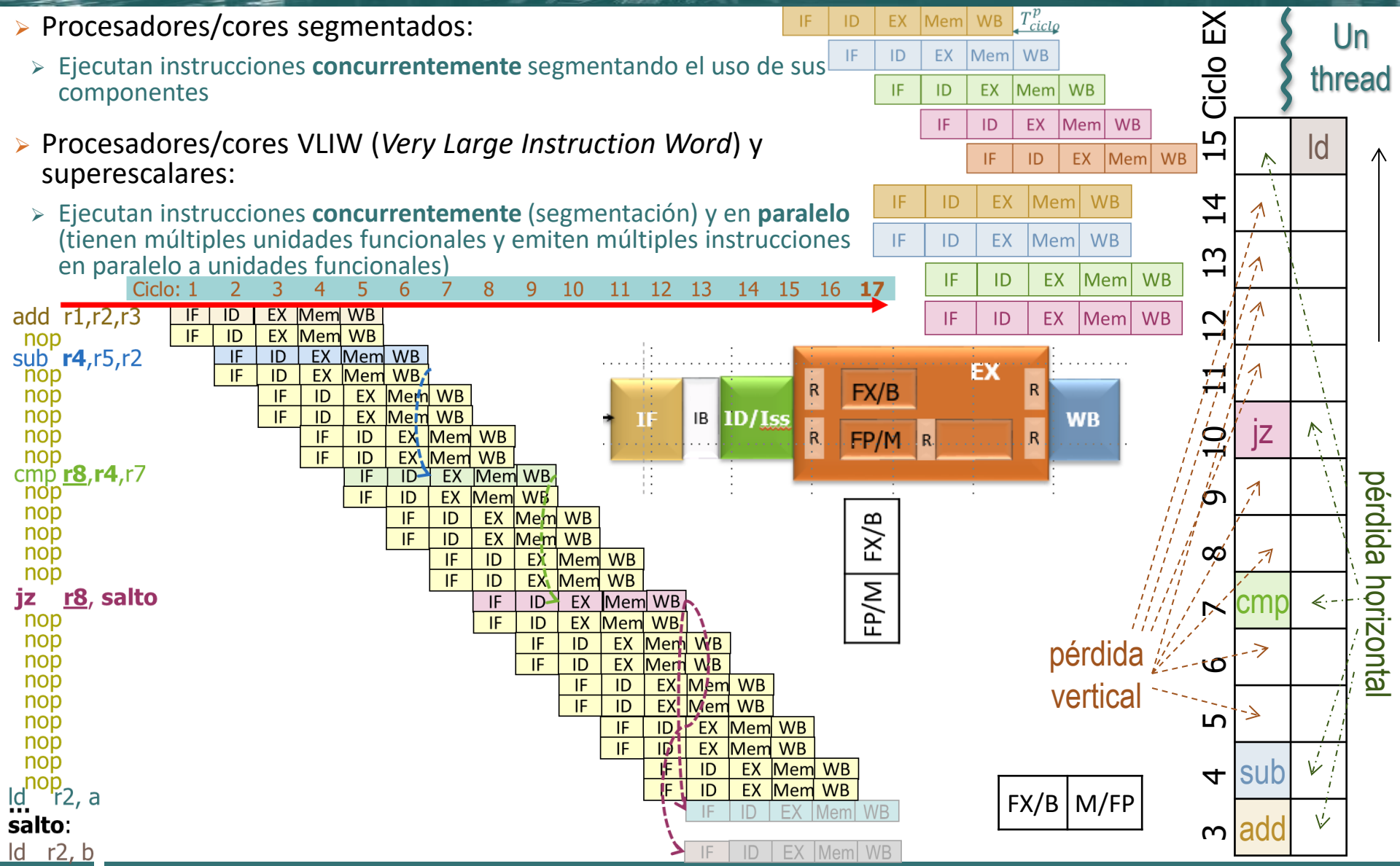


LLC: *Last Level caché*. Icaché: *Instruction caché*. IB: *Instruction Buffer*. R: *Register*. FX: *Fixed Point*. FP: *Floating Point*. L/S: (*memory*) *Load/Store*. B: *Branch*

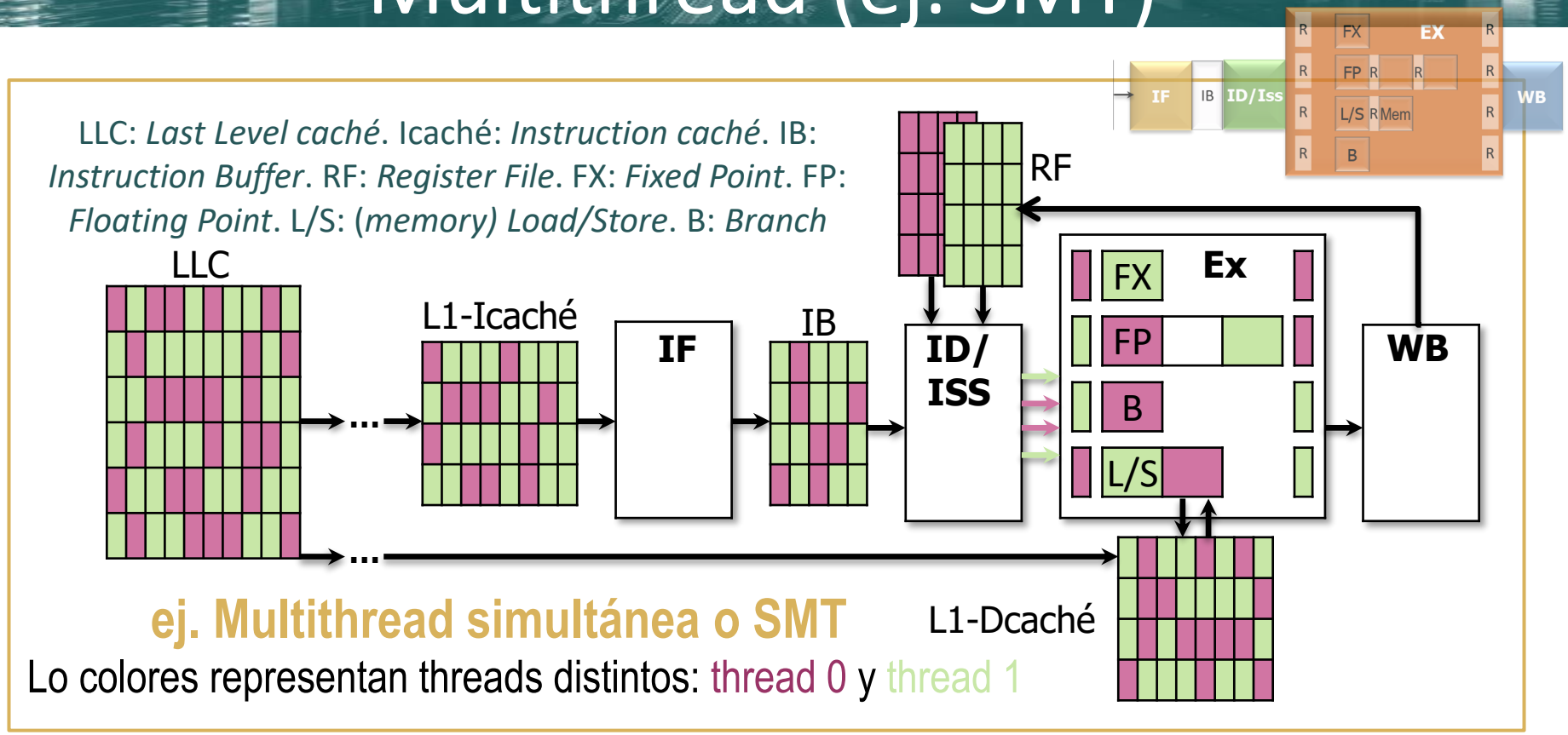
- Etapa de captación de instrucciones (***I**nstruction **F**etch*)
- Etapa de decodificación de instrucciones y emisión a unidades funcionales (***I**nstruction **D**ecode/Instruction **Iss**ue)* (incluye captura de "operandos" de los registros de la arquitectura)
- Etapas de ejecución (***Ex**ecution*). Etapa de acceso a memoria (***Mem**ory*)
- Etapa de almacenamiento de resultados en registros de la arquitectura(***W**rite-**B**ack*)

Cores ILP y motivación cores TLP

- Procesadores/cores segmentados:
  - Ejecutan instrucciones **concurrentemente** segmentando el uso de sus componentes
- Procesadores/cores VLIW (*Very Large Instruction Word*) y superescalares:
  - Ejecutan instrucciones **concurrentemente** (segmentación) y en **paralelo** (tienen múltiples unidades funcionales y emiten múltiples instrucciones en paralelo a unidades funcionales)

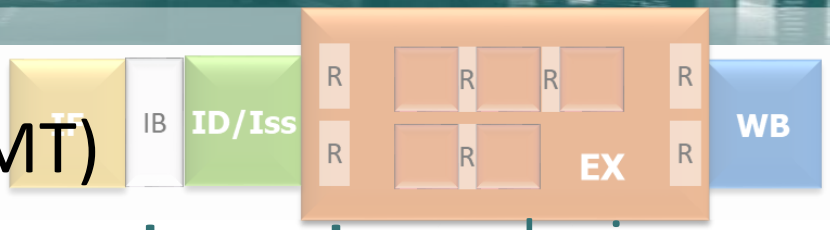


Modificación de la arquitectura ILP en Core Multithread (ej. SMT)

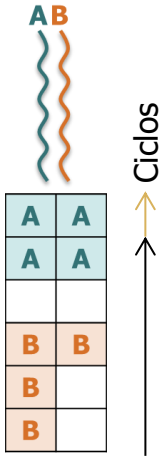


- Almacenamiento: se multiplexa, se reparte o comparte entre threads, o se replica
  - Con SMT: repartir, compartir o replicar
- Hardware dentro de etapas: se multiplexa, o se reparte o comparte entre threads
  - Con SMT: unidades funcionales (etapa Ex) compartidas, resto etapas repartidas o compartidas; multiplexación es posible (p. ej. predicción de saltos y decodificación)

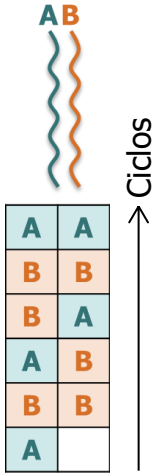
Clasificación de cores multithread



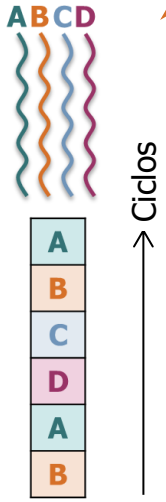
- *Temporal Multithreading* (TMT)
  - Ejecutan varios threads **concurrentemente** en el mismo core
    - La conmutación entre threads la decide y controla el hardware
  - Emite instrucciones de **un único thread** en un ciclo



- *Simultaneous MultiThreading* (SMT) o multihilo simultáneo o *horizontal multithread*
  - Ejecutan, en un core superescalar, varios threads en **paralelo**
  - Pueden emitir (para su ejecución) instrucciones de **varios threads** en un ciclo
  - No implementa conmutación entre threads



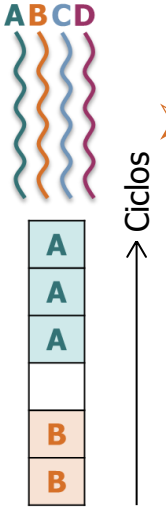
Clasificación de cores con TMT



- *Fine-grain multithreading* (FGMT) o *interleaved multithreading*



  - La conmutación entre threads la decide el hardware **cada ciclo** (coste 0)
    - por turno rotatorio (*round-robin*) o
    - por eventos de cierta latencia combinado con alguna técnica de planificación (ej. thread menos recientemente ejecutado)
      - Eventos: dependencia funcional, acceso a datos a caché L1, salto no predecible, una operación de cierta latencia (ej. div), …



- *Coarse-grain multithreading* (CGMT) o *blocked multithreading*
  - La conmutación entre threads la decide el hardware (coste de 0 a varios ciclos)
    - tras intervalos de tiempo prefijados (*timeslice multithreading*) o
    - por eventos de cierta latencia (*switch-on-event multithreading)*.

Clasificación de cores con CGMT con conmutación por eventos

- Estática:
  - Conmutación
    - Explícita: instrucciones explícitas para conmutación (instrucciones añadidas al repertorio)
    - Implícita: instrucciones de carga, almacenamiento, salto
  - Ventaja/Inconveniente:
    - coste cambio contexto bajo (0 o 1 ciclo) / cambios de contextos innecesarios
- Dinámica:
  - Conmutación típicamente por:
    - fallo en la última caché dentro del chip de procesamiento (conmutación por fallo de caché), interrupción (conmutación por señal), …
  - Ventaja/Inconveniente:
    - reduce cambios de contexto innecesarios / mayor sobrecarga al cambiar de contexto

Alternativas en un core escalar segmentado



- En un core escalar se emite una instrucción cada ciclo de reloj

Un thread (no TLP)

Multithread grano fino

Multithread grano grueso

Dinámico

Estático

Orden D-C-B-A

```
      ...
1  load r1,a
2  add  r1,r2
3  mul  r3,r4
      ...
```

ILP

Ciclos

pérdida vertical

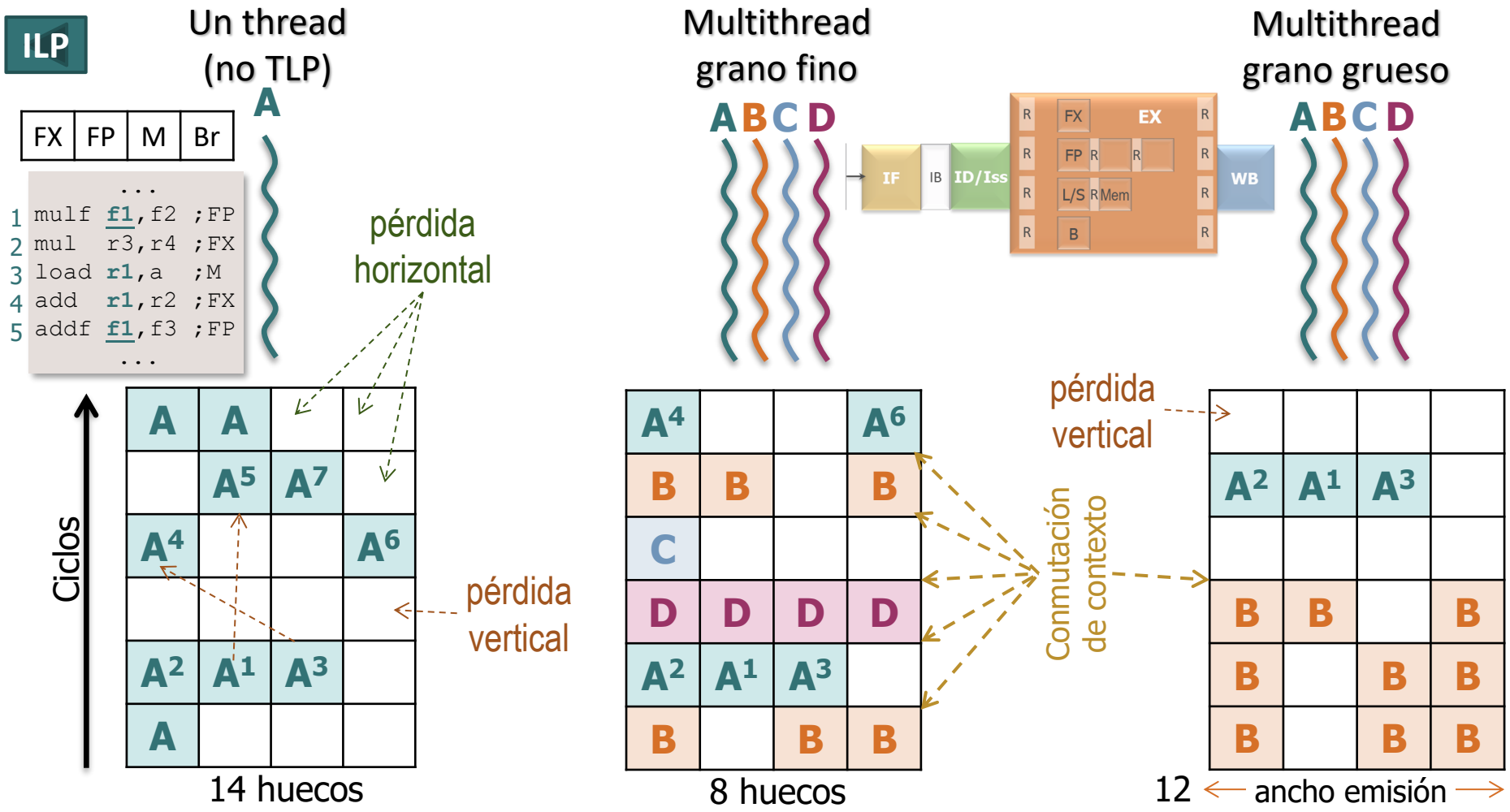
Conmutación de contexto

pérdida vertical

Por instrucción que dispara cambio de contexto

Alternativas en un core con emisión múltiple de instrucciones de un thread

- En un core superescalar o VLIW se emiten más de una instrucción cada ciclo de reloj; en las alternativas de está página, de un único thread



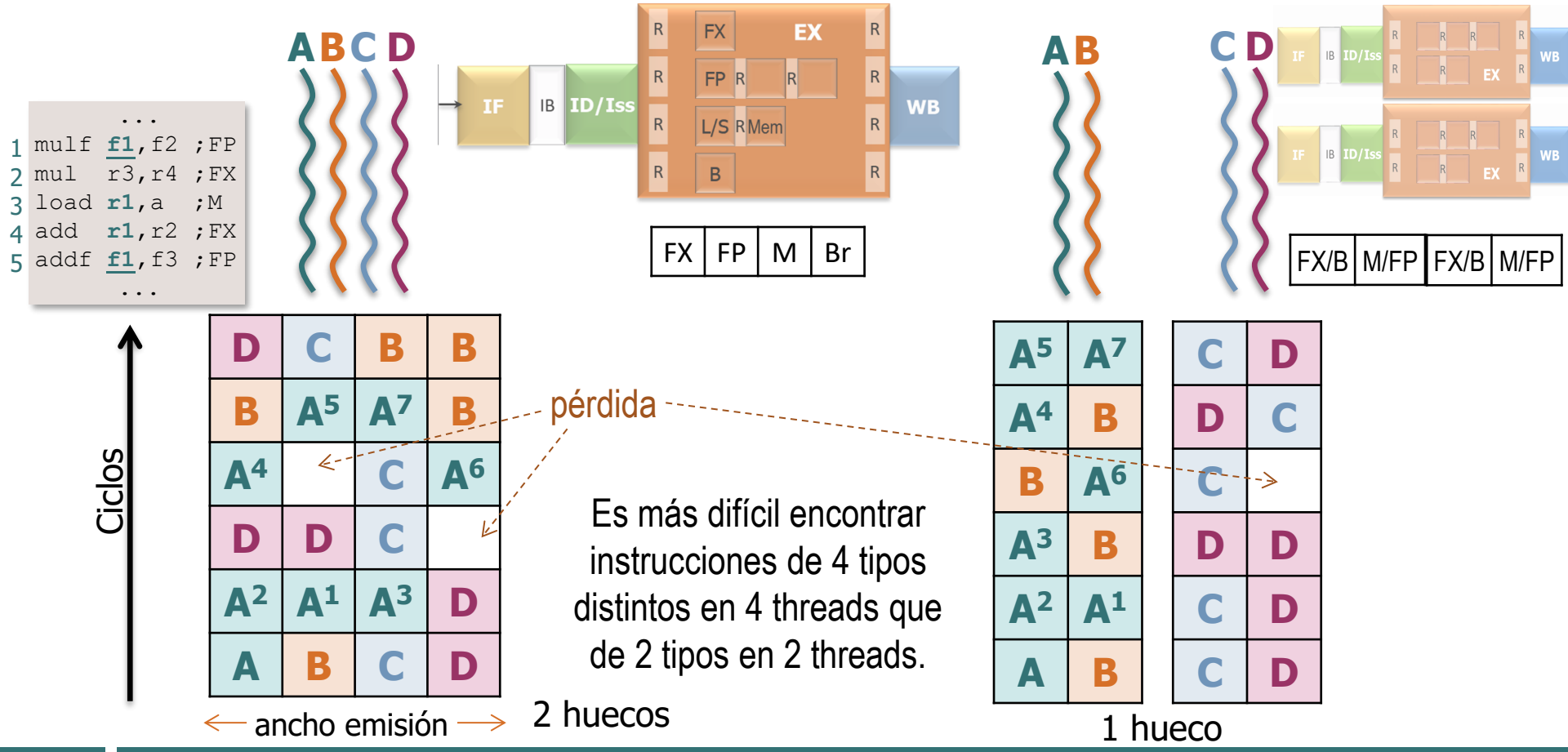
Core multithread simultánea y multicores

- En un multicore y en un core superescalar con SMT (*Simultaneous MultiThread*) se pueden emitir a ejecución instrucciones de distintos threads cada ciclo de reloj

ILP

Core con multithread simultánea (4 thread por core)

Multicore con 2 cores con multithread simultánea (2 threads por core)

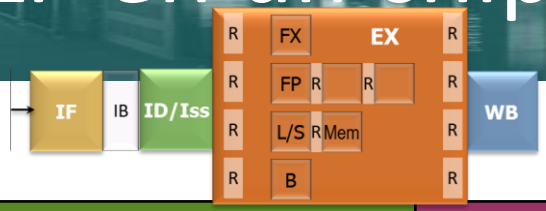


Contenido Lección 7

- Clasificación y estructura de arquitecturas con TLP explícito y una instancia del SO
- Multiprocesadores y multicores
- Cores Multithread
- Hardware y arquitecturas TLP en un chip

Hardware y arquitecturas TLP en un chip

Multiplexar Repartir Compartir Replicar



Hardware	*CGMT*	*FGMT*	*SMT*	*CMP*
Registros de la arquitectura	**replicado** (al menos PC)	**replicado**	**replicado**	**replicado**
Almacenamiento	multiplexado	multiplexado, repartido, compartido o replicado	repartido, compartido o replicado	**replicado**
Otro hardware de las etapas del cauce	multiplexado	Captación: repartida o compartida; **Resto: multiplexadas**	**UF: compartidas**; Resto: repartidas o compartidas	**replicado**
Etiquetas para distinguir el thread de una instr.	Sí	Sí	Sí	**No**
Hardware para conmutar entre threads	Sí	Sí	**No**	**No**

Para ampliar …

- Webs
  - An Introduction to the Intel® QuickPath Interconnect, http://www.intel.com/content/www/us/en/io/quickpath-technology/quick-path-interconnect-introduction-paper.html
  - Intel® QuickPath Technology Animated Demo [119 K] https://www.youtube.com/watch?v=XPBX3cl_clE
- Artículos en revistas
  - Kongetira, P.; Aingaran, K.; Olukotun, K.; , "Niagara: a 32-way multithreaded Sparc processor," *Micro, IEEE* , vol.25, no.2, pp. 21-29, March-April 2005. Disponible en línea (biblioteca UGR): http://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=1453485&isnumber=31213

2º curso / 2º cuatr.

Grados en
Ing. Informática

Arquitectura de Computadores
Tema 3

Lección 8. Coherencia

Material elaborado por Mancia Anguita
Profesores: Mancia Anguita, Maribel García y Christian Morillas



UNIVERSIDAD
DE GRANADA



ICAR
INGENIERÍA DE COMPUTADORES,
AUTOMÁTICA Y ROBÓTICA

Lecciones

- Lección 7. Arquitecturas TLP
- Lección 8. Coherencia del sistema de memoria
  - Sistema de memoria en multiprocesadores
  - Concepto de coherencia en el sistema de memoria. Situaciones de incoherencia.
  - Protocolos de mantenimiento de coherencia: clasificación y diseño
  - Protocolo MSI de espionaje
  - Protocolo MESI de espionaje
  - Concepto de coherencia en el sistema de memoria. Requisitos para evitar problemas por incoherencia.
  - Protocolo MSI basado en directorios con o sin difusión
- Lección 9. Consistencia del sistema de memoria
- Lección 10. Sincronización

Objetivos Lección 8

- Comparar los métodos de actualización de memoria principal implementados en caché.
- Comparar las alternativas para propagar un escritura en protocolos de coherencia de caché.
- Explicar qué debe garantizar el sistema de memoria para evitar problemas por incoherencias.
- Describir las partes en las que se puede dividir el análisis o el diseño de protocolos de coherencia.
- Distinguir entre protocolos basados en directorios y protocolos de espionaje (snoopy).
- Explicar el protocolo de mantenimiento de coherencia de espionaje MSI.
- Explicar el protocolo de mantenimiento de coherencia de espionaje MESI.
- Explicar el protocolo de mantenimiento de coherencia MSI basado en directorios con difusión y sin difusión.

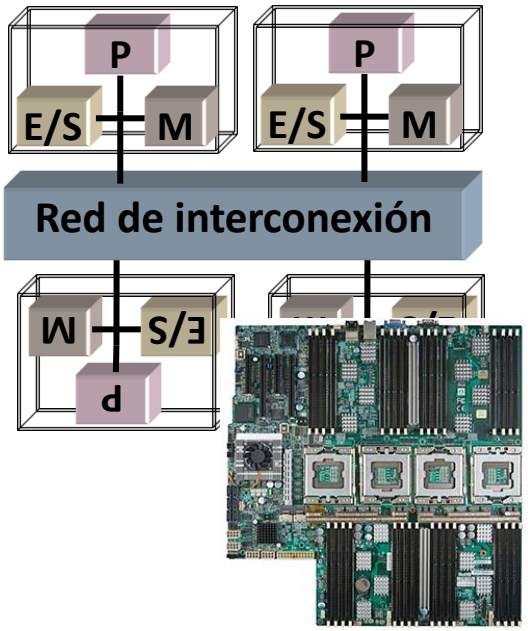
Bibliografía Lección 8

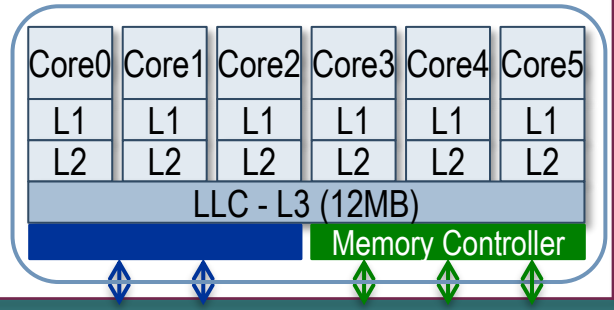
- Fundamental
  - Cap. 3, Secc. 3. M. Anguita, J. Ortega. *Fundamentos y Problemas de Arquitectura de Computadores.* Librería Fleming/Ed. Avicam, 2016.
  - Secc. 10.1. J. Ortega, M. Anguita, A. Prieto. *Arquitectura de Computadores*. Thomson, 2005. ESII/C.1 ORT arq
- Complementaria
  - T. Rauber, G. Ründer. *Parallel Programming: for Multicore and Cluster Systems*. Springer 2010. Disponible en línea (biblioteca UGR): http://dx.doi.org/10.1007/978-3-642-04818-0

Computadores que implementan en hardware mantenimiento de coherencia

Multi-computadores Memoria no compartida	**NORMA** *No Remote Memory Access*	nivel de sistema (*Cluster*), armario, chasis (*blade server*)	Memoria físicamente distribuida
Multi-procesadores Memoria compartida Un único espacio de direcciones	**NUMA** *Non-Uniform Memory Access*	**NUMA** (nivel de sistema, n. armario/chasis, n. placa)	
		CC-NUMA (nivel de armario: SGI Altix; nivel de placa)	
		COMA	
	UMA *Uniform Memory Access*	**SMP** *Symmetric MultiProcessor* (nivel de placa; nivel de chip: multicores como Intel Core i7, i5, i3)	Memoria físicamente centralizada

Coherencia por hardware





Escalabilidad + −

Contenido Lección 8

- Sistema de memoria en multiprocesadores
- Concepto de coherencia en el sistema de memoria. Situaciones de incoherencia.
- Protocolos de mantenimiento de coherencia: clasificación y diseño
- Protocolo MSI de espionaje
- Protocolo MESI de espionaje
- Concepto de coherencia en el sistema de memoria. Requisitos para evitar problemas por incoherencia.
- Protocolo MSI basado en directorios con o sin difusión

Sistema de memoria en multiprocesadores

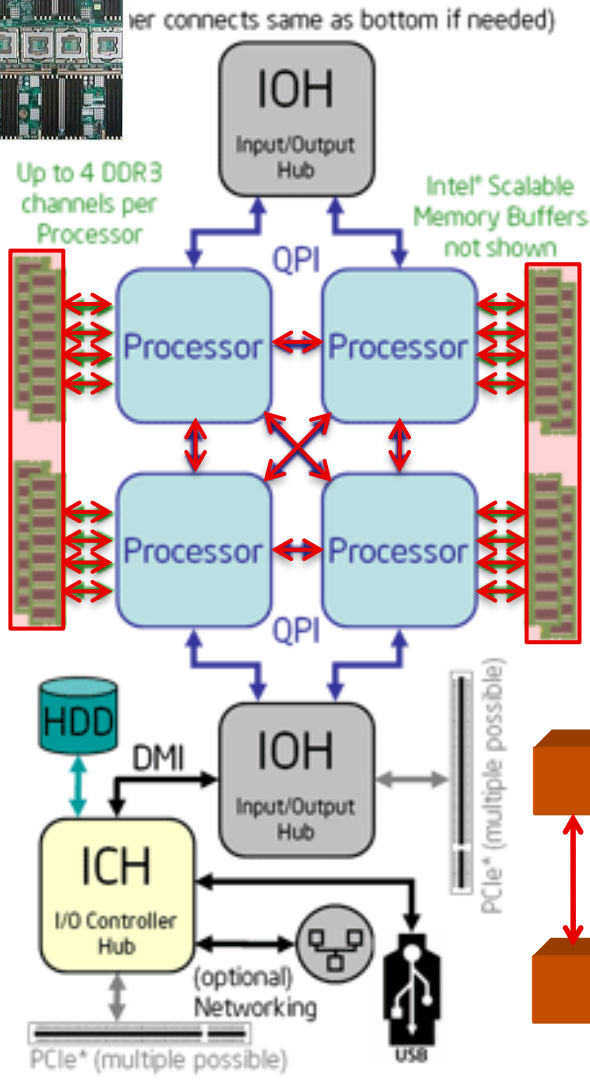
- ¿Qué incluye?
  - cachés de todos los nodos
  - Memoria principal
  - Controladores
  - Buffers:
    - Buffer de escritura/almacenamiento
    - Buffer que combinan escrituras/almacenamientos, etc.
  - Medio de comunicación de todos estos componentes (red de interconexión)
- La comunicación de datos entre procesadores la realiza el sistema de memoria

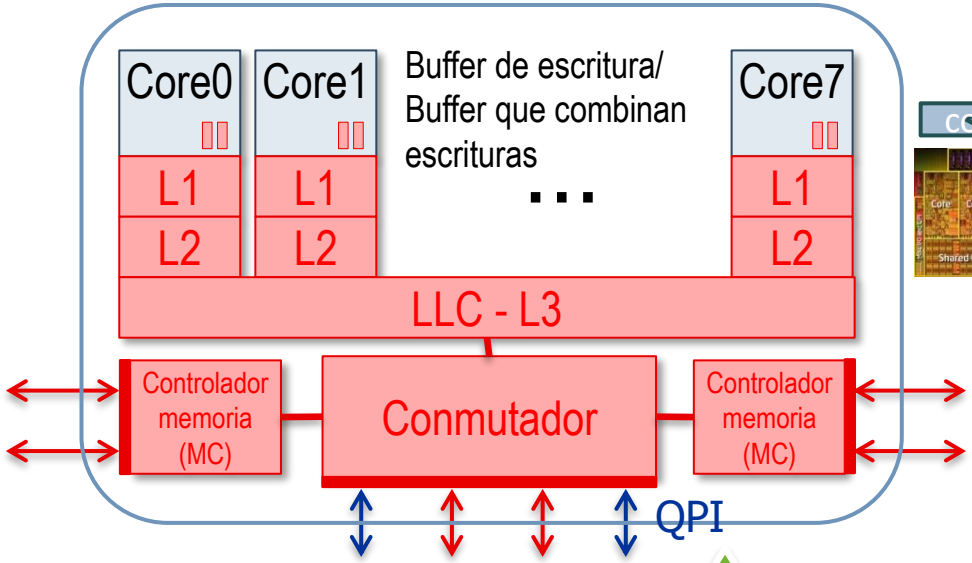
comunicación

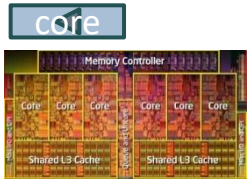
  - La lectura de una dirección debe devolver lo último que se ha escrito (desde el punto de vista de todos los componentes del sistema)

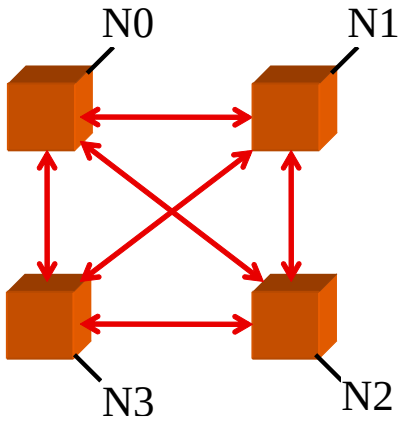
Sistema de memoria

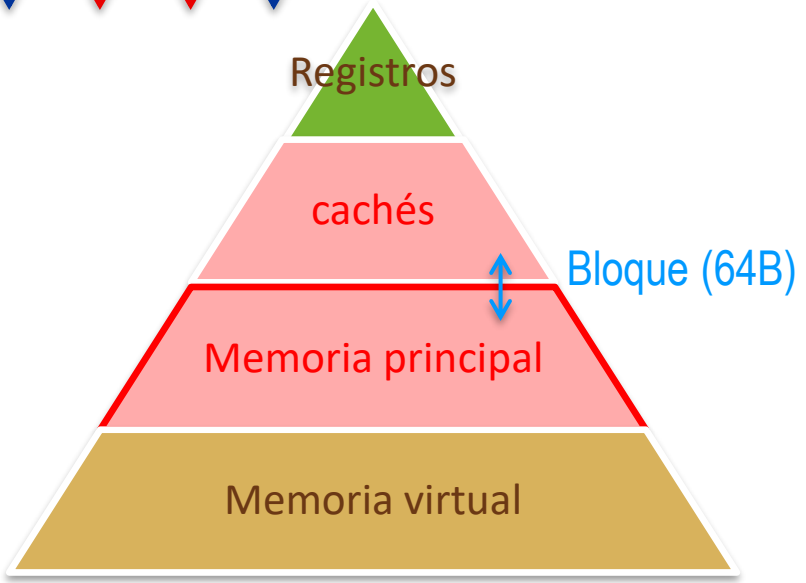










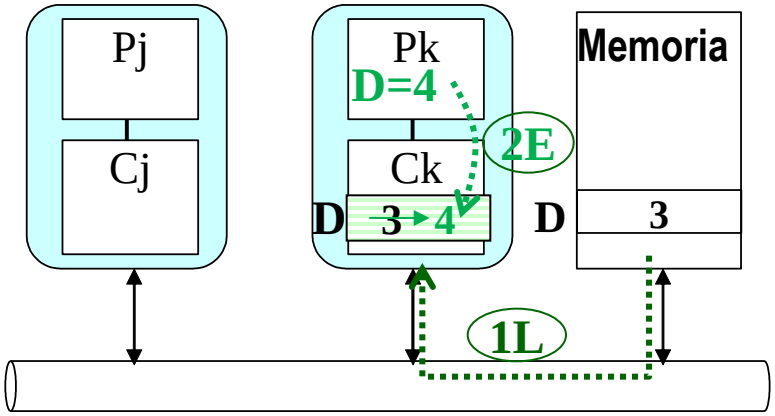


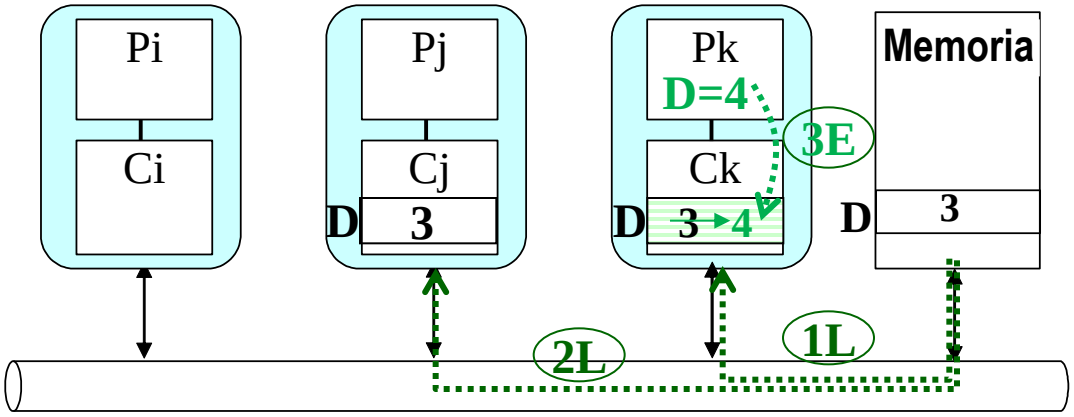
Contenido Lección 8

- Sistema de memoria en multiprocesadores
- Concepto de coherencia en el sistema de memoria. Situaciones de incoherencia.
- Protocolos de mantenimiento de coherencia: clasificación y diseño
- Protocolo MSI de espionaje
- Protocolo MESI de espionaje
- Concepto de coherencia en el sistema de memoria. Requisitos para evitar problemas por incoherencia.
- Protocolo MSI basado en directorios con o sin difusión

Incoherencia en el sistema de memoria

Clases de estructuras de datos	Eventos que ponen de manifiesto faltas de coherencia	Tipos de Falta de coherencia
Datos **modificables**	E/S	caché-MP
Datos **modificables** compartidos	Fallo de caché	caché-MP
Datos **modificables** privados	Emigra thread/proceso→Fallo caché	caché-MP
Datos **modificables** compartidos	Lectura de caché no actualizada	caché-caché



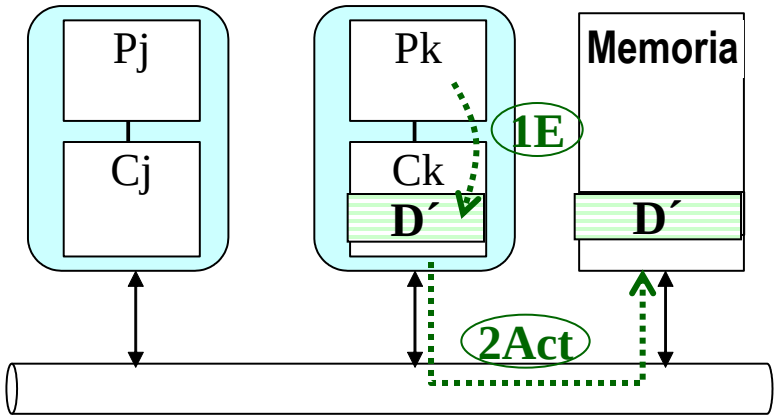


Métodos de actualización de memoria principal implementados en cachés

core

```
for(i = 0; i < N; i++)
    for (j = 0; j < N; j++)
        c[i] += m[i][j] * b[j];
```

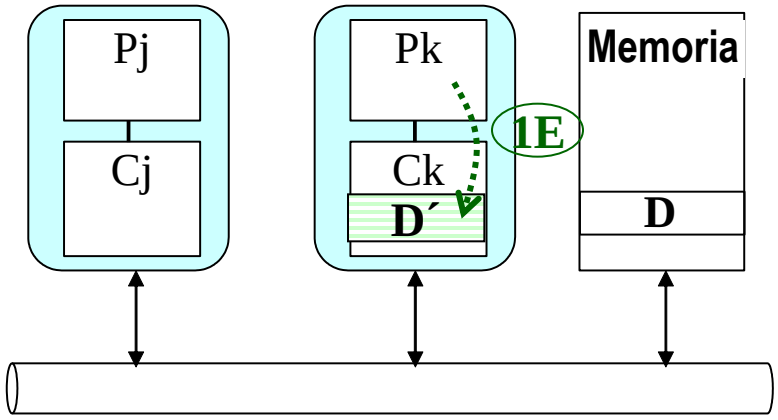
1. Escritura inmediata (*write-through*):



Cada vez que un procesador escribe en su caché escribe también en memoria principal

Por los principios de localidad temporal y espacial sería más rentable si se escribe todo el bloque en MP una vez realizadas las múltiples escrituras

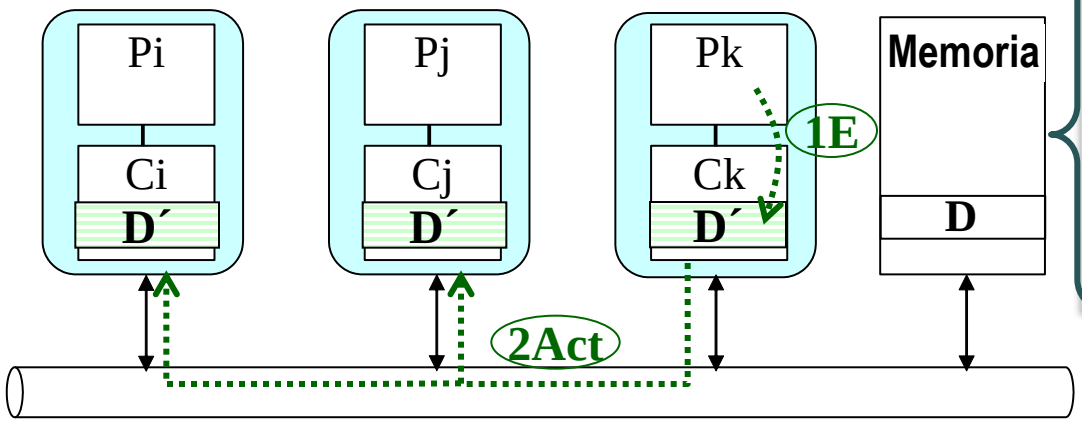
2. Posescritura (*write-back*):



Se actualiza memoria principal escribiendo todo el bloque cuando se **desaloja** de la caché

Alternativas para propagar una escritura en protocolos de coherencia de caché

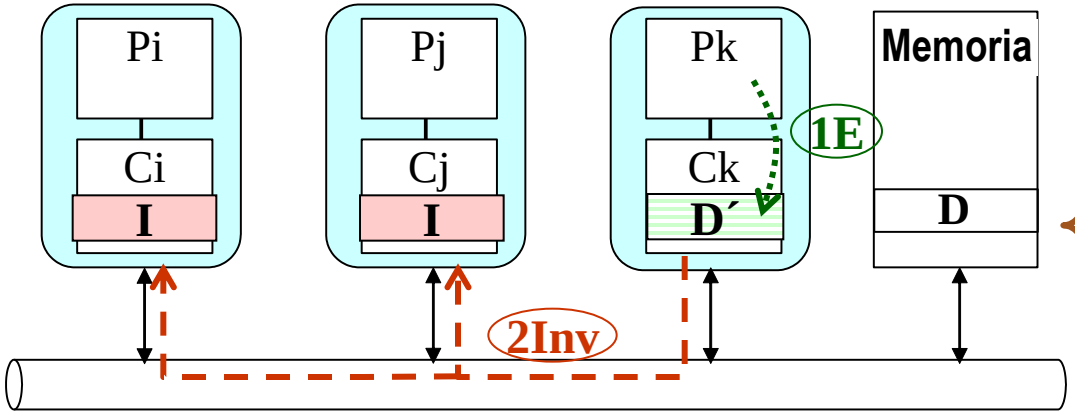
1. Escritura con actualización (*write-update*):



Cada vez que un procesador escribe en una dirección en su caché se escribe en las copias de esa dirección en otras cachés

Para reducir tráfico, sobre todo si los datos están compartidos por pocos procesadores

2. Escritura con invalidación (*write-invalidate*):



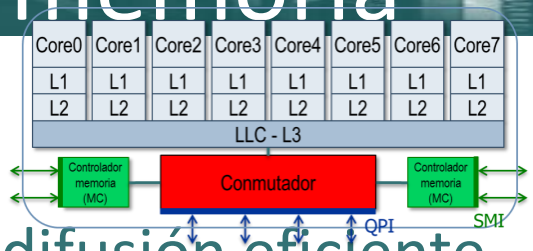
Antes que un procesador modifique una dirección en su caché se invalidan las copias del bloque de la dirección en otras cachés

Contenido Lección 8

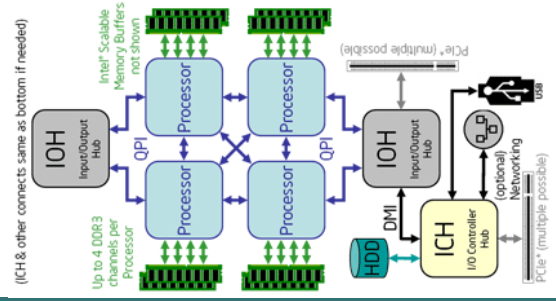
- Sistema de memoria en multiprocesadores
- Concepto de coherencia en el sistema de memoria. Situaciones de incoherencia.
- Protocolos de mantenimiento de coherencia: clasificación y diseño
- Protocolo MSI de espionaje
- Protocolo MESI de espionaje
- Concepto de coherencia en el sistema de memoria. Requisitos para evitar problemas por incoherencia.
- Protocolo MSI basado en directorios con o sin difusión

Clasificación de protocolos para mantener coherencia en el sistema de memoria

bus



- Protocolos de espionaje (snoopy)
  - Para buses, y en general sistemas con una difusión eficiente (bien porque el número de nodos es pequeño o porque la red implementa difusión).
- Protocolos basados en directorios.
  - Para redes sin difusión o escalables (multietapa y estáticas).
- Esquemas jerárquicos.
  - Para redes jerárquicas: jerarquía de buses, jerarquía de redes escalables, redes escalables-buses.



Facetas de diseño lógico en protocolos para coherencia

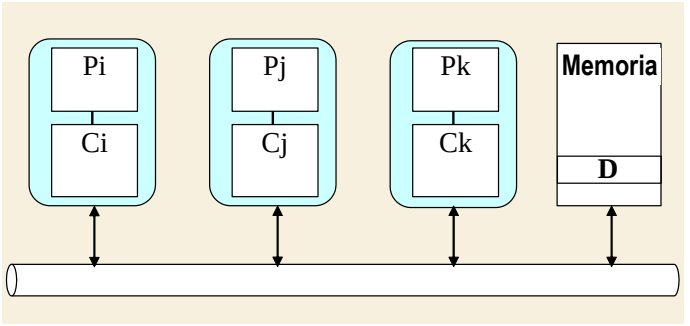
- Política de actualización de MP:
  - escritura inmediata, posescritura, mixta
- Política de coherencia entre cachés:
  - escritura con invalidación, escritura con actualización, mixta
- Describir comportamiento:
  - Definir posibles estados de los bloques en caché, y en memoria
  - Definir transferencias (indicando nodos que intervienen y orden entre ellas) a generar ante eventos:
    - lecturas/escrituras del procesador del nodo
    - como consecuencia de la llegada de paquetes de otros nodos.
  - Definir transiciones de estados para un bloque en caché, y en memoria

posescr

Ej. tras.

Contenido Lección 8

- Sistema de memoria en multiprocesadores
- Concepto de coherencia en el sistema de memoria. Situaciones de incoherencia.
- Protocolos de mantenimiento de coherencia: clasificación y diseño
- Protocolo MSI de espionaje
- Protocolo MESI de espionaje



- Concepto de coherencia en el sistema de memoria. Requisitos para evitar problemas por incoherencia.
- Protocolo MSI basado en directorios con o sin difusión

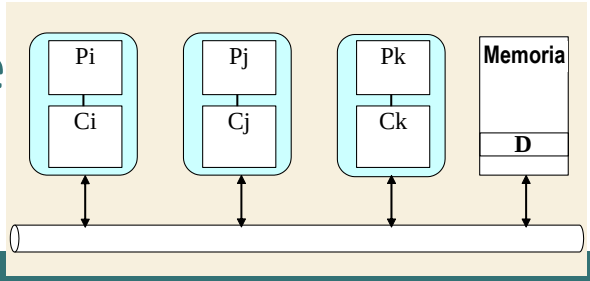
Protocolo de espionaje de tres estados (MSI) – posescritura e invalidación

- Estados de un bloque en caché:
  - **Modificado** (M): es la única copia del bloque válida en todo el sistema
  - **Compartido** (C,S): está válido, también válido en memoria y puede que haya copia válida en otras cachés
  - **Inválido** (I): se ha invalidado o no está físicamente
- Estados de un bloque en memoria (en realidad se evita almacenar esta información):
  - **Válido**: puede haber copia válida en una o varias cachés
  - **Inválido**: habrá copia valida en una caché

direct.

posescr

invalid.



Protocolo de espionaje de tres estados (MSI) – posescritura e invalidación

- Transferencias generadas por un nodo con caché (tipos de paquetes):
  - Petición de lectura de un bloque (**PtLec**): por lectura con fallo de caché del procesador del nodo (PrLec)
  - Petición de acceso exclusivo (**PtLecEx**): por escritura del procesador (PrEsc) en bloque ***compartido*** o ***inválido*** (alternativa: ***compartido*** PtEx, ***inválido*** **PtLecEx**)
  - Petición de posescritura (**PtPEsc**): por el reemplazo del bloque ***modificado*** (el procesador del nodo no espera respuesta)
  - Respuesta con bloque (**RpBloque**): al tener en estado ***modificado*** el bloque solicitado por una PtLec o PtLecEx recibida

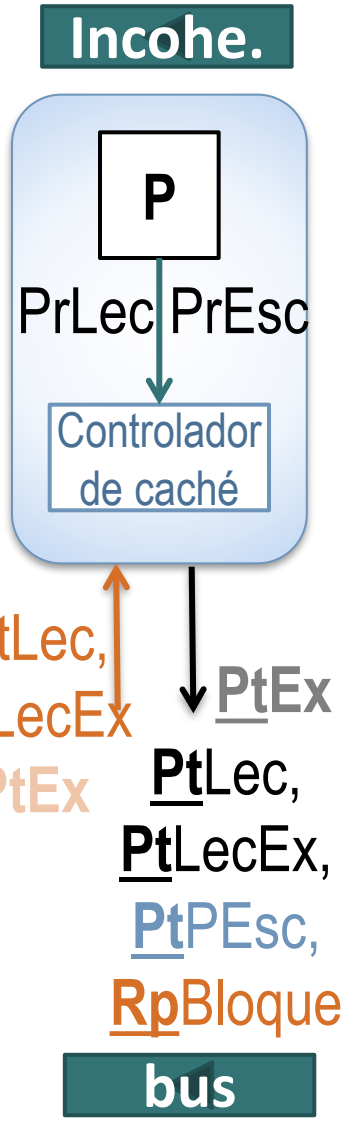
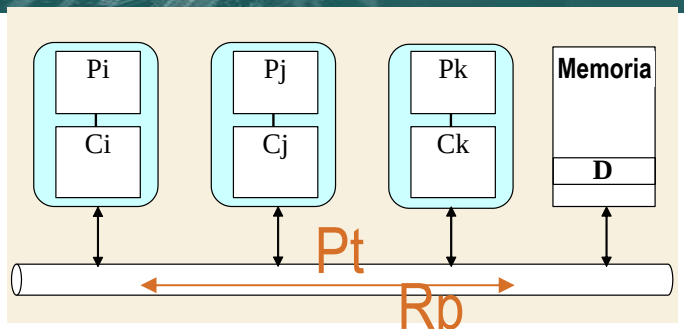
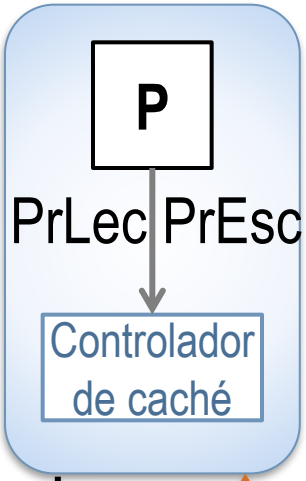
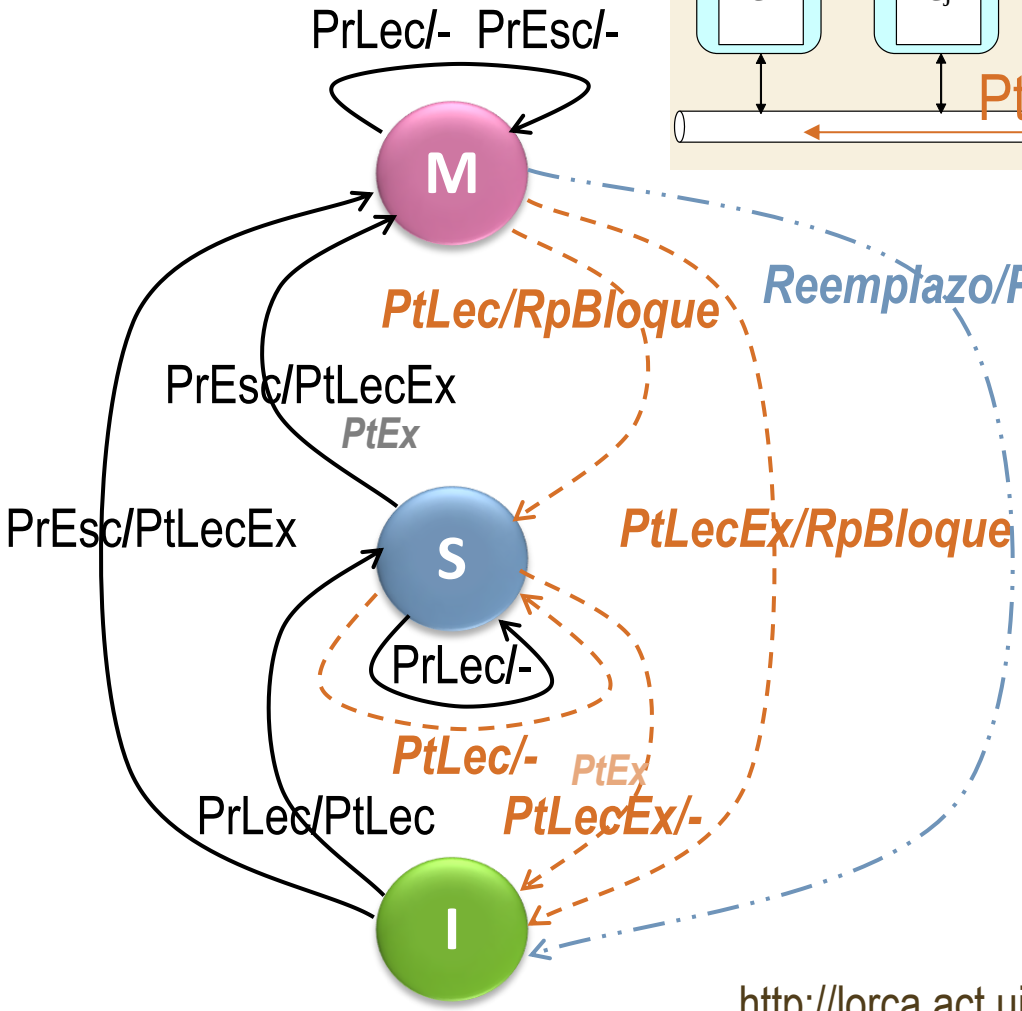


Diagrama MSI de transiciones de estados







PtLec, PtEx, PtLecEx, PtPEsc, RpBloque

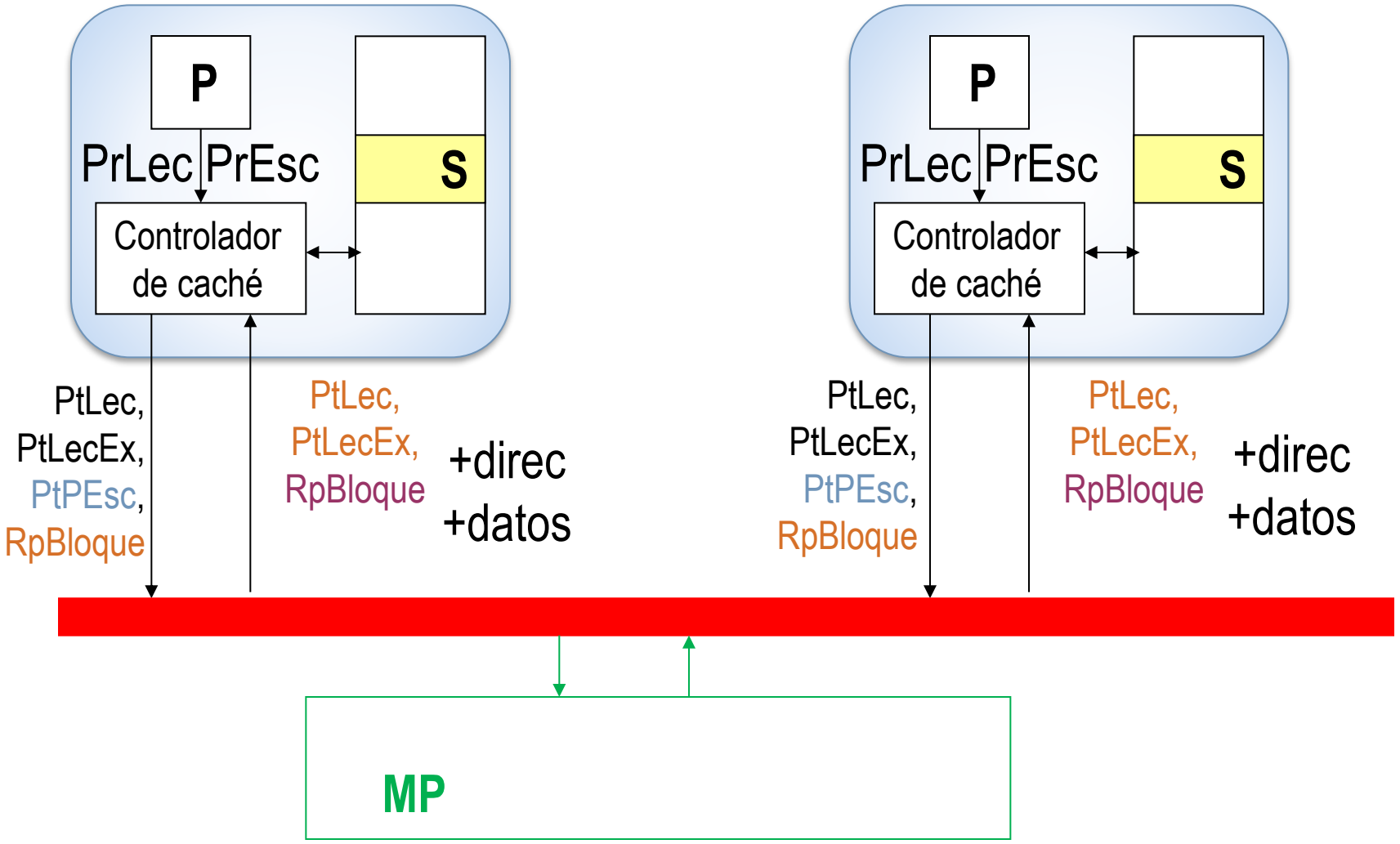
PtLec, PtLecEx, PtEx, RpBloque

http://lorca.act.uji.es/projects/ccp/

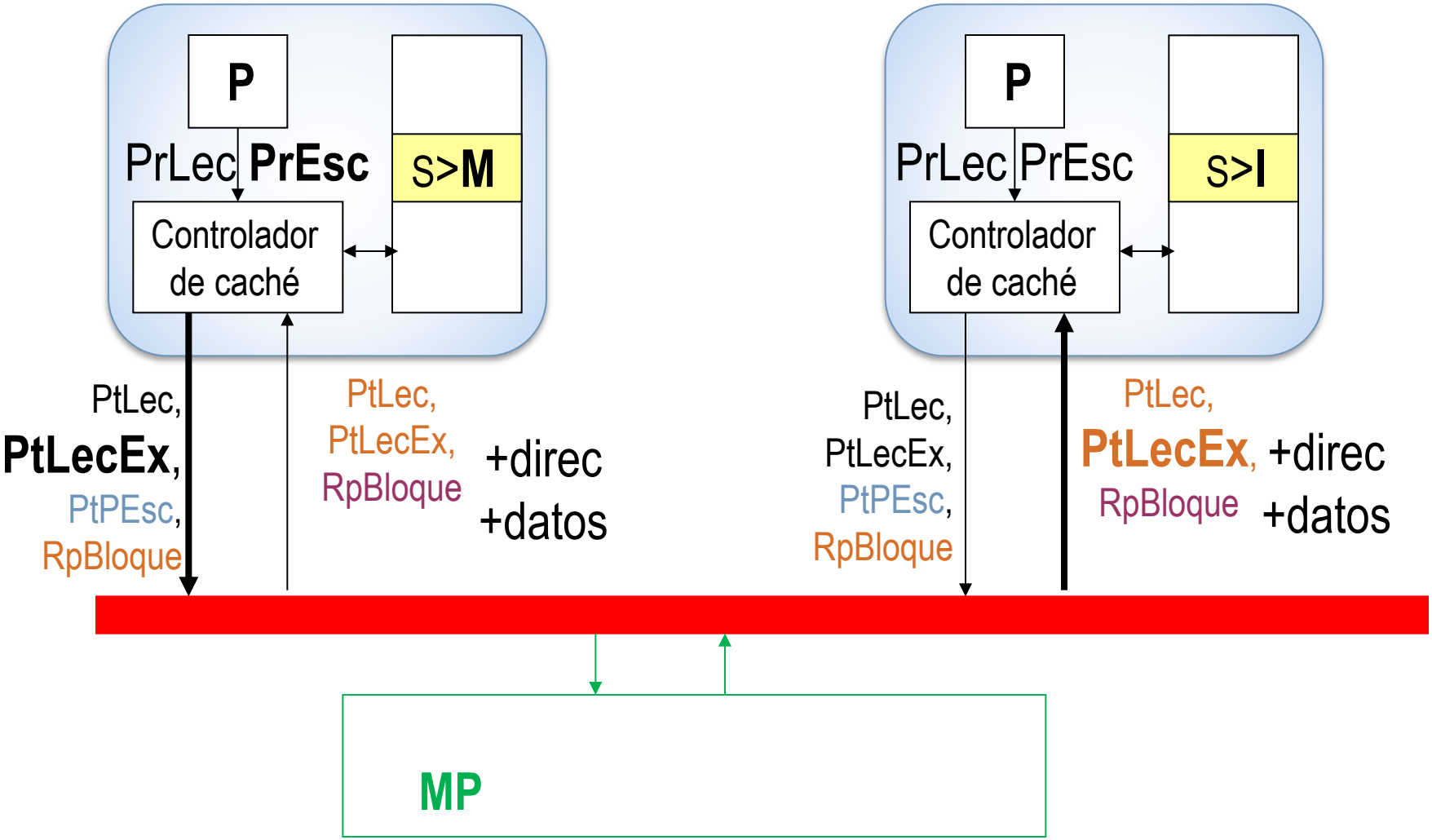
Tabla de descripción de MSI

EST. ACT. k	EVENTO	ACCIÓN	SIGUIENTE
Modificado (M)	PrLec/PrEsc		Modificado
	PtLec	Genera paquete respuesta (RpBloque)	Compartido
	PtLecEx	Genera paquete respuesta (RpBloque) Invalida copia local	Inválido
	Reemplazo	*Genera paquete posescritura (PtPEsc)*	*Inválido*
Compart. (S)	PrLec		Compartido
	PrEsc posescr	Genera paquete PtLecEx (PtEx)	Modificado
	PtLec		Compartido
	PtLecEx invalido	Invalida copia local	Inválido
Inválido (I)	PrLec	Genera paquete PtLec	Compartido
	PrEsc	Genera paquete PtLecEx	Modificado
	PtLec/PtLecEx		Inválido

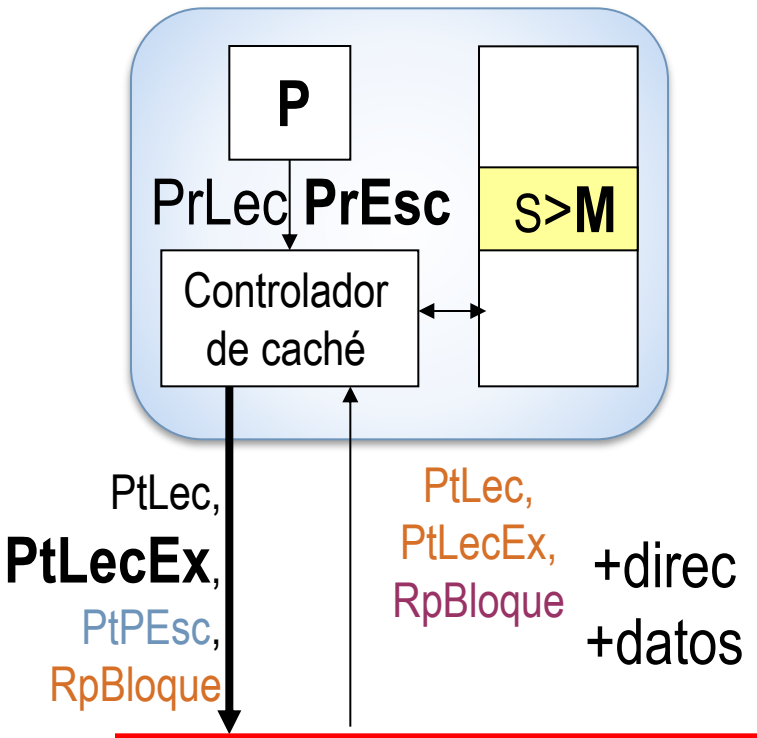
Ejemplo MSI I

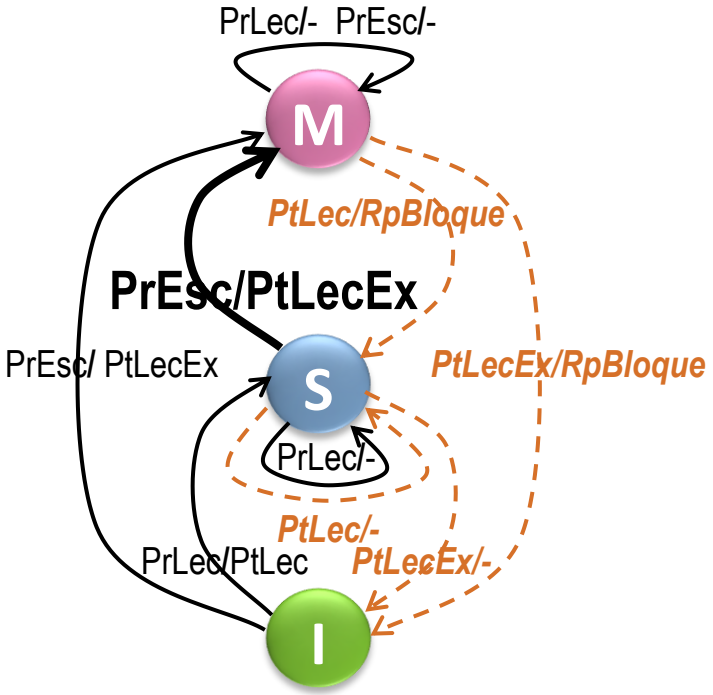


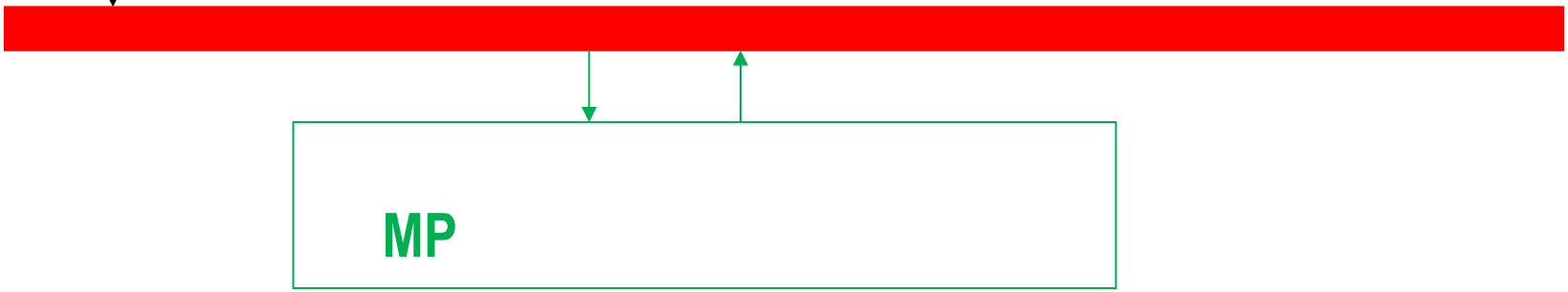
Ejemplo MSI II



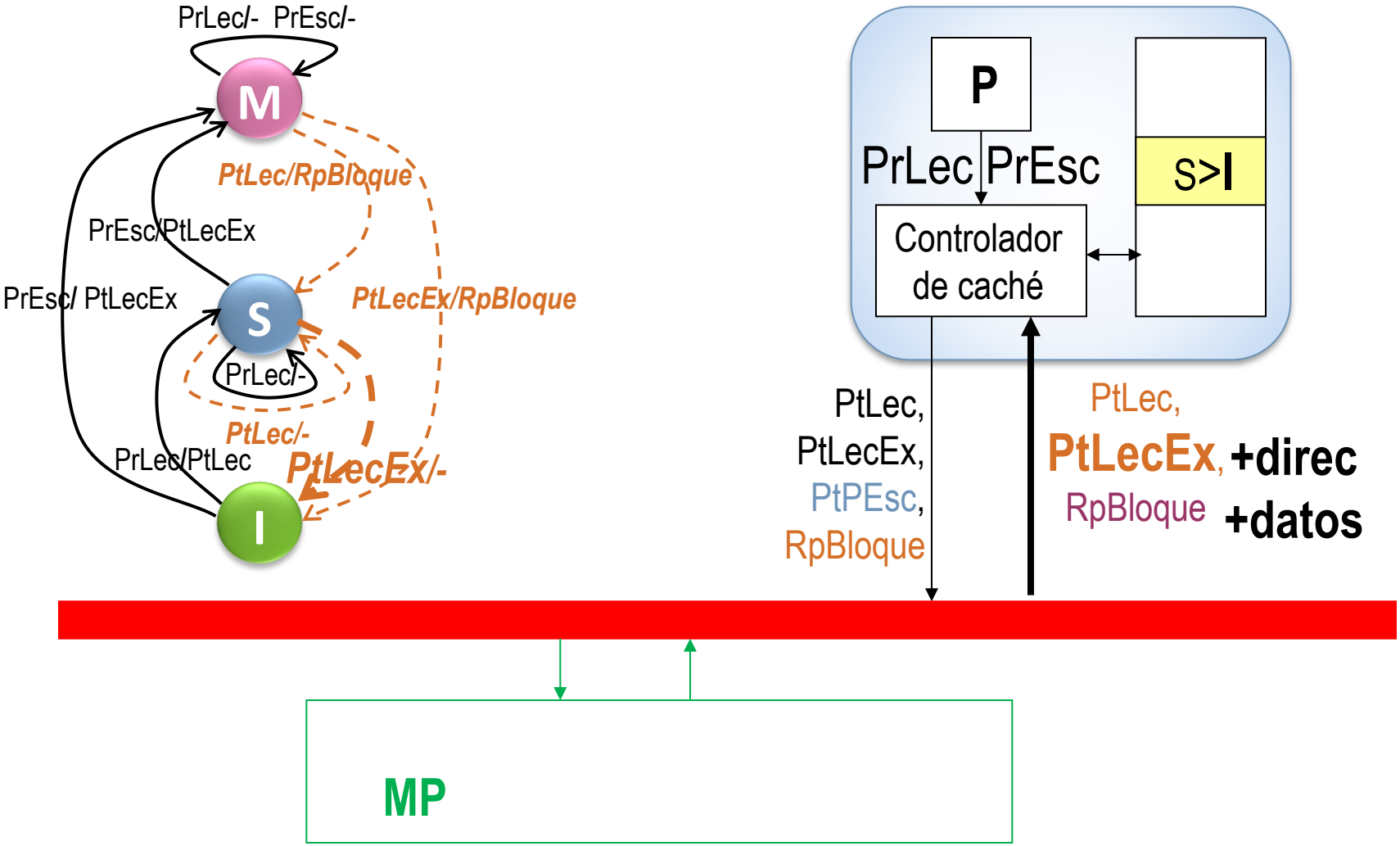
Ejemplo MSI III





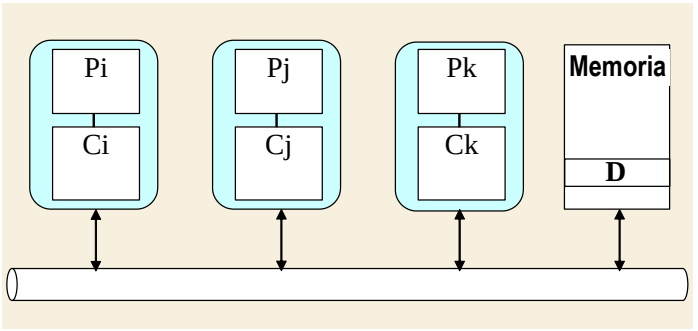


Ejemplo MSI IV



Contenido Lección 8

- Sistema de memoria en multiprocesadores
- Concepto de coherencia en el sistema de memoria. Situaciones de incoherencia.
- Protocolos de mantenimiento de coherencia: clasificación y diseño
- Protocolo MSI de espionaje
- Protocolo MESI de espionaje

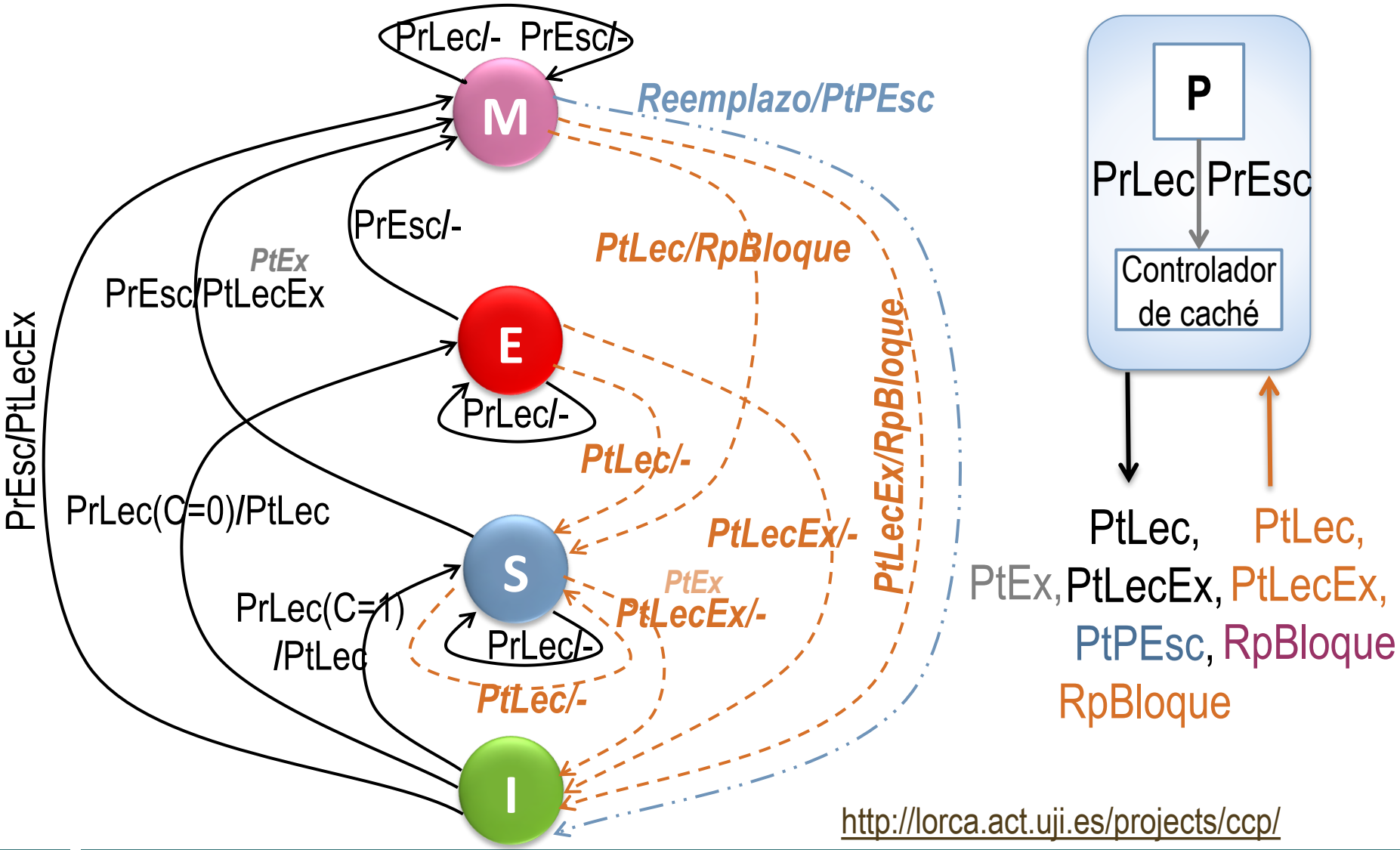


- Concepto de coherencia en el sistema de memoria. Requisitos para evitar problemas por incoherencia.
- Protocolo MSI basado en directorios con o sin difusión

Protocolo de espionaje de cuatro estados (MESI) – posescritura e invalidación

- Estados de un bloque en caché:
  - **Modificado** (M):es la única copia del bloque válida en todo el sistema
  - **Exclusivo** (E): es la *única* copia del bloque válida en cachés, la memoria también está actualizada
  - **Compartido** (C,Shared): es válido, también válido en memoria y en *al menos* otra caché
  - **Inválido** (I): se ha invalidado o no está físicamente
- Estados de un bloque en memoria(en realidad se evita almacenar esta información):
  - **Válido**: puede haber copia válida en una o varias cachés
  - **Inválido**: habrá copia valida en una caché

Diagrama MESI de transiciones de estados



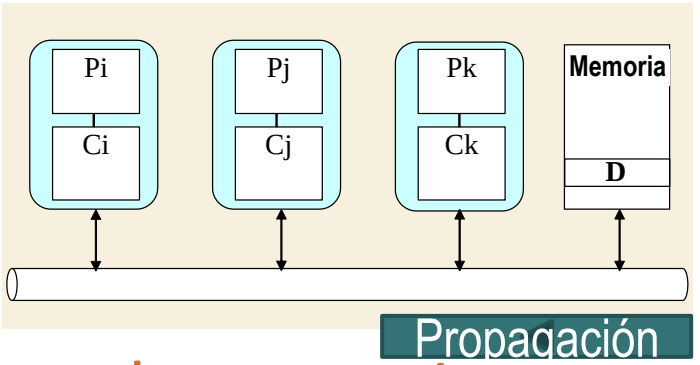
http://lorca.act.uji.es/projects/ccp/

Tabla de descripción de MESI

Modificado (M)	PrLec/PrEsc		Modificado
	PtLec	Genera RpBloque	Compartido
	PtLecEx	Genera RpBloque. Invalida copia local	Inválido
	Reemplazo	*Genera PtPEsc*	*Inválido*
Exclusivo (E)	PrLec		Exclusivo
	PrEsc		Modificado
	PtLec		Compartido
	PtLecEx	Invalida copia local	Inválido
Compartido (S)	PrLec/PtLec		Compartido
	PrEsc	Genera PtLecEx (PtEx)	Modificado
	PtLecEx	Invalida copia local	Inválido
Inválido (I)	PrLec (C=1)	Genera PtLec	Compartido
	PrLec (C=0)	Genera PtLec	Exclusivo
	PrEsc	Genera PtLecEx	Modificado
	PtLec/PtLecEx		Inválido

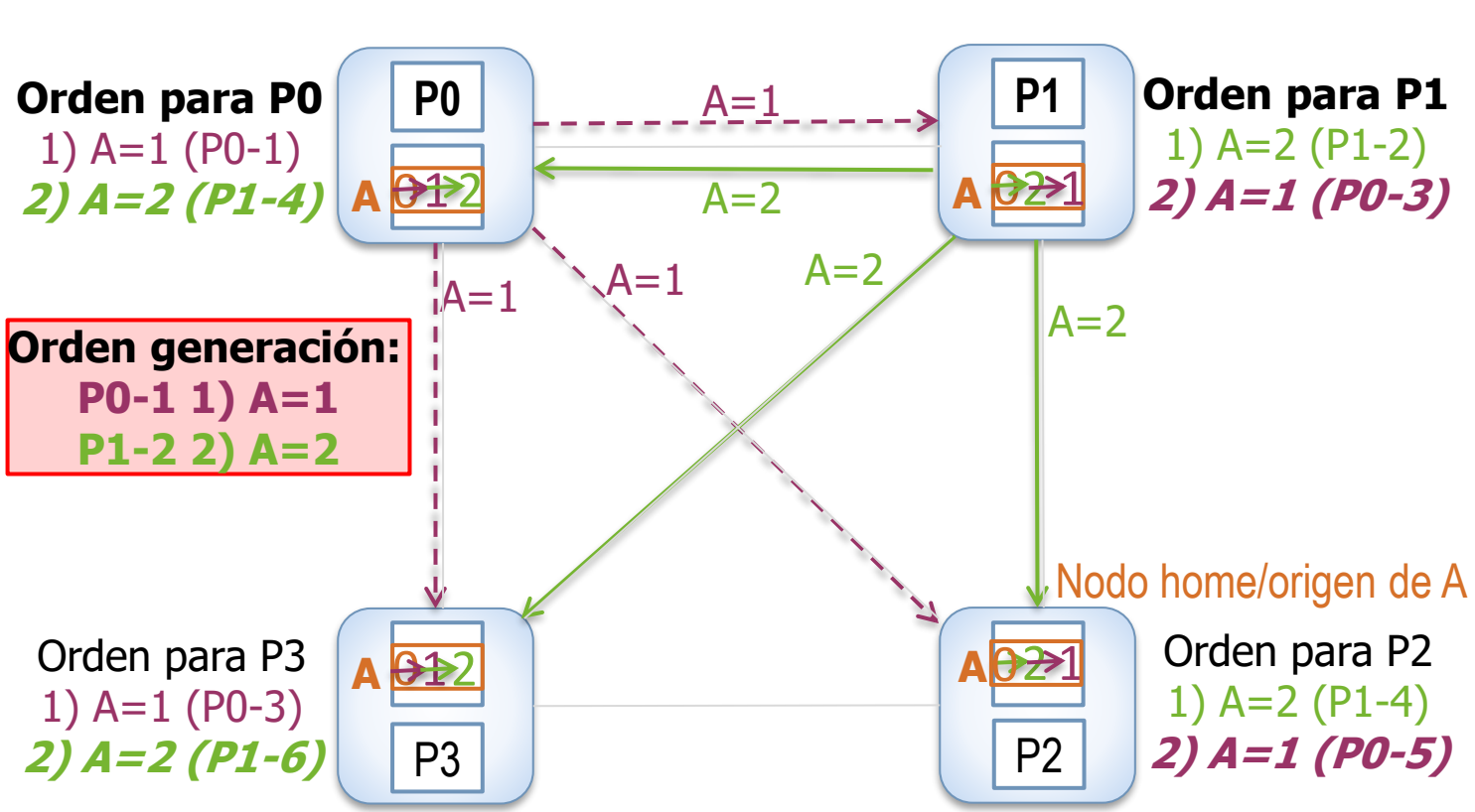
Contenido Lección 8

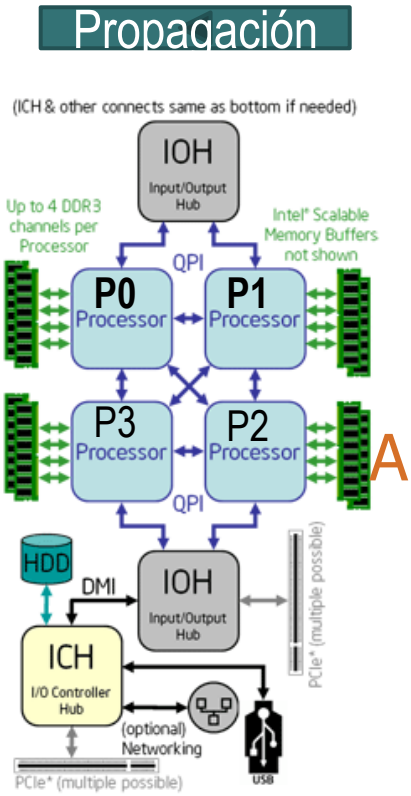
- Sistema de memoria en multiprocesadores
- Concepto de coherencia en el sistema de memoria. Situaciones de incoherencia.
- Protocolos de mantenimiento de coherencia: clasificación y diseño
- Protocolo MSI de espionaje
- Protocolo MESI de espionaje



- Concepto de coherencia en el sistema de memoria. Requisitos para evitar problemas por incoherencia.
- Protocolo MSI basado en directorios con o sin difusión

Situación de incoherencia aunque se propagan las escrituras (**usa difusión**)

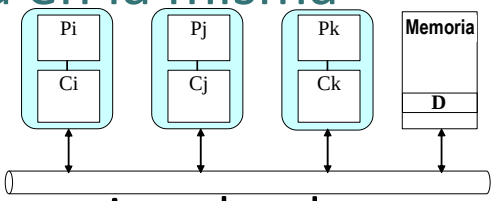




- Contenido inicial de las copias de la dirección A en **calabaza**. P0 escribe en A un 1 y, después, P1 escribe en A un 2.
- Se utiliza **actualización** para propagar las escrituras (las propagación se nota con flechas)
- Llegan en distinto orden las escrituras debido al distinto tiempo de propagación (**se está suponiendo que los procesadores están ubicados en la placa tal y como aparecen en el dibujo**). En ***cursiva*** se puede ver el contenido de las copias de la dirección A tras las dos escrituras.
  - Se da una situación de incoherencia aunque se propagan las escrituras: P0 y P3 acaban con 2 en A y P1 y P2 con 1.

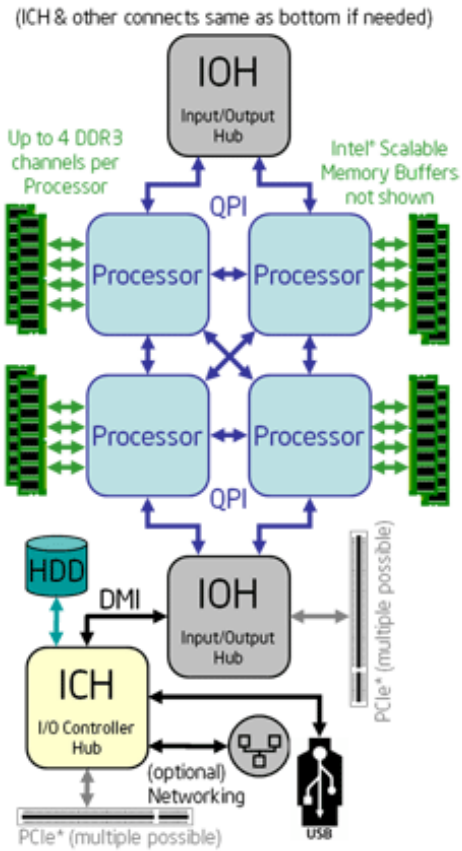
Requisitos del sistema de memoria para evitar problemas por incoherencia I

- **Propagar** las escrituras en una dirección
  - La escritura en una dirección debe hacerse visible en un tiempo finito a otros procesadores
  - Componentes conectados con un bus:
    - Los paquetes de actualización/invalidación son visibles a todos los nodos conectados al bus (controladores de caché)
- **Serializar** las escrituras en una dirección
  - Las escrituras en una dirección deben verse en el mismo orden por todos los procesadores (el sistema de memoria debe **parecer** que realiza en serie las operaciones de escritura en la misma dirección)
  - Componentes conectados con un bus:
    - El orden en que los paquetes aparecen en el bus determina el orden en que se ven por todos los nodos.



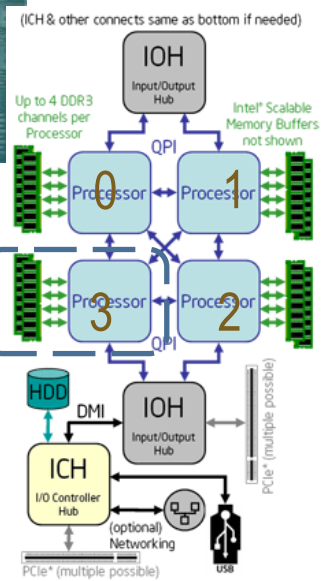
Requisitos del SM para evitar problemas por incoherencia II: la red no es un bus

- **Propagar** escrituras en una dirección
  - Usando difusión:
    - Los paquetes de actualización/invalidación se envían a todas las cachés
  - Para conseguir mayor escalabilidad:
    - Se debería enviar paquetes de actualización/invalidación **sólo a cachés (nodos) con copia del bloque**
    - Mantener en un directorio, **para cada bloque, los nodos con copia del mismo**



- **Serializar** escrituras en una dirección
  - El orden en el que las peticiones de escritura llegan a su ***home*** (nodo que tiene en MP la dirección) o al directorio centralizado sirve para serializar en sistemas de comunicación que garantizan el orden en las trasferencias entre dos puntos

Directorio de memoria principal



CC: Controlador caché
CM: Controlador de Memoria

C0

Directorio caché

0 0

Estado bloque caché

Nodo de cómputo

P

CC

C

E/S

Conmu-tador

CM

Memoria

conexión a otros nodos de cómputo

Memoria

bloque

bloque

bloque

Directorio

C0 C1 ... CN-1

1 0 ... 1 0

Estado bloque memoria
(0: válido, 1: inválido)

Vector de bits (un bit por caché en el sistema) con información sobre cachés con copia

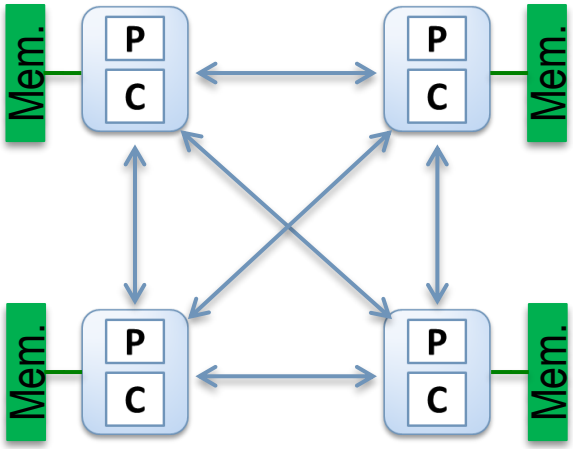
Alternativas para implementar el directorio

Centralizado

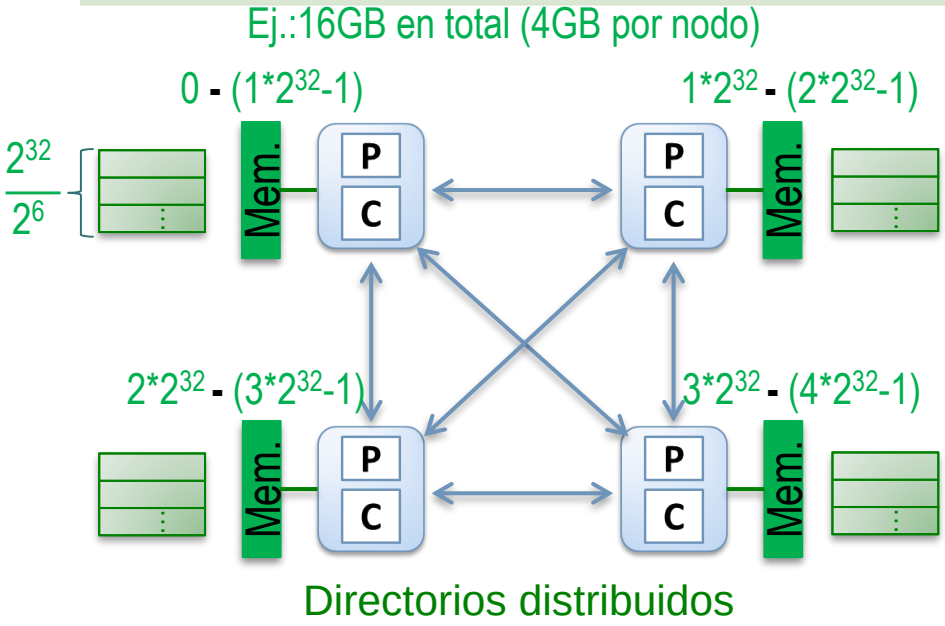
- Compartido por todos los nodos
- Contiene información de los bloques de todos los módulos de memoria

Distribuido

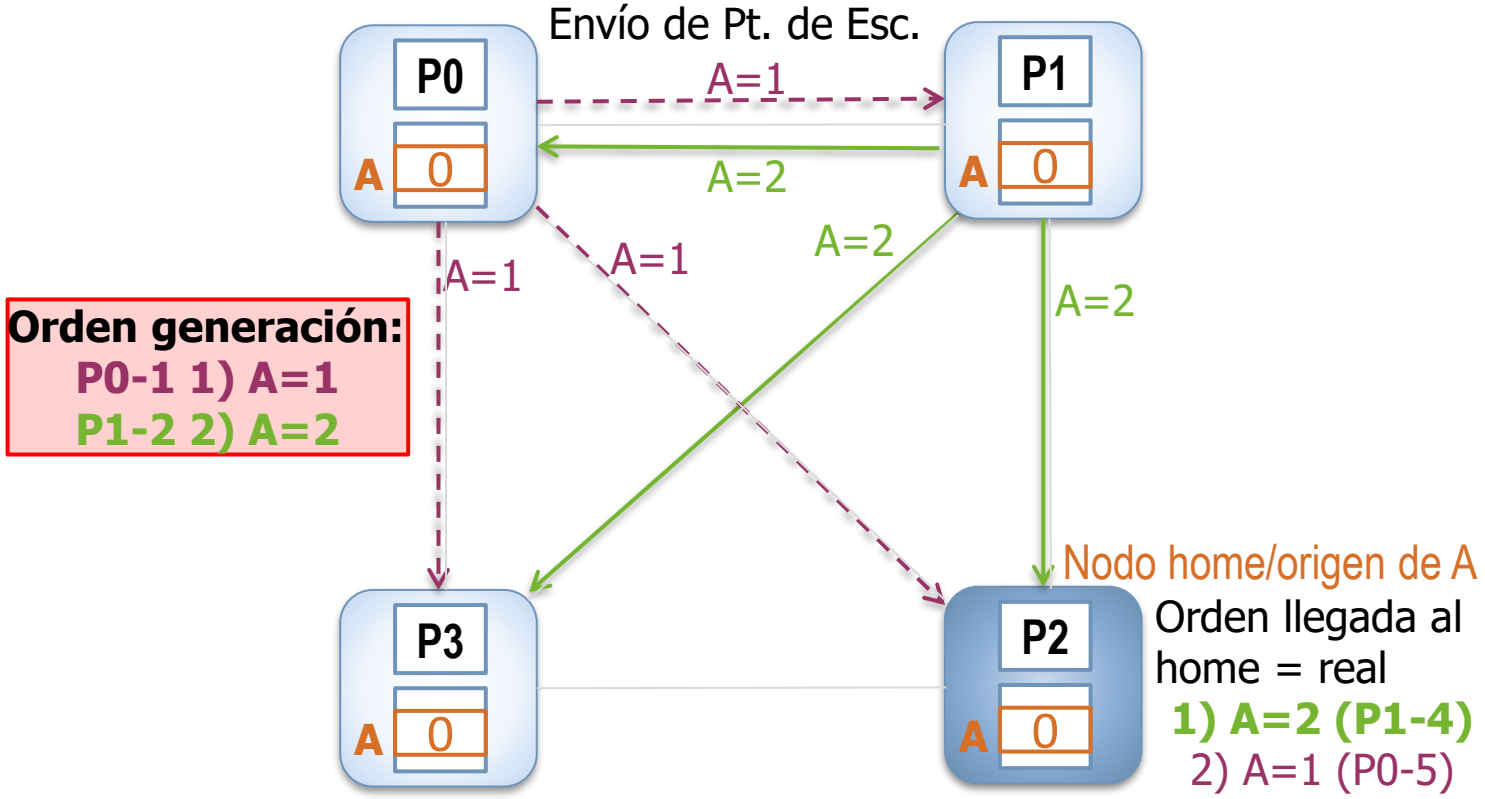
- Las filas se distribuyen entre los nodos
- Típicamente el directorio de un nodo contiene información de los bloques de sus módulos de memoria

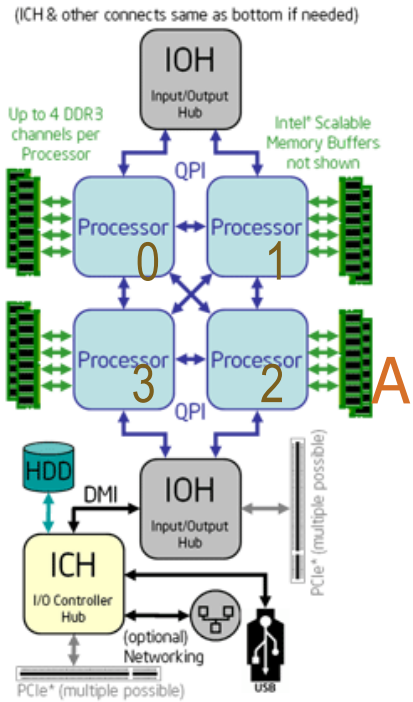






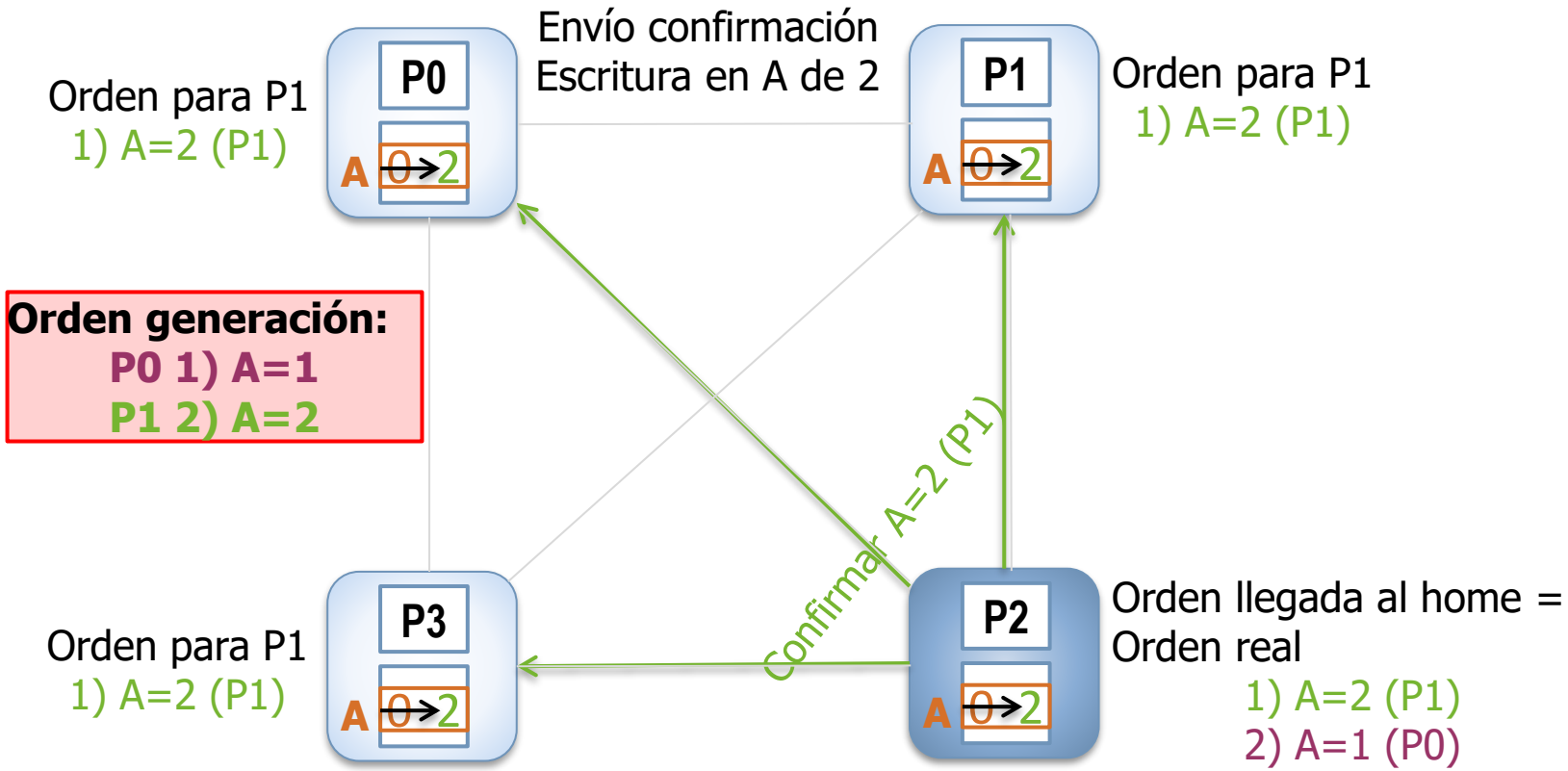
Serialización de las escrituras por el home. Usando **difusión** I





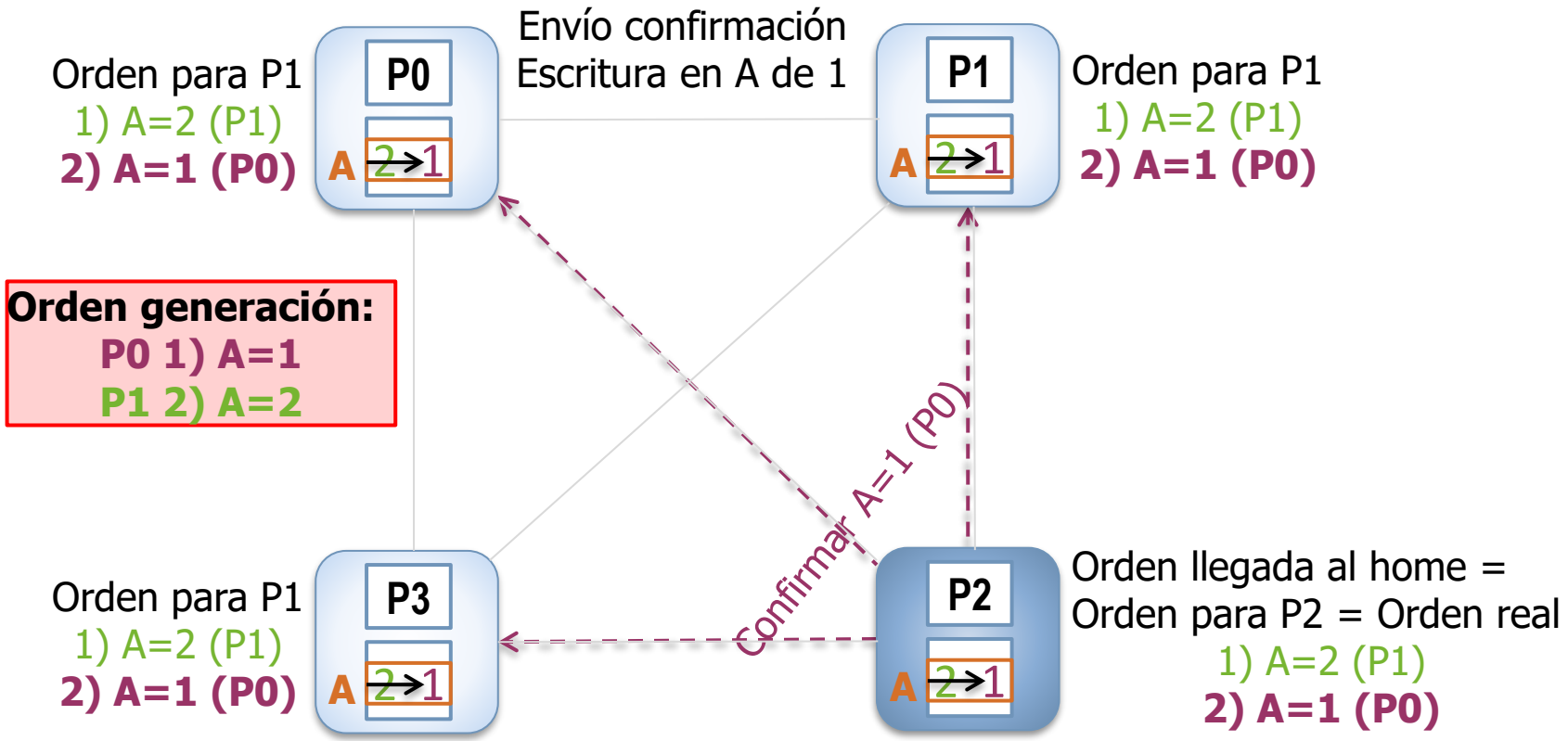
- Contenido inicial de las copias de la dirección A en **calabaza**. P0 escribe en A un 1 y, después, P1 escribe en A un 2.
- Se utiliza **actualización** para propagar las escrituras
- El orden de llegada al home es el orden real para todos

Serialización de las escrituras por el home. Usando **difusión** II



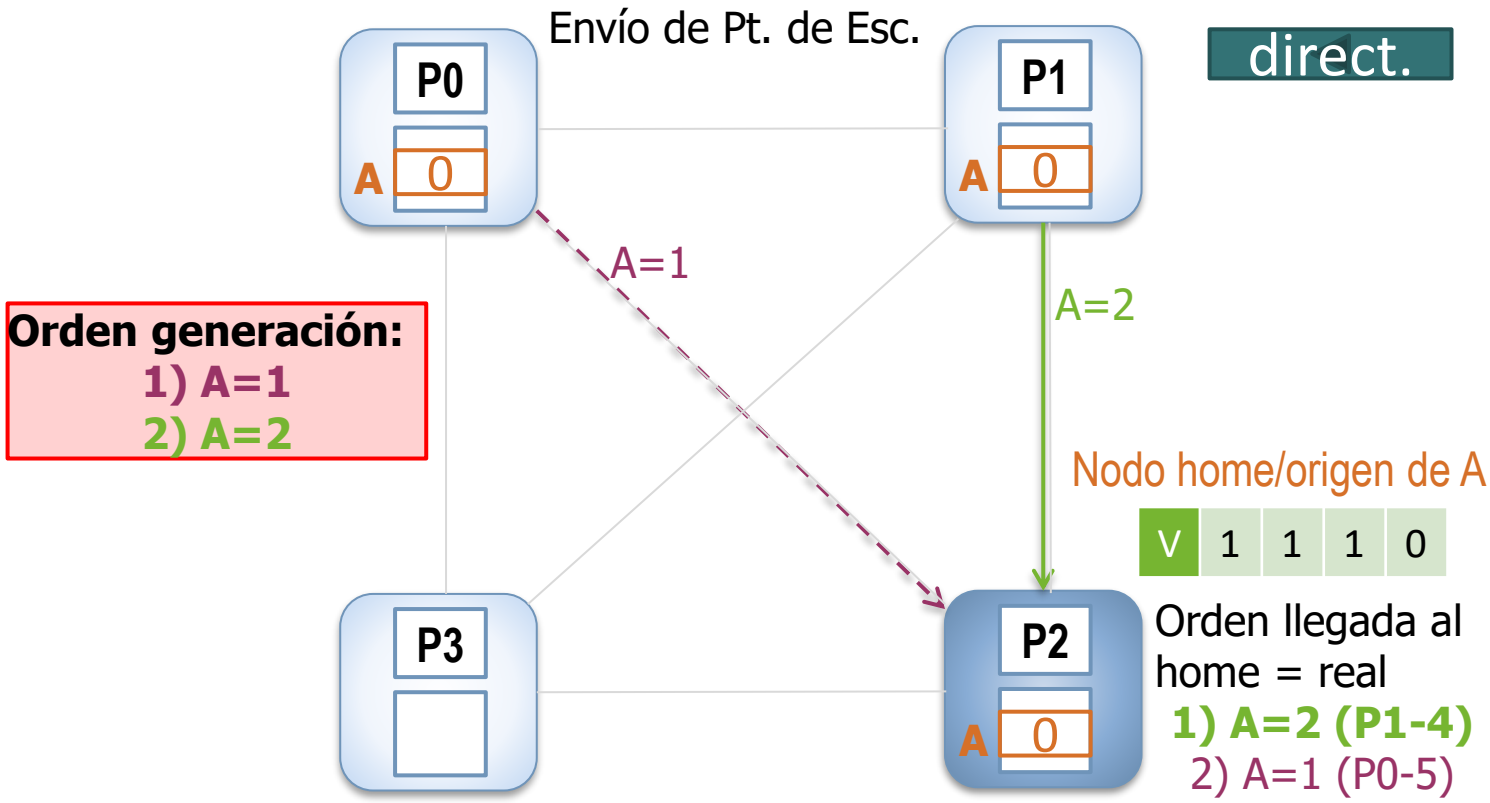
- Contenido inicial de las copias de la dirección A en **calabaza**
- Se utiliza actualización para propagar las escrituras (la propagación se nota con flechas)

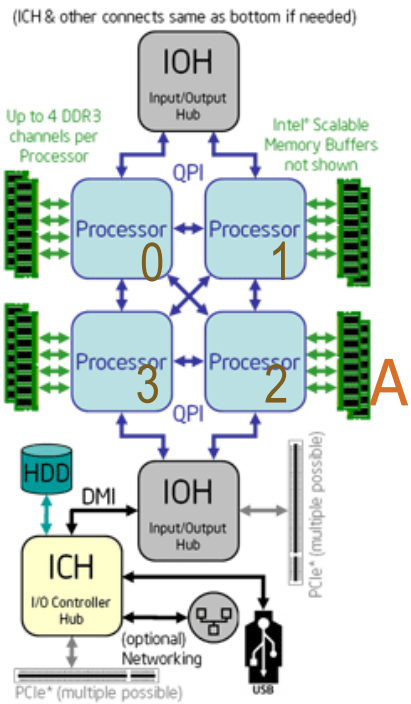
Serialización de las escrituras por el home. Usando **difusión** III



- Se utiliza actualización para propagar las escrituras (la propagación se nota con flechas)

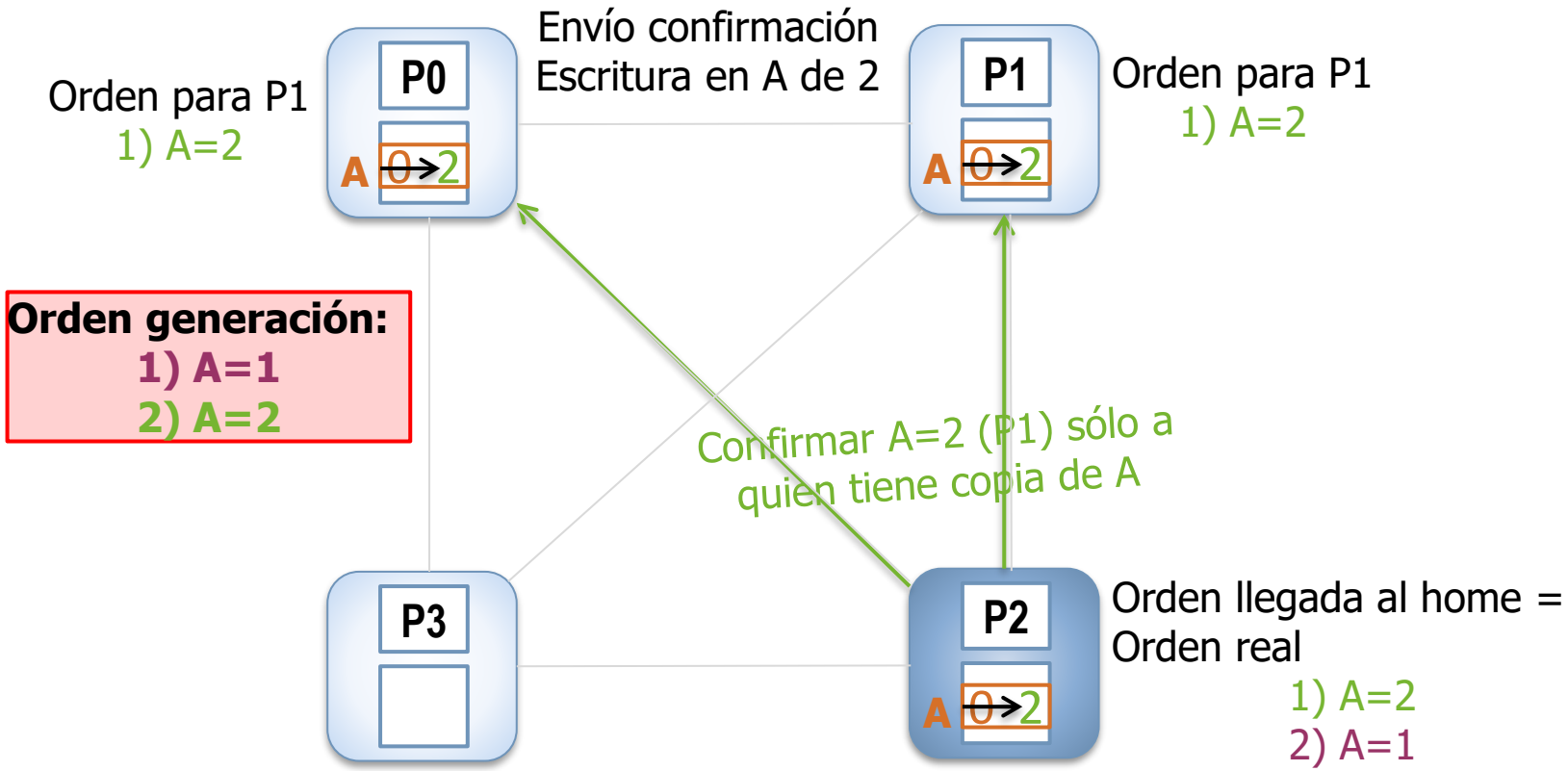
Serialización de las escrituras por el home. **Sin difusión** y con **directorio distribuido** I





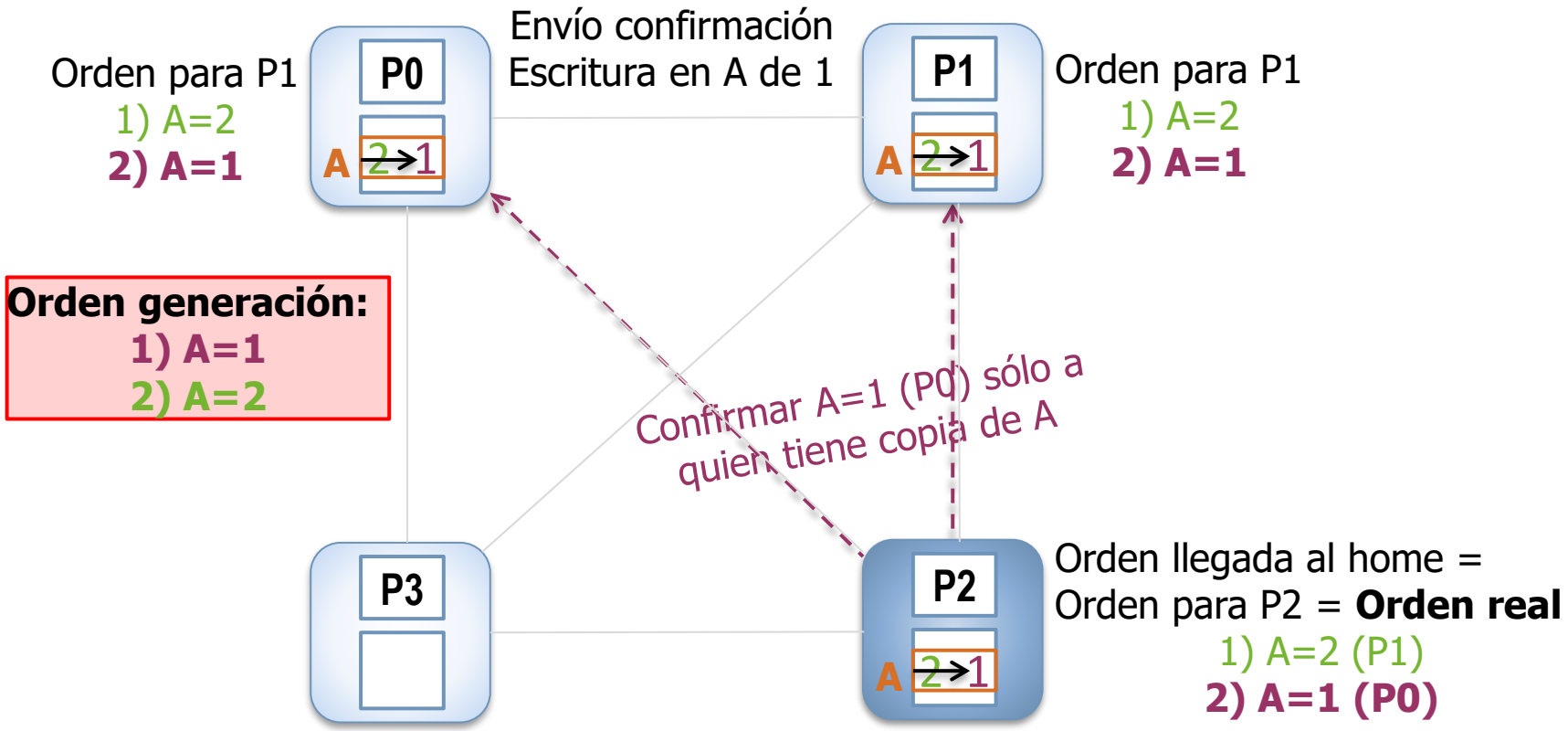
- Contenido inicial de las copias de la dirección A en **calabaza**. P0 escribe en A un 1 y, después, P1 escribe en A un 2.
- Se utiliza actualización para propagar las escrituras (la propagación se nota con flechas)
- El orden de llegada al home es el orden real para todos

Serialización de las escrituras por el home. **Sin difusión** y con **directorio distribuido** II



- Contenido inicial de las copias de la dirección A en **calabaza**
- Se utiliza actualización para propagar las escrituras (la propagación se nota con flechas)

Serialización de las escrituras por el home. **Sin difusión** y con **directorio distribuido** III



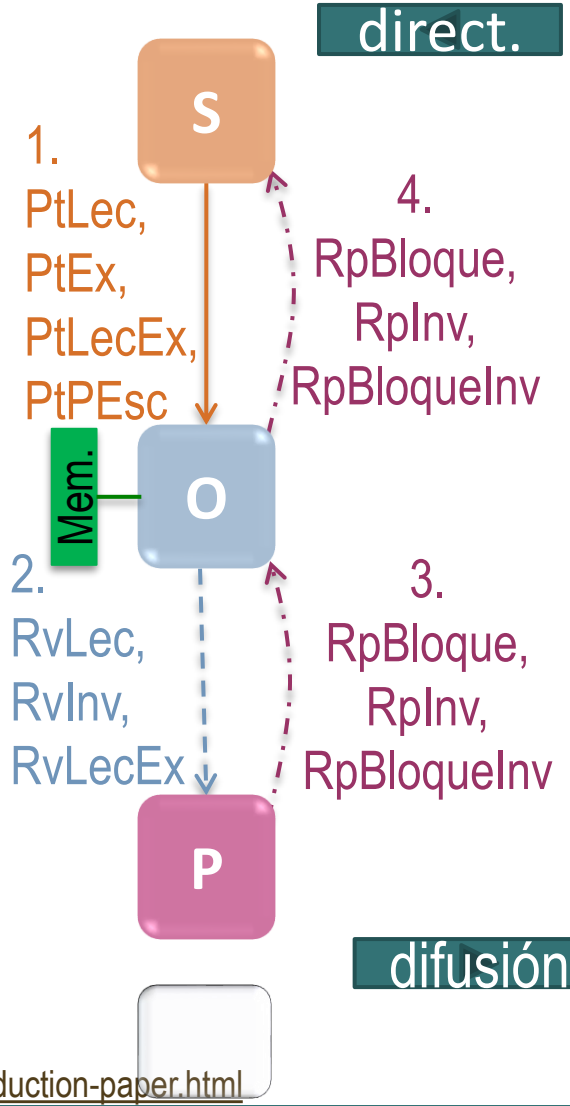
- Se utiliza actualización para propagar las escrituras (la propagación se nota con flechas)

Contenido Lección 8

- Sistema de memoria en multiprocesadores
- Concepto de coherencia en el sistema de memoria. Situaciones de incoherencia.
- Protocolos de mantenimiento de coherencia: clasificación y diseño
- Protocolo MSI de espionaje
- Protocolo MESI de espionaje
- Concepto de coherencia en el sistema de memoria. Requisitos para evitar problemas por incoherencia.
- Protocolo MSI basado en directorios con o sin difusión

MSI con directorios (sin difusión) I

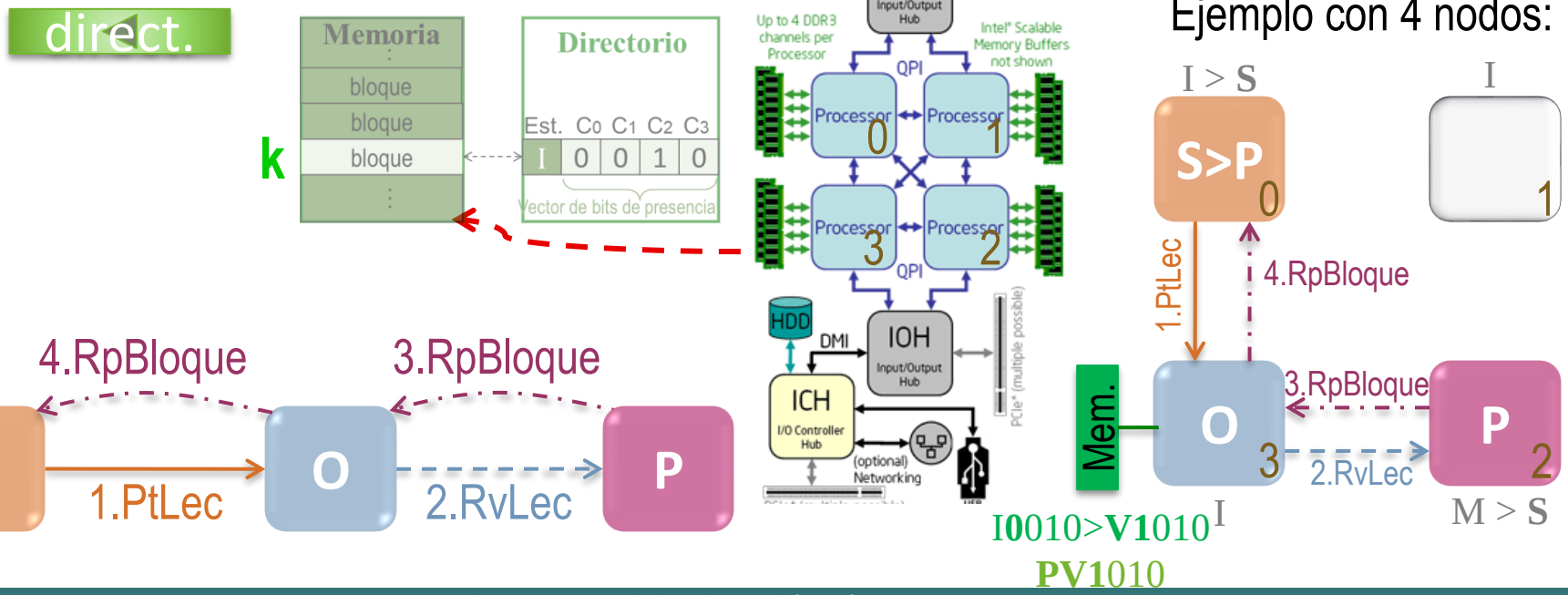
- Estados de un bloque en caché:
  - Modificado (M), Compartido (S), Inválido (I)
- Estados de un bloque en MP:
  - Válido e inválido
- Transferencias (tipos de paquetes) :
  - Tipos de nodos: solicitante (S), origen (O), modificado (M), propietario (P) y compartidor (C)
  - **Petición** de
    - nodo **S** a **O**: lectura de un bloque (**PtLec**), lectura con acceso exclusivo (**PtLecEx**), petición de acceso exclusivo sin lectura (**PtEx**), posescritura (**PtPEsc**)
  - **Reenvío** de petición de
    - nodo **O** a nodos con copia (**P**, **M**, **C**): invalidación (**RvInv**), lectura (**RvLec**, **RvLecEx**).
  - **Respuesta** de
    - nodo **P** a **O**: respuesta con bloque (**RpBloque**), resp. con o sin bloque confirmando inv. (**RpInv**, **RpBloqueInv**)
    - nodo **O** a **S**: resp. con bloque (**RpBloque**), resp. con o sin bloque confirmando que se han invalidado todas las copias del bloque en cachés (**RpInv**, **RpBloqueInv**)



https://www.intel.com/content/www/us/en/io/quickpath-technology/quick-path-interconnect-introduction-paper.html

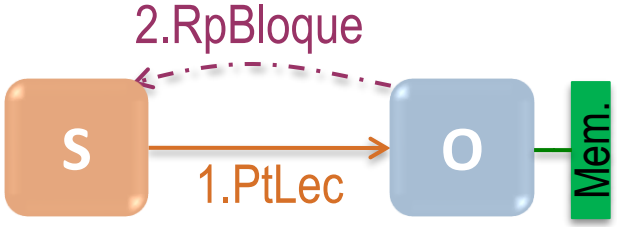
MSI con directorios (sin difusión) II

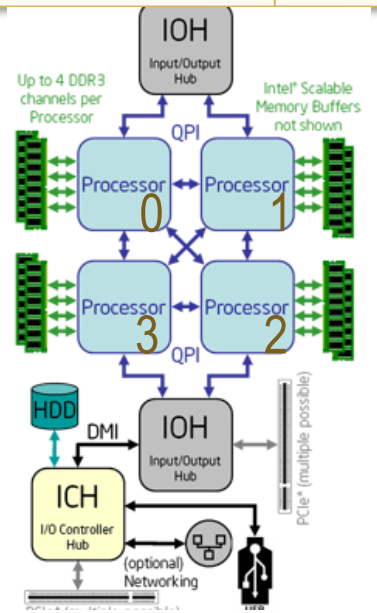
Estado inicial (k)	Evento	Estado final (bloque k)
D) Inválido S) Inválido P) Modificado Acceso remoto	Fallo de lectura	D) Válido S) Compartido P) Compartido

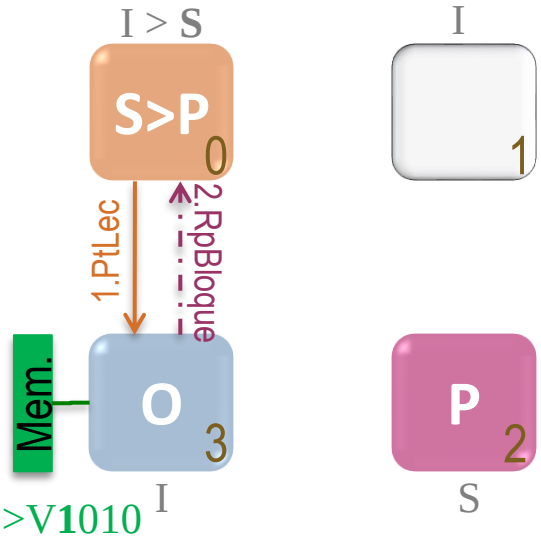


MSI con directorios (sin difusión) III

Estado inicial	Evento	Estado final
D) Válido — V **0** 0 1 0 (C0 C1 C2 C3) S) Inválido P) Compartido Acceso remoto	Fallo de lectura	D) Válido — V **1** 0 1 0 (C0 C1 C2 C3) S) Compartido P) Compartido





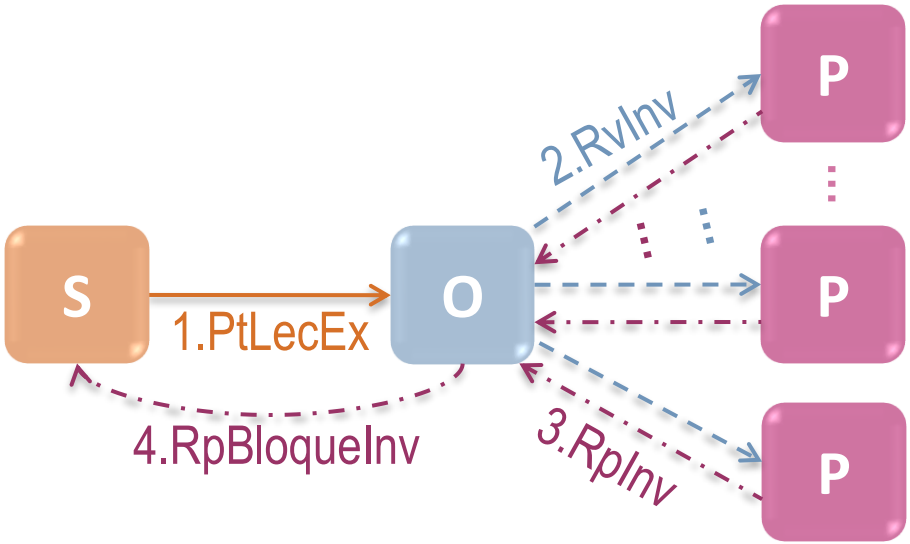


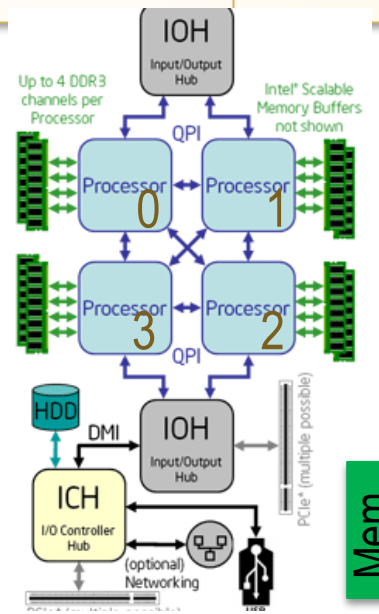
MSI con directorios (sin difusión) IV

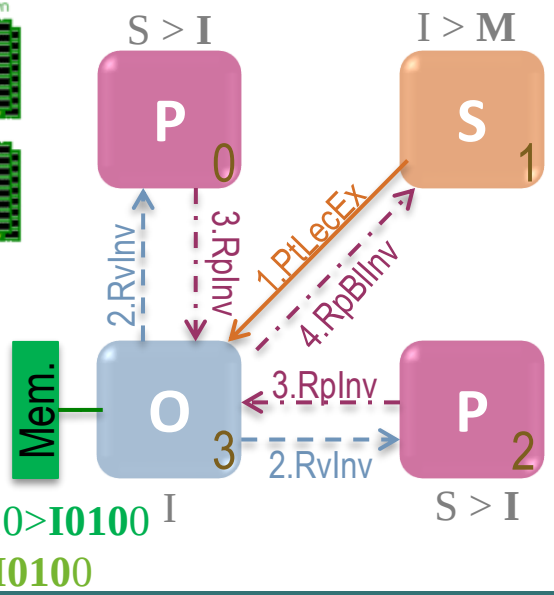
Estado inicial	Evento	Estado final
D) Válido S) Inválido P) Compartido Acceso remoto	Fallo de escritura	D) Inválido S) Modificado P) Inválido

	C0	C1	C2	C3
V	1	**0**	1	0

	C0	C1	C2	C3
I	0	**1**	0	0

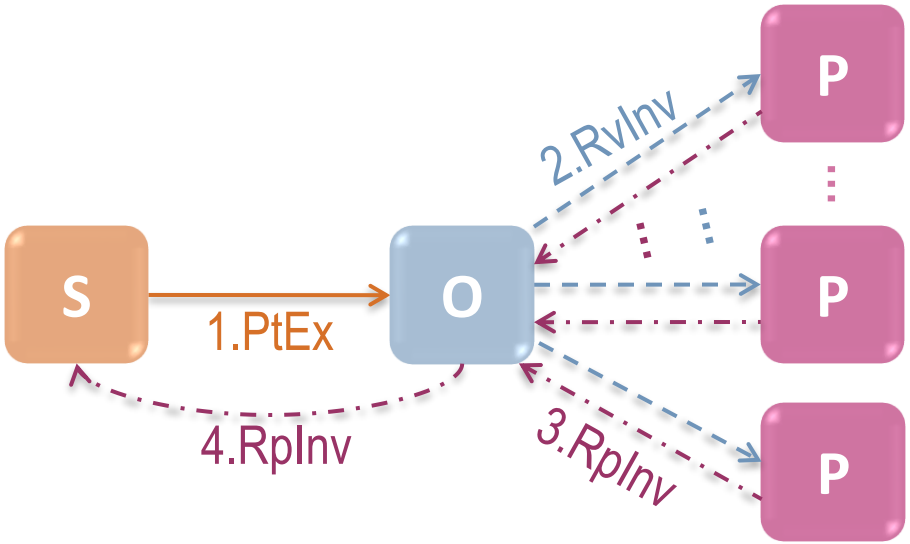


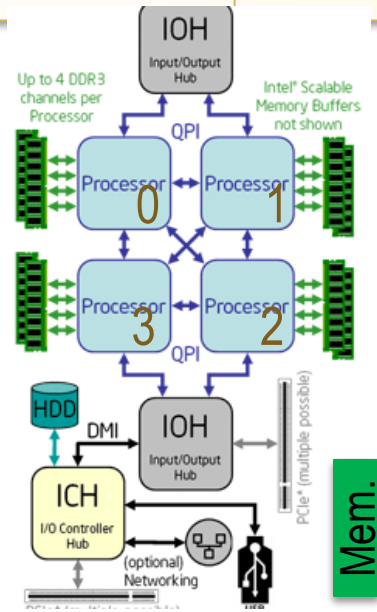


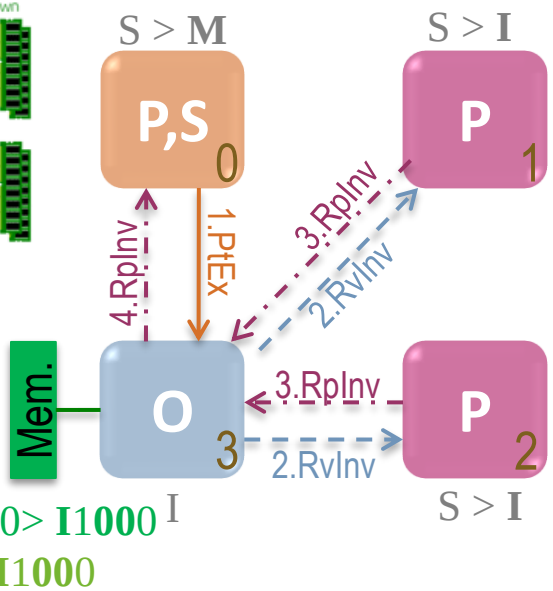


MSI con directorios (sin difusión) V

Estado inicial	Evento	Estado final
D) Válido — V | **1** | 1 | 1 | 0 (C0 C1 C2 C3) S) Compartido P) Compartido Acceso remoto	Fallo de escritura	D) Inválido — I | 1 | **0** | **0** | 0 S) Modificado P) Inválido

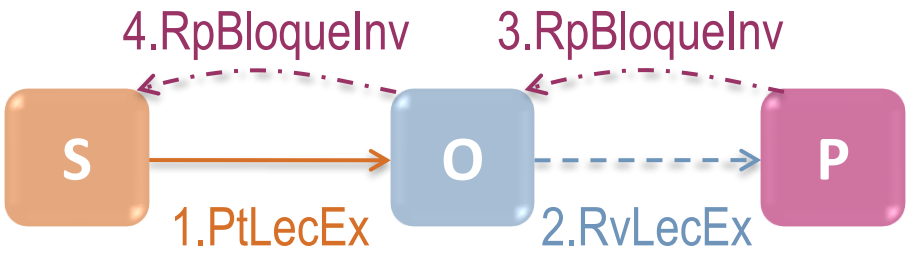


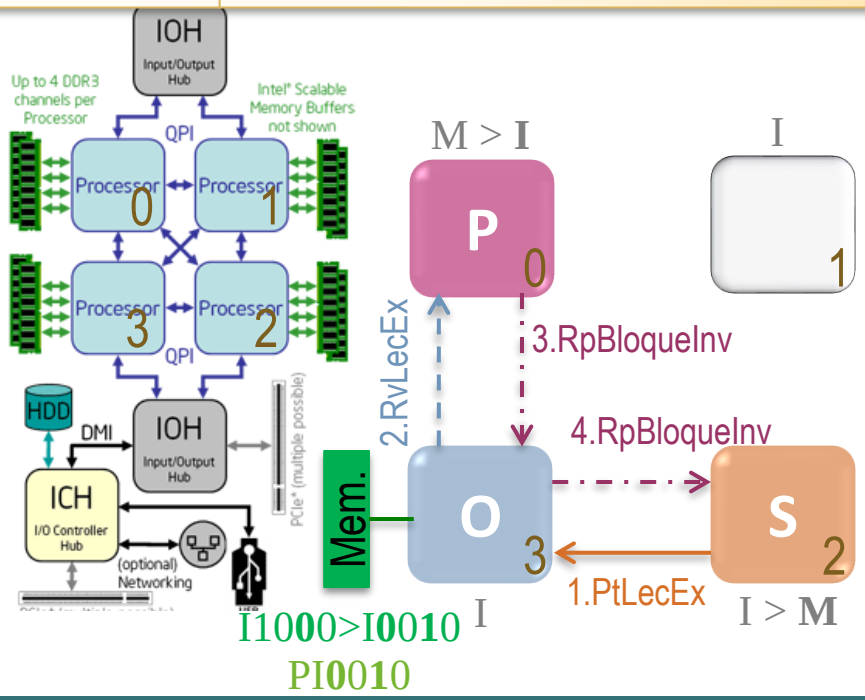




MSI con directorios (sin difusión) VI

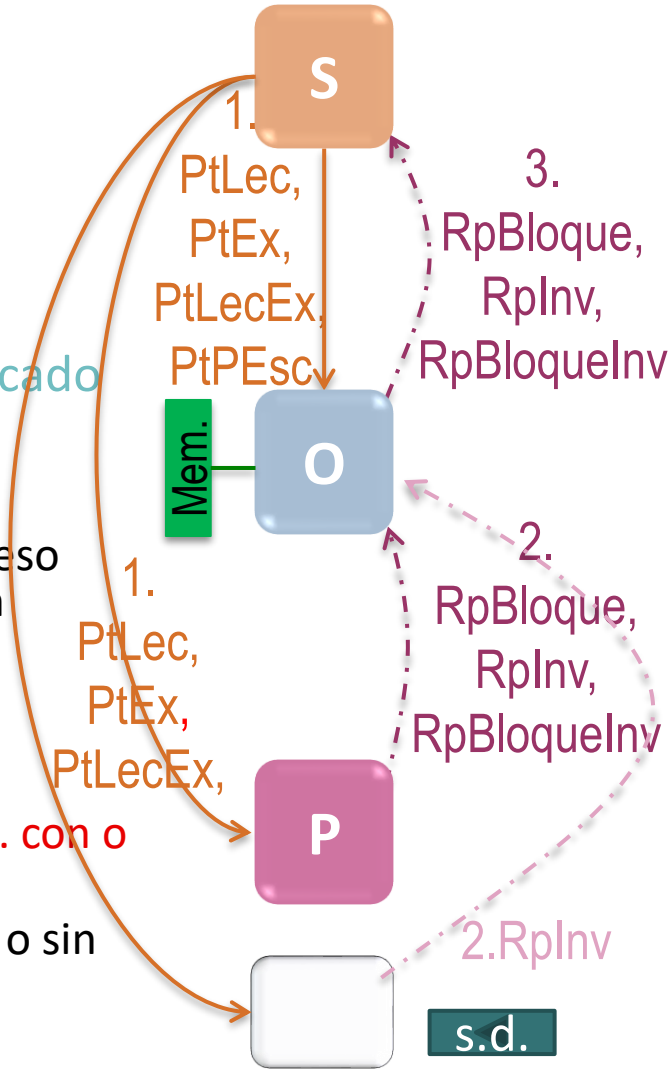
Estado inicial						Evento	Estado final					
		C0	C1	C2	C3							
D) Inválido	I	1	0	**0**	0	Fallo de escritura	D) Inválido	I	**0**	0	**1**	0
S) Inválido							S) Modificado					
P) Modificado							P) Inválido					
Acceso remoto												





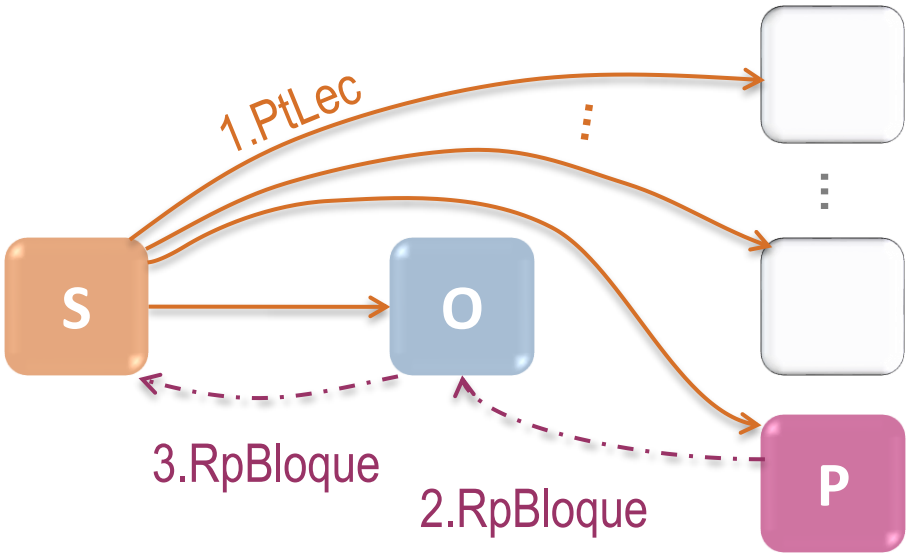
MSI con directorios (con difusión) I

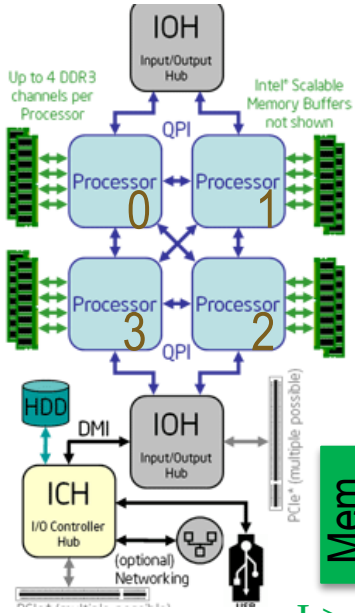
- Estados de un bloque en caché:
  - Modificado (M), Compartido (C), Inválido (I)
- Estados de un bloque en MP:
  - Válido e inválido
- Transferencias (tipos de paquetes) :
  - Tipos de nodos: solicitante (S), origen (O), modificado (M), propietario (P) y compartidor (C)
  - Difusión de **petición** del nodo **S** a
    - **O** y **P**: lectura de un bloque (**PtLec**), lectura con acceso exclusivo (**PtLecEx**), petición de acceso exclusivo sin lectura (**PtEx**)
    - **O**: posescritura (**PtPEsc**)
  - **Respuesta** de
    - nodo **P** a **O**: respuesta con bloque (**RpBloque**), resp. con o sin bloque confirmando inv. (**RpInv**, **RpBloqueInv**)
    - nodo **O** a **S**: resp. con bloque (**RpBloque**), resp. con o sin bloque confirmando fin inv. (**RpInv**, **RpBloqueInv**)

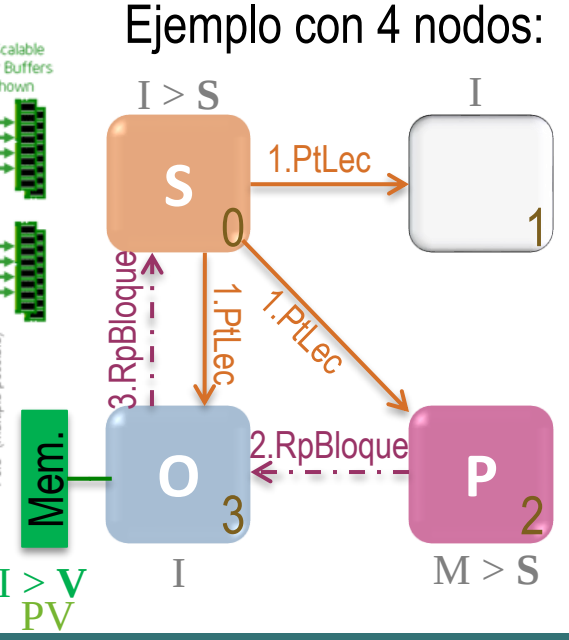


MSI con directorios (con difusión) II

Estado inicial	Evento	Estado final
D) Inválido **I** S) Inválido P) Modificado Acceso remoto	Fallo de lectura	D) Válido **V** S) Compartido P) Compartido



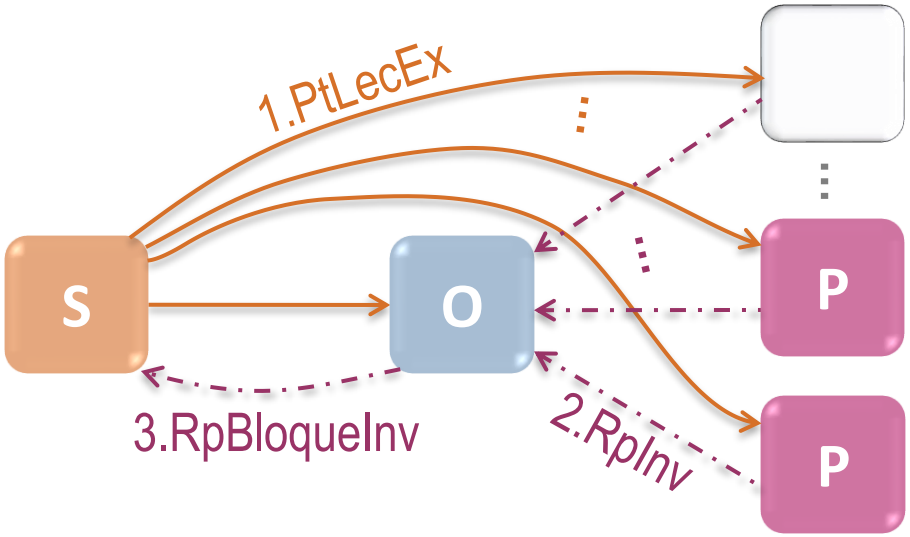


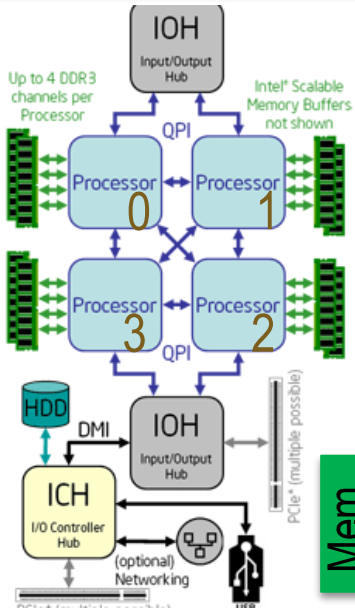


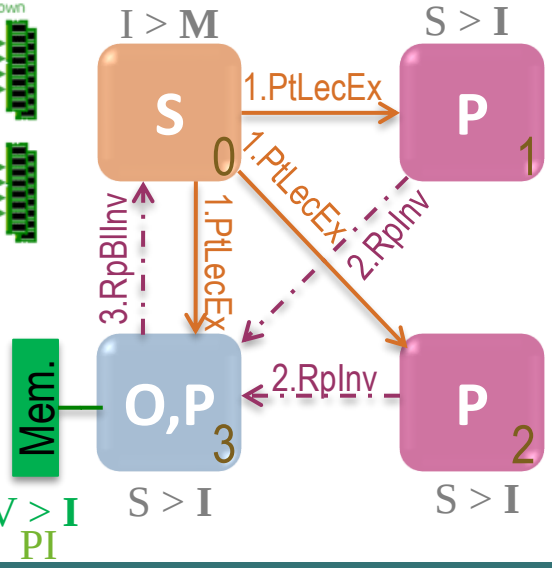
s.d.

MSI con directorios (con difusión) III

Estado inicial	Evento	Estado final
D) Válido V S) Inválido P) Compartido Acceso remoto	Fallo de escritura (Procesador escribe)	D) Inválido I S) Modificado P) Inválido



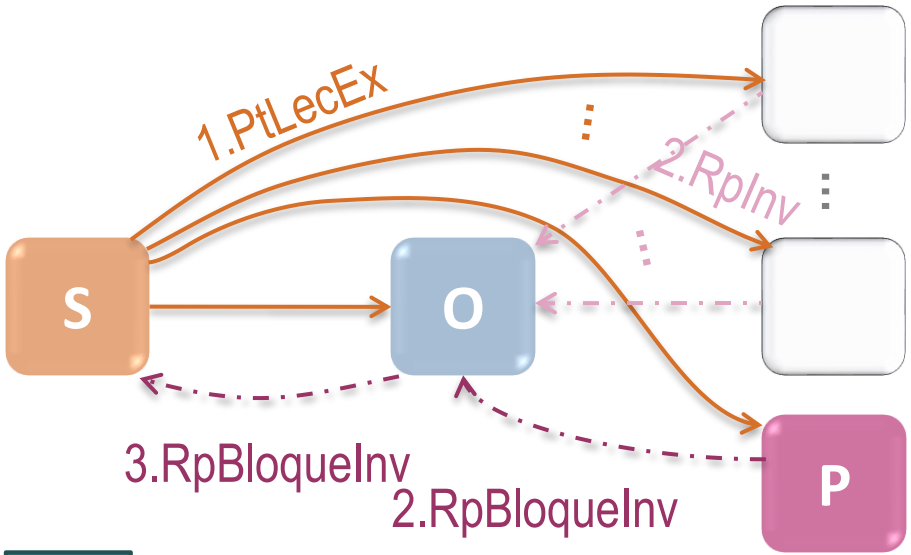


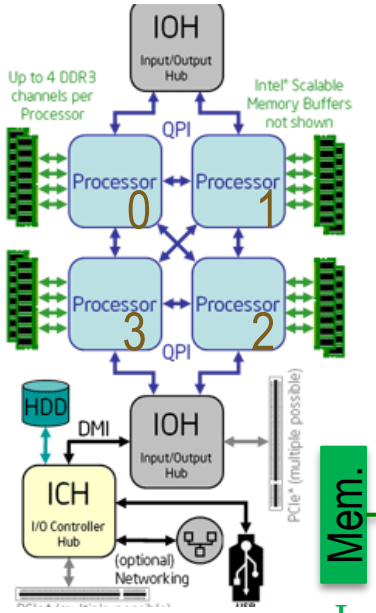


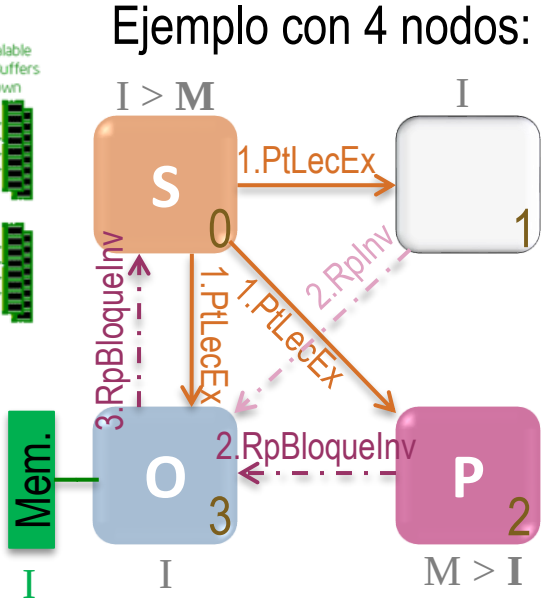
s.d.

MSI con directorios (con difusión) IV

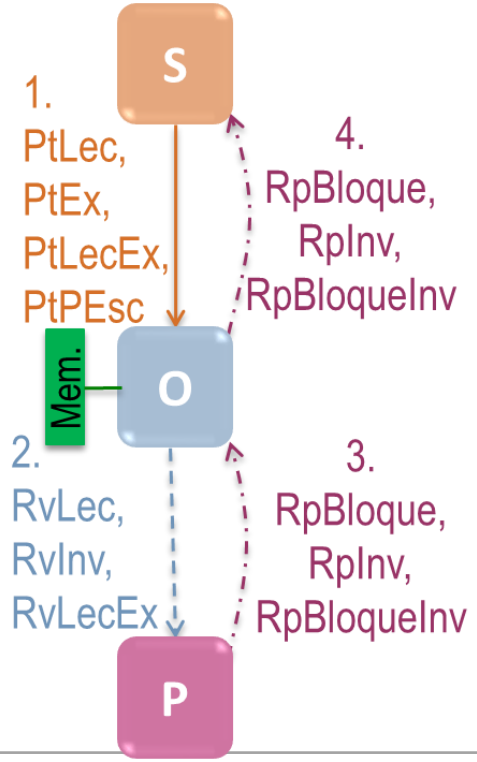
Estado inicial	Evento	Estado final
D) Inválido I S) Inválido P) Modificado Acceso remoto	Fallo de escritura	D) Inválido I S) Modificado P) Inválido

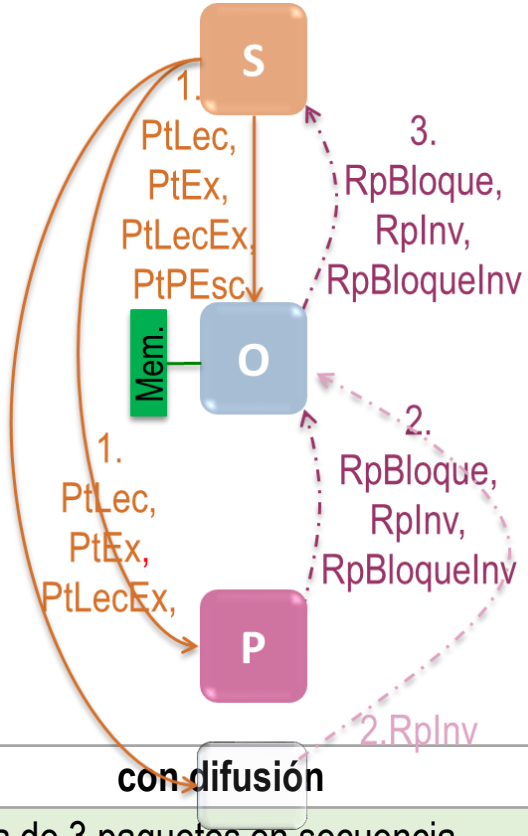






MSI con directorios (con difusión vs sin difusión)





MSI NUMA	sin difusión	con difusión
Latencia en el peor caso	latencia de 4 paquetes en secuencia	Latencia de 3 paquetes en secuencia
Latencia en el mejor caso	Si coinciden Origen y Solicitante	Si coinciden Origen y Solicitante
Latencia (tiempo de respuesta) media	Podría ser peor (compartición de datos modificables)	
Ancho de Banda (productividad) medio		Podría ser peor (poca compartición de datos modificables)

Para ampliar …

- Webs
  - An Introduction to the Intel® QuickPath Interconnect, http://www.intel.com/content/www/us/en/io/quickpath-technology/quick-path-interconnect-introduction-paper.html
  - Animaciones de protocolos de coherencia de cachés http://lorca.act.uji.es/projects/ccp/ (necesita un plugin que ahora no se instala por defecto en los navegadores)

2º curso / 2º cuatr.

Grados en
Ing. Informática

Arquitectura de Computadores
Tema 3

Lección 9. Consistencia del sistema de memoria

Material elaborado por Mancia Anguita
Profesores: Mancia Anguita, Maribel García y Christian Morillas



UNIVERSIDAD
DE GRANADA



ICAR
INGENIERÍA DE COMPUTADORES,
AUTOMÁTICA Y ROBÓTICA

Lecciones

- Lección 7. Arquitecturas TLP
- Lección 8. Coherencia del sistema de memoria
- Lección 9. Consistencia del sistema de memoria
  - Concepto de consistencia de memoria
  - Consistencia secuencial
  - Modelos de consistencia relajados
- Lección 10. Sincronización

Objetivos Lección 9

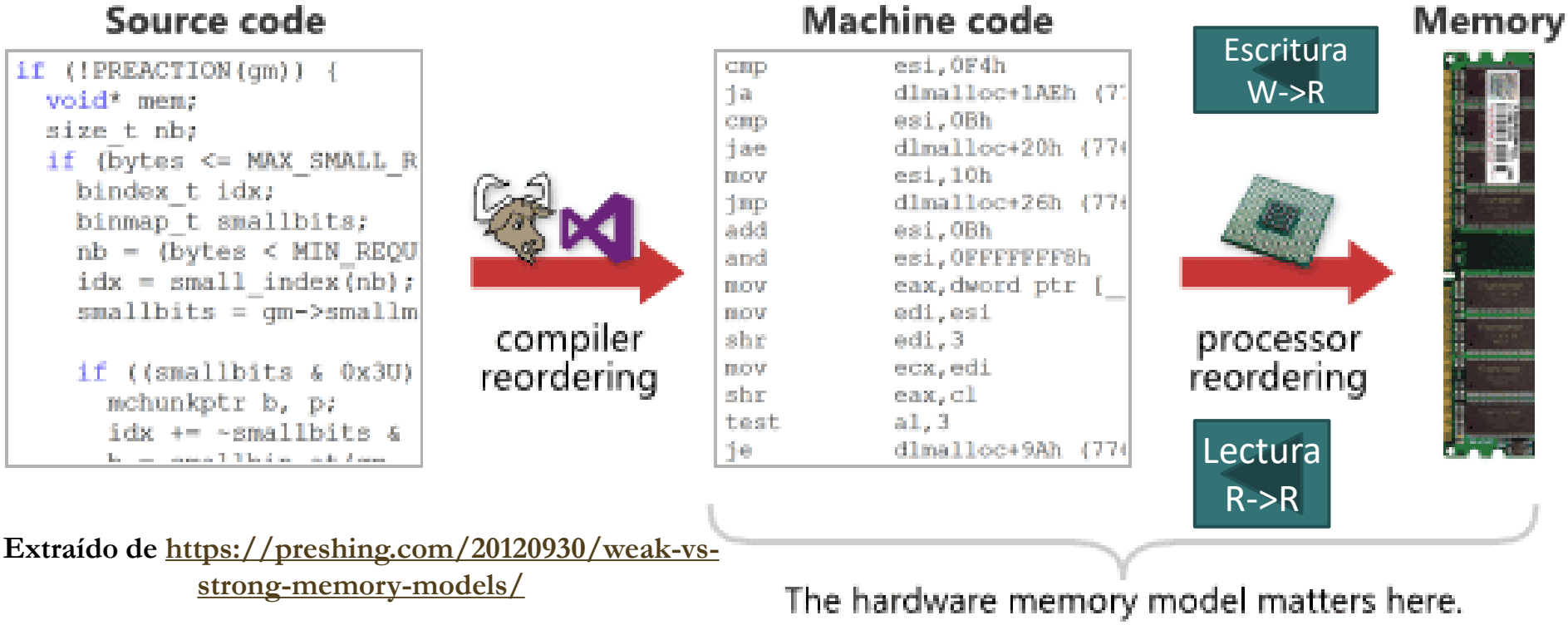
- Explicar el concepto de consistencia.
- Distinguir entre coherencia y consistencia.
- Distinguir entre el modelo de consistencia secuencial y los modelos relajados.
- Distinguir entre los diferentes modelos de consistencia relajados.

Bibliografía Lección 9

- Fundamental
  - Cap.3, Secc. 4. M. Anguita, J. Ortega. *Fundamentos y Problemas de Arquitectura de Computadores.* Librería Fleming/Ed. Avicam, 2016.
  - Secc. 10.2. J. Ortega, M. Anguita, A. Prieto. “Arquitectura de Computadores”. ESII/C.1 ORT arq

Contenido Lección 9

- Concepto de consistencia de memoria
- Consistencia secuencial
- Modelos de consistencia relajados

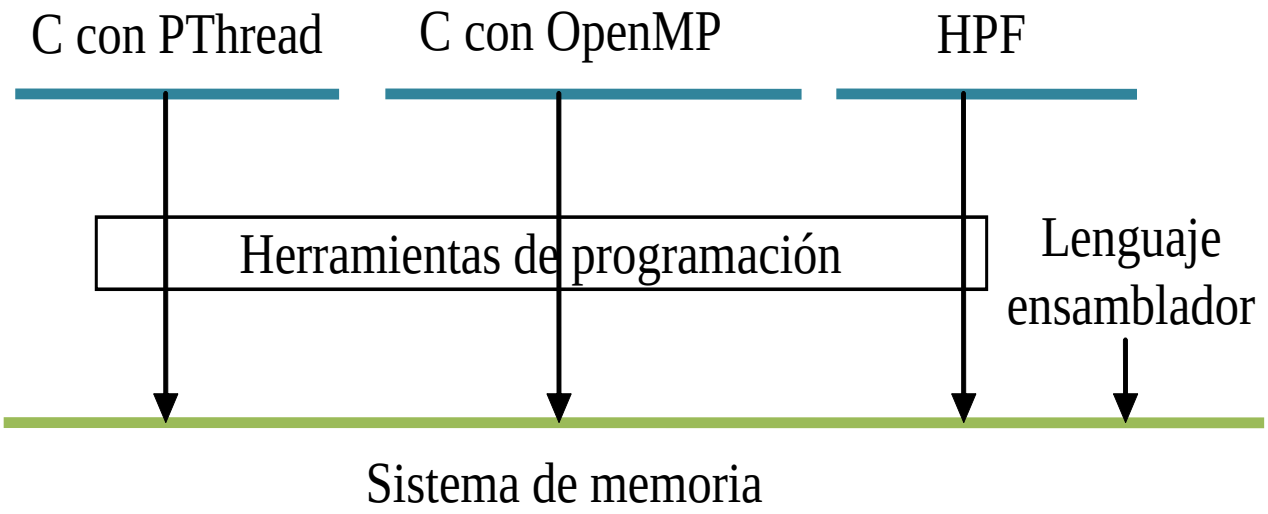


Extraído de https://preshing.com/20120930/weak-vs-strong-memory-models/

Consistencia de memoria

Modelo de consistencia de memoria Software

Modelo de consistencia de memoria Hardware



- Especifica (las restricciones en) el orden en el cual las operaciones de memoria (lectura, escritura) deben ***parecer*** haberse realizado (operaciones a las mismas o distintas direcciones y emitidas por el mismo o distinto flujo de instrucciones/procesador)
- La coherencia sólo abarca operaciones realizadas en una misma dirección

Contenido Lección 9

- Concepto de consistencia de memoria
- Consistencia secuencial
- Modelos de consistencia relajados

Consistencia secuencial (SC)

- SC es el modelo de consistencia que espera el programador de las herramientas de alto nivel
- SC requiere que:
  - Todas las operaciones de un único procesador (flujo de instrucciones) parezcan ejecutarse en el orden descrito por el programa de entrada al procesador (**orden del programa**)
  - Todas las operaciones de memoria parezcan ser ejecutadas una cada vez (**ejecución atómica**) -> serialización global

Inicialmente A=0,k=0	
P1	**P2**
A=1; W(A)	while (k==0) {}; R(k)
k=1; W(k)	copia=A; R(A)

Consistencia Secuencial

//Inicialmente k1=k2=0
P1
k1=1;
if (k2==0) {
Sección crítica
};
		P2
		k2=1;
		if (k1==0) {
		Sección crítica
		};
			1
			¿Qué espera el programador?

P1 A=1;	*//Inicialmente* **P2** if (A==1) B=1;	*//A=B=0, reg1=4* **P3** if (B==1) reg1=A;	2 ¿Qué espera el programador que se almacene en reg1 si llega a ejecutarse reg1=A? Escritura W->R
//Inicialmente A=0,k=0 **P1** A=1; k=1;		*//copia es local* **P2** while (k==0) {}; copia=A;	3 ¿Qué espera el programador que se almacene en copia? sincronización

Órdenes que cumplen consistencia secuencial (orden del programa)

P1 (1) Escribir K1=1

(2) Leer K2 (¿K2==0?)

P2 (a) Escribir K2=1

(b) Leer K1 (¿K1==0?)

1



Orden con Consistencia Secuencial:

(1)(2)(a)(b) (a)(b)(1)(2)

(1)(a)(2)(b) (a)(1)(b)(2)

(1)(a)(b)(2) (a)(1)(2)(b)

Escritura W->R

Escritura W->R

P1 (1) Escribir A=1

(2) Escribir K=1

(a) Leer K (while k==0 {})

P2 (b) Leer A

3



Orden con Consistencia Secuencial:

(1)(a)….(a)(2)(a)(b)

(1)(2)(a)(b)

(a)..(a)(1)(a)..(a)(2)(a)(b)

(a)..(a)(1)(2)(a)(b)

Leer k última iterac. R->R

Órdenes que no cumplen orden del programa

P1 (1) Escribir K1=1

(2) Leer K2 (¿K2==0?)

P2 (a) Escribir K2=1

(b) Leer K1 (¿K1==0?)

1

Orden sin Consistencia Secuencial:

(2)(a)(b)(1) W->R P1 y P2 entran en SC

(b)(1)(2)(a) W->R P1 y P2 entran en SC

(2)(1)(b)(a) W->R P1 entra en SC, P2 no

//Inicialmente A=0,k=0	//copia es local
P1	**P2**
A=1;	while (k=0) {};
k=1;	copia=A;

3

P1 (1) Escribir A=1

(2) Escribir K=1

(a) Leer K (while k==0 {})

P2 (b) Leer A

Orden sin Consistencia Secuencial:

(1)(a)….(a)(b)(2)(a) R->R copia es 1

(a)..(a)(2)(a)(b)(1) W->W copia es 0

(a)..(a)(a)..(a)(b)(1)(2)(a) R->R copia es 0

(a)..(a)(b)(2)(a)(1) R->R W->W copia es 0

Contenido Lección 9

- Concepto de consistencia de memoria
- Consistencia secuencial
- Modelos de consistencia relajados

Modelos de consistencia relajados

- Difieren en cuanto a los requisitos para garantizar SC que relajan (los relajan para incrementar prestaciones):
  - Orden del programa:
    - Hay modelos que permiten que se relaje en el código ejecutado en un procesador el orden entre dos acceso a distintas direcciones (W→R, W→W, R→RW)
  - Atomicidad (orden global):
    - Hay modelos que permiten que un procesador pueda ver el valor escrito por otro antes de que este valor sea visible al resto de los procesadores del sistema
- Los modelos relajados comprenden:
  - Los órdenes de acceso a memoria que no garantiza el sistema de memoria (tanto órdenes de un mismo procesador como atomicidad en las escrituras).
  - Mecanismos que ofrece el hardware para garantizar un orden cuando sea necesario.

Ejemplos de modelos de consistencia hardware relajados

Modelo	Orden del programa relajado W→R	W→W	R→RW	Orden global Lec. anticipada propia	de otro	Instrucciones para garantizar los órdenes relajados por el modelo
Sparc-TSO, x86-TSO	Si			Si		l-m-e (instruc. lectura-modificación-escritura atómica)
Sparc-PSO	Si	Si		Si		l-m-e, `STBAR` (instrucción *STore BARrier*)
Sparc-RMO	Si	Si	Si	Si		`MEMBAR` (instrucción *MEMory BARrier*)
PowerPC	Si	Si	Si	Si	Si	`SYNC`, `ISYNC` (instrucciones *SYNChronization*), LWSYNC
Itanium	Si	Si	Si	Si		`LD.ACQ`, `ST.REL`, `MF` (*ACQuisition LoaD, RELease STore, Memory Fence*),y `cmpxchg8.acq` y otras l-m-e
ARMv7	Si	Si	Si	Si	Si	`DMB` (*Data Memory Barrier*)
ARMv8	Si	Si	Si	Si	Si	`LDA` | `LDAR`, `STL` | `STLR` (*LoaD-Acquire*, *STore-reLease* 32b|64b), `LDAEX` | `LDAXR`, `STLEX` | `STLXR` (*LoaD-Acquire eXclusive*, *Store-reLease eXclusive* 32b|64b), `DMB`,l-m-e (LSE–*Large System Extension*) en ARMv8.1

Sigue la tabla de Adve y Gharachorloo en (biblioteca ugr) http://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=546611&isnumber=11956

¿Qué pasa si tenemos un modelo relajado?

//Inicialmente
P1
k1=1;
if (k2==0) {
Sección crítica
};
		//k1=k2=0
		P2
		k2=1;
		if (k1==0) {
		Sección crítica
		};
			1
			NO se comporta como SC los que relajan el orden **W→R**

P1 A=1;	*//Inicialmente* **P2** if (A==1) B=1;	*//A=B=0, reg1=4* **P3** if (B==1) reg1=A;	2 NO se comporta como SC los que no garantizan **atomicidad**

//Inicialmente A=0,k=0
P1
A=1;
k=1;
		//copia es local
		P2
		while (k==0) {};
		copia=A;
			3
			NO se comporta como SC los que relajan el orden **W→ W o R → R**

Modelo relajado y RAW

//Inicialmente A=0,k=0
P1
A=1;
B=2;
k=1;
	P2
	while (k=0) {};
	B=A;
	printf("%d=%d",B,A);

	- La escritura en B no puede adelantar nunca a la lectura de A porque son dependientes. Código ensamblador para B=A (dependencia **RAW**):
	`movl (%eax),esi ; destino esi`
	`movl esi, (%ebx)`
	
	- La lectura de B no puede adelantar nunca a escritura de B, porque son accesos a la **misma variable (RAW)**.

Para ampliar …

- Artículos en revistas
  - Adve, S.V.; Gharachorloo, K.; , "Shared memory consistency models: a tutorial," *Computer* , vol.29, no.12, pp.66-76, Dec 1996.

2º curso / 2º cuatr.

Grados en
Ing. Informática

Arquitectura de Computadores
Tema 3

Lección 10. Sincronización

Material elaborado por Mancia Anguita
Profesores: Mancia Anguita, Maribel García y Christian Morillas



UNIVERSIDAD
DE GRANADA



ICAR
INGENIERÍA DE COMPUTADORES,
AUTOMÁTICA Y ROBÓTICA

Lecciones

- Lección 7. Arquitecturas TLP
- Lección 8. Coherencia del sistema de memoria
- Lección 9. Consistencia del sistema de memoria
- Lección 10. Sincronización
  - Comunicación en multiprocesadores y necesidad de usar código de sincronización
  - Soporte software y hardware para sincronización
  - Cerrojos
    - Cerrojos simples
    - Cerrojos con etiqueta
  - Barreras
  - Apoyo hardware a primitivas software

Objetivos Lección 10

- Explicar por qué es necesaria la sincronización en multiprocesadores.
- Describir las primitivas para sincronización que ofrece el hardware.
- Implementar cerrojos simples, cerrojos con etiqueta y barreras a partir de instrucciones máquina de sincronización y ordenación de accesos a memoria.

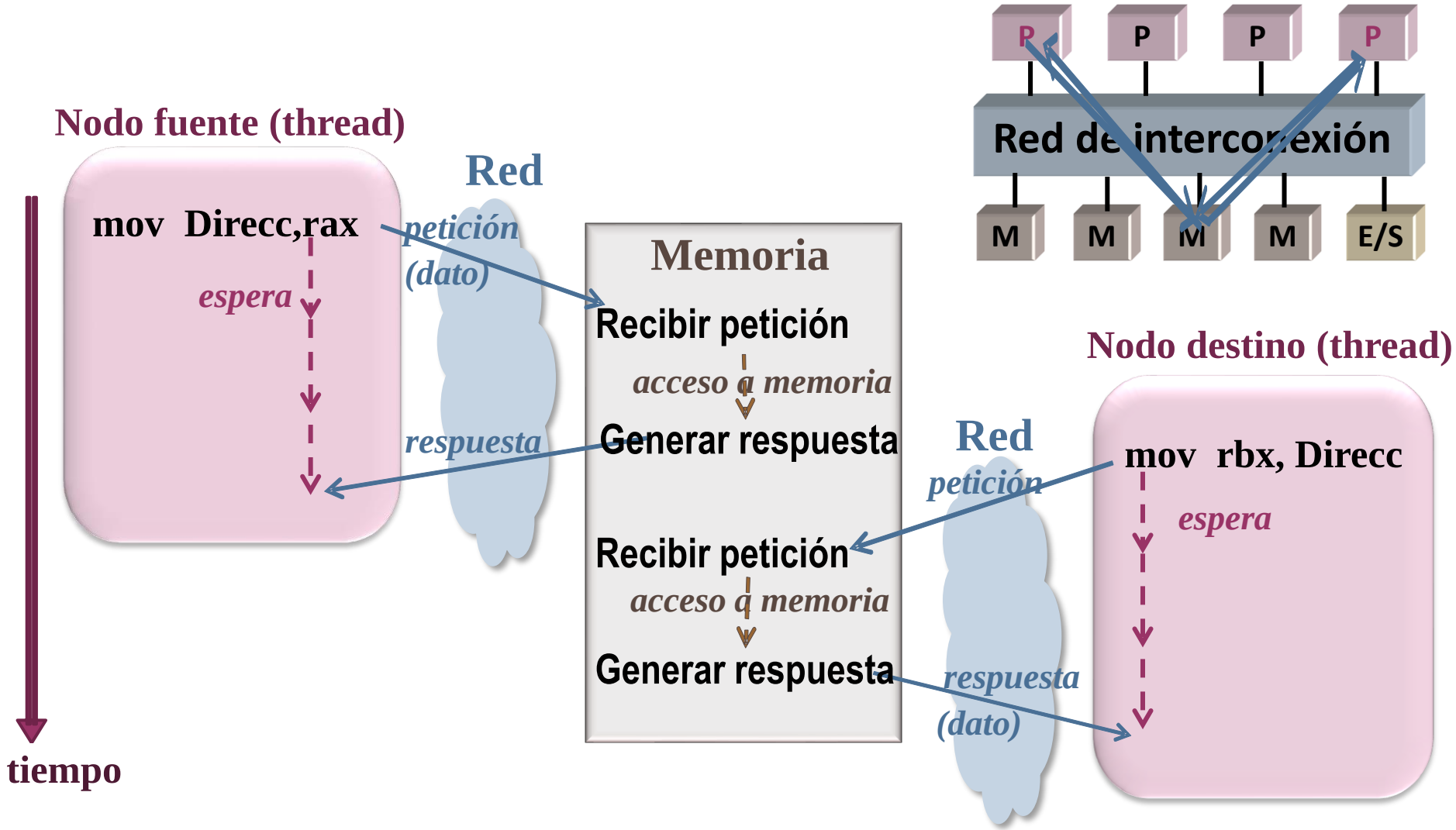
Bibliografía Lección 10

- Fundamental
  - Cap.3, Secc. 5. M. Anguita, J. Ortega. *Fundamentos y Problemas de Arquitectura de Computadores.* Librería Fleming/Ed. Avicam, 2016.
  - Secc. 10.3. J. Ortega, M. Anguita, A. Prieto. “Arquitectura de Computadores”. ESII/C.1 ORT arq

Contenido Lección 10

- Comunicación en multiprocesadores y necesidad de usar código de sincronización
- Soporte software y hardware para sincronización
- Cerrojos
- Barreras
- Apoyo hardware a primitivas software

Comunicación en un multiprocesador



Comunicación uno-a-uno

Secuencial	Paralela	
... **A**=valor; ... copia=**A**; ...	**P1** ... **A**=valor; ...	**P2** ... copia=**A**; ...
... mov **A**,rax ... mov rbx,**A** ...	**P1** ... mov **A**,rax ...	**P2** ... mov rbx,**A** ...

Comunicación uno-a-uno. Necesidad de sincronización

- Se debe garantizar que el proceso que recibe lea la variable compartida cuando el proceso que envía haya escrito en la variable el dato a enviar
- Si se reutiliza la variable para comunicación, se debe garantizar que no se envía un nuevo dato en la variable hasta que no se haya leído el anterior

Paralela (inicialmente K=0)	
P1	**P2**
…	…
A=1;	*while (K==0) { };*
K=1;	copia=A;
…	…

Comunicación colectiva

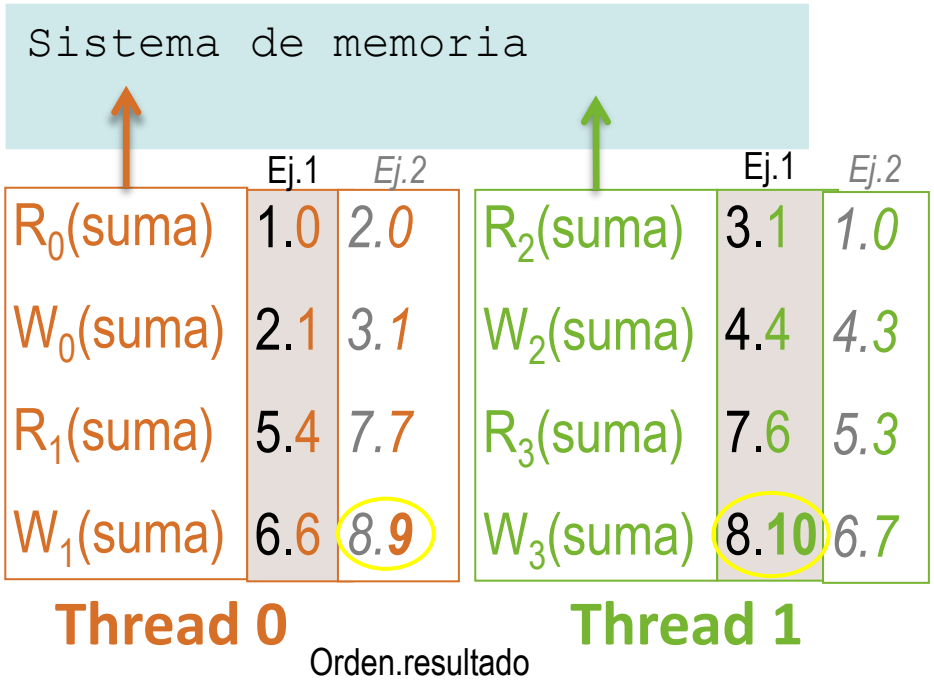
Secuencial	**Paralela** (sum=0)

for (i=0 ; i<n ; i++) {
  sum = sum + a[i];
}
printf(**sum**);
	for (*i*=*ithread* ; *i*<n ; *i*=*i*+nthread) {
	  sump = *sump* + a[*i*];
	}
	sum = **sum** + *sump*; /* SC, sum compart. */
	if (*ithread*==0) printf(**sum**);

Race condition

- Ejemplo de comunicación colectiva: suma de `n` números:
  - La lectura-modificación-escritura de `sum` se debería hacer en exclusión mutua (es una sección crítica) => **cerrojos**
    - Sección crítica: Secuencia de instrucciones con una o varias direcciones compartidas (variables) que se deben acceder en exclusión mutua
  - El proceso 0 no debería imprimir hasta que no hayan acumulado `sump` en `sum` todos los flujos de instrucciones=> **barreras**

Comunicación colectiva en multiprocesadores (carrera)



- Ej. para n=4, el compilador no optimiza
- a={1,2,3,4}
- R_i (suma): Lectura de suma en la iteración i

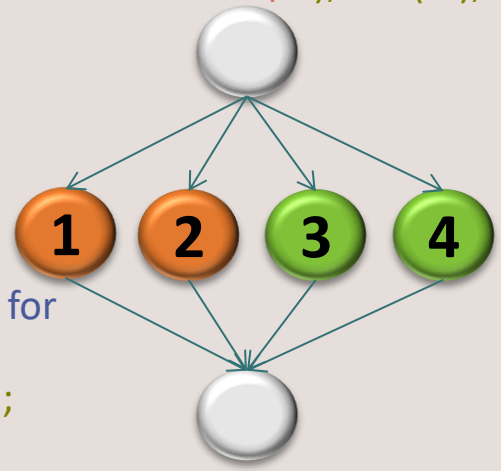
$R_i(suma)$ → $W_i(suma)$

sin exclusión mutua en el acceso a suma

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
main(int argc, char **argv)  {
  int i, n=20, a[n],suma=0;
  if(argc < 2)  {
    fprintf(stderr,"\nFalta iteraciones\n"); exit(-1);
  }
  n = atoi(argv[1]);
  if (n>20) n=20;
  for (i=0; i<n; i++)
    a[i] = i+1;

#pragma omp parallel for
  for (i=0; i<n; i++)
    suma = suma + a[i];

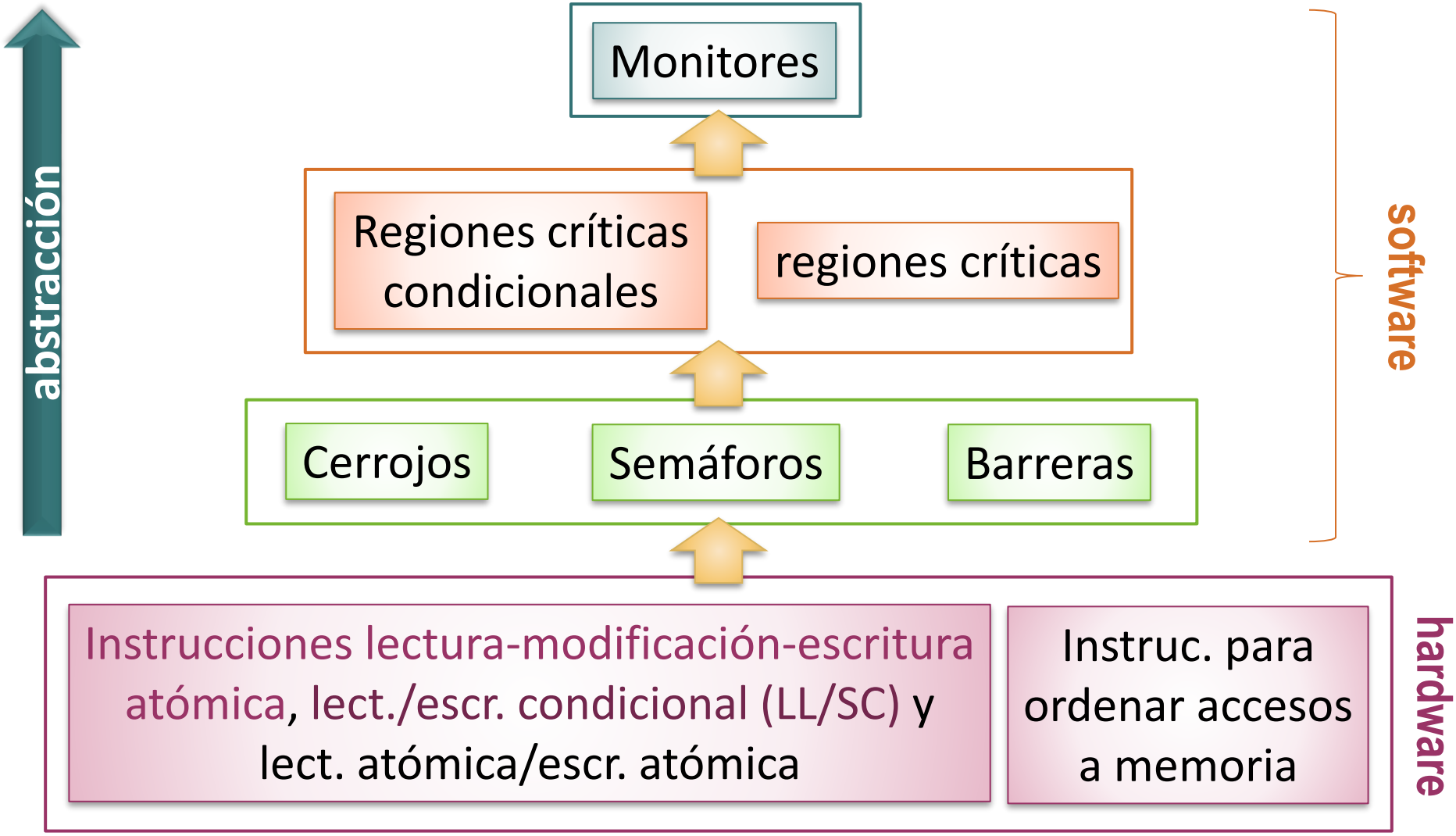
  printf("Fuera de 'parallel' suma=%d\n",suma);
  return(0);
}
```



Contenido Lección 10

- Comunicación en multiprocesadores y necesidad de usar código de sincronización
- Soporte software y hardware para sincronización
- Cerrojos
- Barreras
- Apoyo hardware a primitivas software

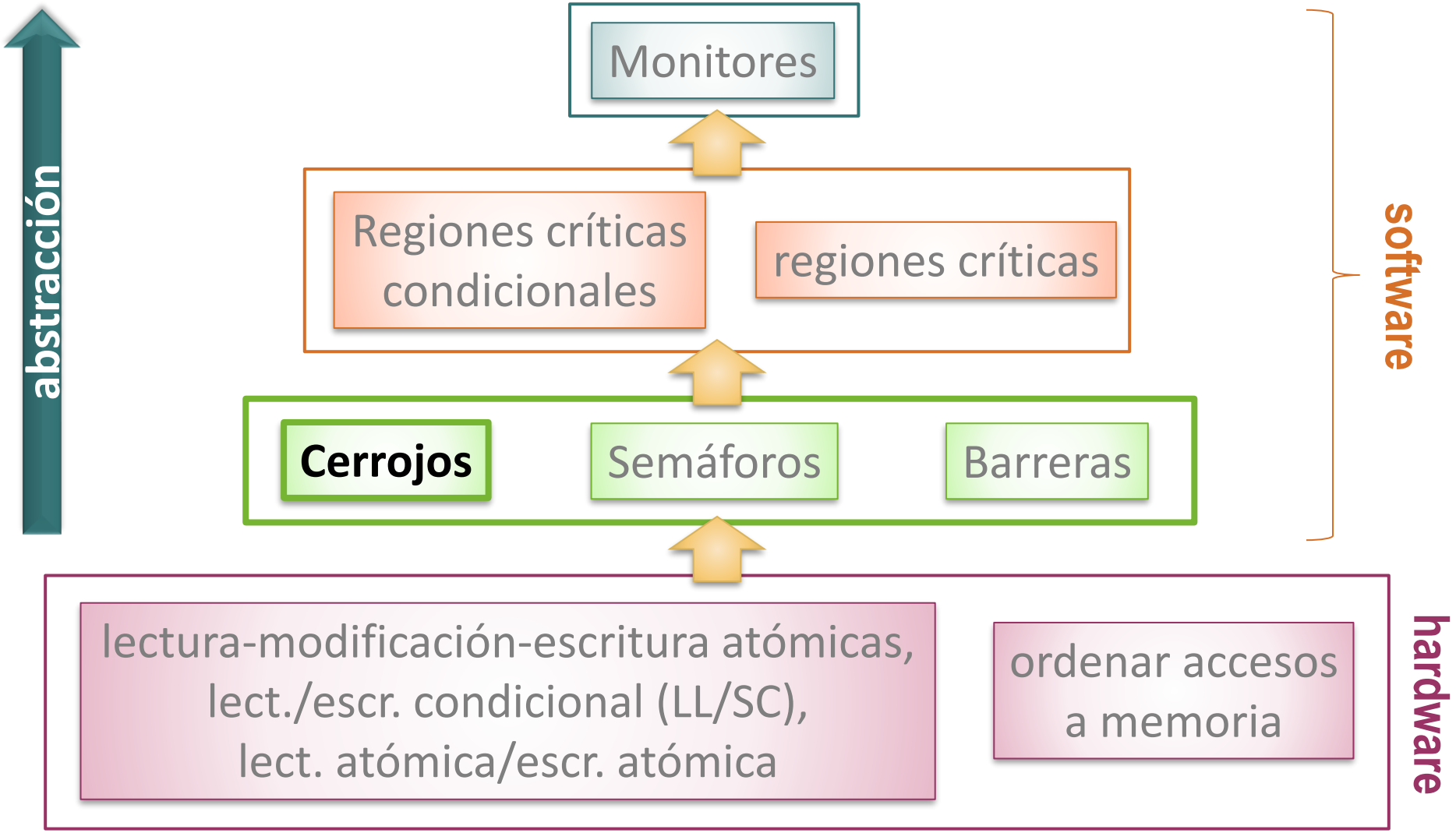
Soporte software y hardware de sincronización



Contenido Lección 10

- Comunicación en multiprocesadores y necesidad de usar código de sincronización
- Soporte software y hardware para sincronización
- Cerrojos
  - Cerrojos simples
  - Cerrojos con etiqueta
- Barreras
- Apoyo hardware a primitivas software

Soporte software y hardware de sincronización



Cerrojos

- Permiten sincronizar mediante dos operaciones:
  - **Cierre** del cerrojo o `lock(k)`: intenta ***adquirir*** el derecho a acceder a una sección crítica (*cerrando* o *adquiriendo* el cerrojo `k`).
    - Si varios threads intentan la **adquisición** (cierre) a la vez, sólo uno de ellos lo debe conseguir, el resto debe pasar a una *etapa de **espera***.
    - Todos los threads que ejecuten `lock()` con el cerrojo cerrado deben quedar ***esperando***.
  - **Apertura** del cerrojo o `unlock(k)`: ***libera*** a uno de los threads que esperan el acceso a una sección crítica (éste *adquiere* el cerrojo).
    - Si no hay threads en ***espera***, permitirá que el siguiente thread que ejecute la función `lock()` adquiera el cerrojo `k` sin espera

Cerrojos en ejemplo suma

Secuencial	Paralela

for (i=0 ; i<n ; i++) {
  sum = sum + a[i];
}
	for (*i*=*ithread* ; *i*<n ; *i*=*i*+nthread) {
	  sump = *sump* + a[*i*];
	}
	lock(k); //código de adquisición
	sum = **sum** + *sump*; /* SC, sum compart. */
	unlock(k); //código de liberación

- Alternativas para implementar la espera:
  - Espera ocupada.
  - Suspensión del proceso/thread, éste queda esperando en una cola, el procesador conmuta a otro proceso/thread.

Componentes en un código para sincronización

- Método de **adquisición**
  - Método por el que un thread trata de adquirir el derecho a pasar a utilizar unas direcciones compartidas. Ej.:
    - Utilizando lectura-modificación-escritura atómicas: Intel x86, Intel Itanium, Sun Sparc, ARMv6, ARMv8.1
    - Utilizando LL/SC (*Load Linked –Load Reserved / Store Conditional*): IBM Power/PowerPC, ARMv7, ARMv8
- Método de **espera**
  - Método por el que un thread espera a adquirir el derecho a pasar a utilizar unas direcciones compartidas:
    - Espera ocupada (*busy-waiting*)
    - Bloqueo
- Método de **liberación**
  - Método utilizado por un thread para liberar a uno (cerrojo) o varios (barrera) threads en espera

Cerrojo Simple I

- Se implementa con una variable compartida `k` que toma dos valores: abierto (0), cerrado (1)
- **Apertura** del cerrojo, `unlock(k)`: abre el cerrojo escribiendo un 0 (*operación **indivisible***)
- **Cierre** del cerrojo, `lock(k)`: Lee el cerrojo y lo cierra escribiendo un 1.
  - Resultado de la lectura:
    - si el cerrojo **estaba cerrado** el thread espera hasta que otro thread ejecute `unlock(k)`,
    - si **estaba abierto** *adquiere* el derecho a pasar a la sección crítica.
  - `leer-asignar_1-escribir` en el cerrojo debe ser ***indivisible*** (***atómica***)

Cerrojo Simple II

- Se debe añadir lo necesario para garantizar el acceso en exclusión mutua a k y el orden imprescindible en los accesos a memoria

ARMv7

lock (k)

```
lock(k) {
   while (leer-asignar_1-escribir(k) == 1) {} ;
} /* k compartida */
```

unlock (k)

```
unlock(k) {
   k = 0 ;
} /* k compartida */
```

Cerrojos en OpenMP

Descripción	Función de la biblioteca OpenMP
Iniciar (estado `unlock`)	`omp_init_lock(&k)`
Destruir un cerrojo	`omp_destroy_lock(&k)`
Cerrar el cerrojo `lock(k)`	`omp_set_lock(&k)`
Abrir el cerrojo `unlock(k)`	`omp_unset_lock(&k)`
Cierre del cerrojo pero sin bloqueo (devuelve 1 si estaba cerrado y 0 si está abierto)	`omp_test_lock(&k)`

Cerrojos con etiqueta (*ticket spinlock*)



- Fijan un orden FIFO en la adquisición del cerrojo (se debe añadir lo necesario para garantizar el acceso en exclusión mutua al contador de adquisición y el orden imprescindible en los accesos a memoria):

lock (contadores)

```
contador_local_adq = contadores.adq;
contadores.adq = (contadores.adq + 1)  mod  max_flujos;
while (contador_local_adq != contadores.lib) {};
```



unlock (contadores)

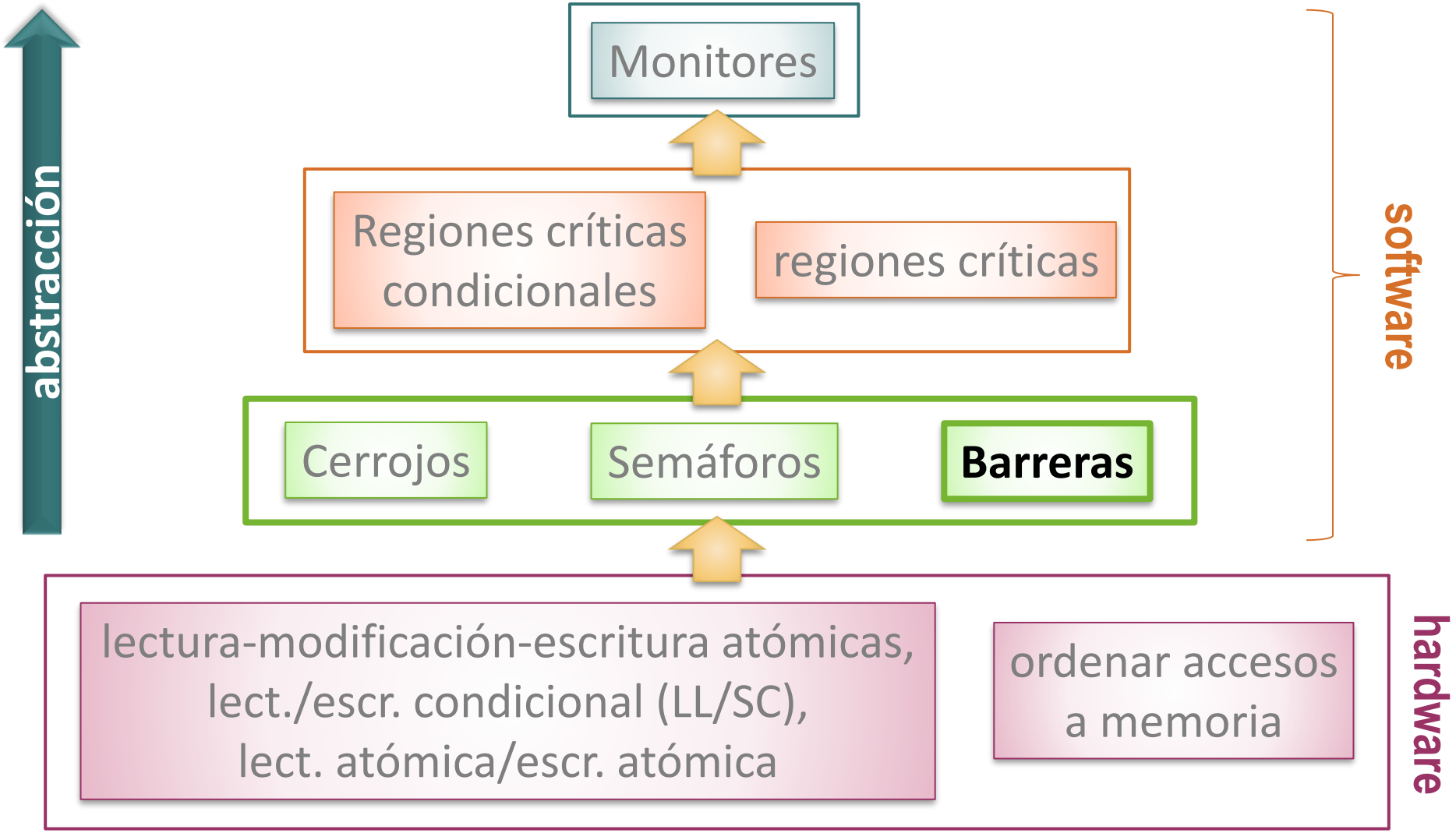
```
contadores.lib = (contadores.lib + 1)  mod  max_flujos;
```



Contenido Lección 10

- Comunicación en multiprocesadores y necesidad de usar código de sincronización
- Soporte software y hardware para sincronización
- Cerrojos
- Barreras
- Apoyo hardware a primitivas software

Soporte software y hardware de sincronización



Barreras

Thread 0

```
main (){
  ...
  Barrera(g,4)
  ...

}
```

Thread 1

```
main (){
  ...
  Barrera(g,4)
  ...

}
```

Thread 2

```
main (){
  ...
  Barrera(g,4)
  ...

}
```

Thread 3

```
main (){
  ...
  Barrera(g,4)
  ...

}
```

```
Barrera(id, num_threads) {
  if (bar[id].cont==0) bar[id].bandera=0;
  cont_local = ++bar[id].cont;
  if (cont_local ==num_threads)  {
     bar[id].cont=0;
     bar[id].bandera=1;
  }
  else espera mientras bar[id].bandera=0;
}
```

-Acceso **Ex. Mutua**.

- Implementar **espera**. Si *espera ocupada*:

while (bar[id].bandera==0) {};

Barreras sin problema de reutilización

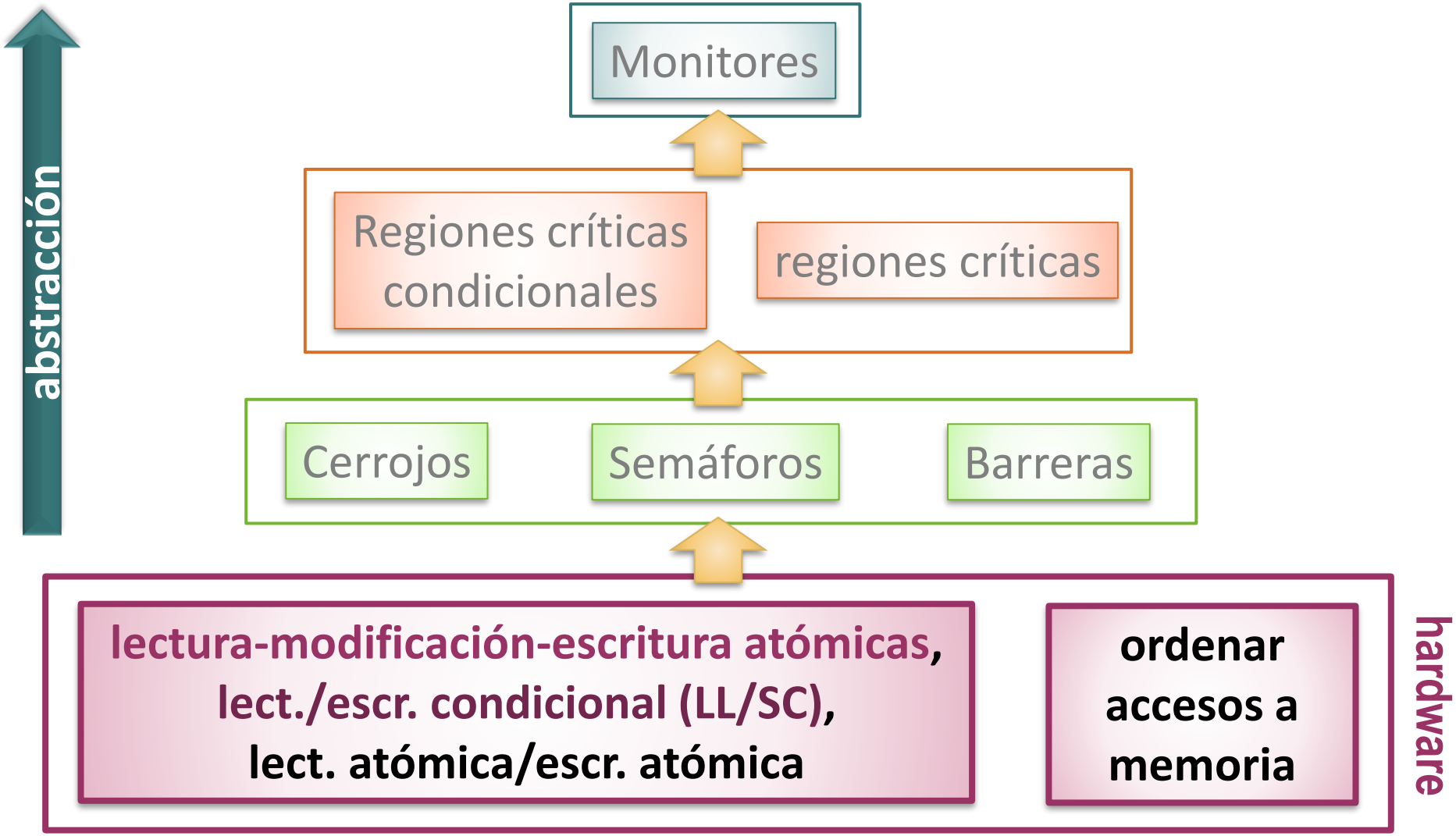
Barrera *sense-reversing*

```
Barrera(id, num_procesos) {
  bandera_local = !(bandera_local)   //se complementa bandera local
  lock(bar[id].cerrojo);
      cont_local = ++bar[id].cont       //cont_local es privada
  unlock(bar[id].cerrojo);
  if (cont_local == num_procesos)  {
     bar[id].cont = 0;                    //se hace 0 el cont. de la barrera
     bar[id].bandera = bandera_local; //para liberar thread en espera
  }
  else  while (bar[id].bandera != bandera_local) {};   //espera ocupada
}
```

Contenido Lección 10

- Comunicación en multiprocesadores y necesidad de usar código de sincronización
- Soporte software y hardware para sincronización
- Cerrojos
- Barreras
- Apoyo hardware a primitivas software
  - Instrucciones de lectura-modificación-escritura atómicas
  - Instrucciones LL/SC (*Load Linked / Store Conditional*)

Soporte software y hardware de sincronización



Instrucciones de lectura-modificación-escritura atómicas

Test&Set (x)

```
Test&Set (x) {
    temp = x ;
    x = 1 ;
    return (temp) ;
}
/* x compartida */
```

x86

```
mov   reg,1
xchg  reg,mem
reg ↔ mem
```

Fetch&Oper(x,a)

```
Fetch&Add(x,a) {
    temp = x ;
    x = x + a ;
    return (temp);
}/* x compartida,
a local */
```

x86

```
lock xadd  reg,mem
reg ← mem |
mem ← reg+mem
```

Compare&Swap(a,b,x)

```
Compare&Swap(a,b,x){
    if (a==x) {
        temp=x ;
        x=b; b=temp ; }
}/* x compartida,
   a y b  locales */
```

x86

```
lock cmpxchg mem,reg
if eax=mem
then mem ← reg
else eax ← mem
```

Cerrojos simples con Test&Set y Fetch&Or

escritura

con Test&Set (x)

```
lock(k) {
   while (test&set(k)==1) {} ;
}
/* k compartida */
```

x86

```
lock:     mov   eax,1
repetir:  xchg  eax,k
          cmp   eax,1
          jz    repetir
```

con Fetch&Oper(x,a)

```
lock(k) {
  while (fetch&or(k,1)==1)  {};
}
/* k compartida */
```

true (1, cerrado)
false (0, abierto)

Cerrojo simple

Cerrojos simples con Compare&Swap

con Compare&Swap(a,b,x)

```
lock(k) {
   b=1
   do
      compare&swap(0,b,k) ;
   while (b==1);
}
/* k compartida, b local */
```



```
compare&swap(0,b,k){
     if (0==k) { b=k | k=b; }
}
```

Cerrojo simple en Itanium (consistencia de liberación) con Compare&Swap

```
lock:                                   //lock(M[lock])
   mov  ar.ccv = 0                      // cmpxchg compara con ar.ccv
                                        // que es un registro de propósito específico
   mov   r2 = 1                         // cmpxchg utilizará r2 para poner el cerrojo a 1
spin:                                   // se implementa espera ocupada
   ld8   r1 = [lock]  ;;                // carga el valor actual del cerrojo en r1
   cmp.eq   p1,p0 = r1, r2;             // si r1=r2 entonces cerrojo está a 1 y se hace p1=1
   (p1) br.cond.spnt   spin ;;          // si p1=1 se repite el ciclo; spnt indica que se
                                        // usa una predicción estática para el salto de "no tomar"
   cmpxchg8.acq   r1 = [lock], r2 ;; //intento de adquisición escribiendo 1
                                        // IF [lock]=ar.ccv THEN [lock]←r2; siempre  r1←[lock]
   cmp.eq   p1, p0 = r1, r2             // si r1!=r2 (r1=0) => cer. era 0 y se hace p1=0
   (p1) br.cond.spnt   spin ;;          // si p1=1 se ejecuta el salto
```

Compare&Swap

Consistencia

```
unlock:                                 //unlock(M[lock])
        st8.rel  [lock] = r0 ;; //liberar asignando un 0, en Itanium r0 siempre es 0
```

Cerrojo simple en PowerPC(consistencia débil) con LL/SC implementando Test&Set

Test&Set

```
lock:                          #lock(M[r3])
        li      r4,1           #para cerrar el cerrojo
bucle:  lwarx   r5,0,r3        #carga y reserva: r5←M[r3]
        cmpwi   r5,0           #si está cerrado (a 1)
        bne-    bucle          #esperar en el bucle, en caso contrario
        stwcx.  r4,0,r3        #poner a 1 (r4=1): M[r3] ← r4
        bne-    bucle          #el thread repite si ha perdido la reserva
        isync                  #accede a datos compartidos cuando sale del bucle
```

Consistencia

```
unlock:                        # unlock(M[r3])
        sync                   #espera hasta que terminen los accesos anteriores
        li      r1,0
        stw     r1,0(r3)       #abre el cerrojo
```

Cerrojo simple en ARMv7 (consistencia débil) con LL/SC implementando Test&Set

LL/SC

Test&Set

```
lock:                          #lock(M[r1])
        mov     r0,#1          #Para posteriormente cerrar el cerrojo asigna a r0 un 1
bucle:  ldrex   r5,[r1]        #Lee cerrojo
        cmp     r5,#0          #Comprueba si el cerrojo es 0 (abierto)
        strexeq r5,r0,[r1]     #Si el cerrojo está abierto intenta escribir (un 1)
        cmpeq   r5,#0          #Comprueba si  ha tenido éxito la escritura
        bne     bucle          #Vuelve a intentarlo si no ha tenido éxito
        dmb                    #Para asegurar que se ha adquirido el cerrojo antes de …
                               #realizar los accesos a memoria que hay después del lock
```

Consistencia

```
unlock:                        # unlock(M[r1])
        dmb                    #Espera a que terminen los accesos anteriores …
        mov     r0,#0          #antes de abrir el cerrojo
        str     r0,[r1]        #Abre el cerrojo
```

Cerrojo simple en ARMv8 (consistencia liberación) con LL/SC para Test&Set

Test&Set

```
lock:                          #lock(M[r1])
        mov     r0,#1          #Para posteriormente cerrar el cerrojo asigna a r0 un 1
bucle:  ldaex   r5,[r1]        #Lee cerrojo con ordenación de adquisición
        cmp     r5,#0          #Comprueba si el cerrojo es 0 (abierto)
        strexeq r5,r0,[r1]     #Si el cerrojo está abierto intenta escribir (un 1)
        cmpeq   r5,#0          #Comprueba si  ha tenido éxito la escritura
        bne     bucle          #Vuelve a intentarlo si no ha tenido éxito
```

```
unlock:                        # unlock(M[r1])
        mov     r0,#0
        stl     r0,[r1]        #Abre el cerrojo usando almacenamiento con liberación
```

Algoritmos eficientes con primitivas hardware

Suma con fetch&add

```
for (i=ithread; i<n; i=i+nthread)
    fetch&add(sum,a[i]);

/* sum variable compartida */
```

Suma con fetch&add

```
for (i=ithread; i<n; i=i+nthread)
        sump = sump + a[i];

fetch&add(sum,sump);
/* sum variable compartida */
```



Suma con compare&swap

```
for (i=ithread; i<n; i=i+nthread)
        sump = sump + a[i];

 do
      a = sum;
      b = a + sump;
      compare&swap(a,b,sum);
  while (a!=b);

/* sum variable compartida */
```

Para ampliar …

- Webs
  - Implementación en el kernel de linux de cerrojos con etiqueta http://lxr.free-electrons.com/source/arch/x86/include/asm/spinlock.h
- Artículos en revistas
  - Graunke, G.; Thakkar, S.; , "Synchronization algorithms for shared-memory multiprocessors," *Computer* , vol.23, no.6, pp.60-69, Jun 1990. Disponible en (biblioteca ugr): http://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=55501&isnumber=2005